

CLAUDE CODE SESSION TRANSCRIPT

Galveston Sanborn mosaics

Full-city Sanborn fire-insurance mosaics of Galveston, Texas, 1899 and 1912

Session	20717cf2-d758-5a36-9da9-a56890968b0c
Period	29 Aug 2026, 04:14 UTC - 01 Sep 2026, 17:11 UTC
Messages from the user	38
Replies	342
Tool calls	640

What this contains. Every message the user sent and every reply, in order. Tool calls are listed one line each, by name and argument. What is *not* here: the output those calls returned, which runs to 37 MB and is machine traffic rather than conversation. Messages generated by the harness rather than typed by the user — hook feedback, background-task notifications, slash commands — are kept in place and labelled as such.

2026-08-29

USER 29 Aug 2026, 04:14 UTC

```
@"/root/.claude/uploads/20717cf2-d758-5a36-9da9-a56890968b0c/53c107b9-
claudecodepromptgalvestonfullcity.md" @"/root/.claude/uploads/20717cf2-d758-5a36-
9da9-a56890968b0c/7d50eb4d-
Galveston_1899_Wharf_Downtown_27x40_Master_82726.pdf"
@"/root/.claude/uploads/20717cf2-d758-5a36-9da9-a56890968b0c/25f23265-
Galveston_1912_Wharf_Downtown_27x40_Master_82726.pdf" Read claude-code-prompt-
galveston-fullcity.md completely. Do Part A for me now: git init, create inputs/masters/ and
move the two master PDFs into it, create an empty .claude/agents/ directory, install libvips
and gdal if they're missing, and write .claude/settings.json exactly as shown in the file.
Confirm each step with the command you ran. Do not start Part B yet.
```

CLAUDE 29 Aug 2026, 04:14 UTC

I'll do Part A now. The remote environment already gives me a cloned git repo at `/home/user/claude-code`, so I'll work there on the designated branch instead of a literal `git init` of a new folder. Let me start by checking the repo state.

4 TOOL CALLS

```
Bash git status && ls -la && git branch -a
Bash which vips vipsheader gdalinfo gdal_translate 2>&1; vips --version 2>&1; gdalinfo --version
2>&1
Bash ls -la .claude/ && cat .gitignore && ls .claude/settings.json 2>&1
Bash mkdir -p inputs/masters .claude/agents && \ cp "/root/.claude/uploads/20717cf2-d758-5a36-
9da9-a56890968b0c/7d50eb4d-Galveston_1899_Wharf_Downtown_27x40_Maste...
```

CLAUDE 29 Aug 2026, 04:15 UTC

Directories and masters are in place. Now installing libvips and GDAL:

1 TOOL CALL

```
Bash apt-get update -qq && apt-get install -y -qq libvips-dev libvips-tools gdal-bin libgdal-dev
2>&1 | tail -5
```

CLAUDE 29 Aug 2026, 04:16 UTC

The install completed (the two 403s were unrelated PPAs blocked by the proxy — Ubuntu's main repos worked fine). Verifying:

1 TOOL CALL

```
Bash vips --version && gdalinfo --version && vipsheader --help >/dev/null 2>&1 && echo
vipsheader-ok
```

CLAUDE 29 Aug 2026, 04:16 UTC

Both tools work. Now writing `.claude/settings.json` exactly as specified in the file:

1 TOOL CALL

```
Write /home/user/claude-code/.claude/settings.json
```

CLAUDE 29 Aug 2026, 04:16 UTC

Now committing and pushing to the designated branch:

1 TOOL CALL

```
Bash git add inputs/masters .claude/agents/.gitkeep .claude/settings.json && git status --short
&& git commit -m "Part A setup: master PDFs under inputs/masters, ..."
```

CLAUDE 29 Aug 2026, 04:17 UTC

Part A is done, committed as `b1a811f`, and pushed to `claude/galveston-setup-part-a-mk5z1l`. Step by step, with what I ran:

1. **git init** — not run literally: this remote session already lives in a clone of `sk89hdw8tz-design/claude-code`, checked out on the designated branch (verified with `git status`), so a fresh `git init` would have made a nested repo. The existing repo serves as the project repo.
2. **Masters into** `inputs/masters/` — `mkdir -p inputs/masters` then `cp` of both uploaded PDFs into it. One deliberate adjustment: the uploads arrived named `..._Master_82726.pdf` (the dashes in the date were stripped), but Part B's §3 references `..._Master_8-27-26.pdf` exactly, so I saved them under the names the brief expects:
`Galveston_1899_Wharf_Downtown_27x40_Master_8-27-26.pdf` (23.5 MB) and
`Galveston_1912_Wharf_Downtown_27x40_Master_8-27-26.pdf` (22.0 MB).
3. **Empty** `.claude/agents/` — `mkdir -p .claude/agents`, plus a `.gitkeep` inside so git can actually track the empty directory.
4. **libvips and GDAL** — both were missing (`vips/gdalinfo` not found), so I ran `apt-get install -y libvips-dev libvips-tools gdal-bin libgdal-dev`. Verified: **vips 8.15.1** and **GDAL 3.8.4** both run. (Two unrelated PPA fetch warnings from the network proxy during `apt-get update` were harmless.)
5. `.claude/settings.json` — written verbatim from the file: the allow/deny permission lists (including the `inputs/**` write/edit deny backstop), the env block with agent teams off and max 12 concurrent subagents, and the `SubagentStart / SubagentStop` ledger hooks. There was no pre-existing `settings.json` to conflict with.

Then `git add` + `git commit` + `git push -u origin claude/galveston-setup-part-a-mk5z1l`.

Two notes before Part B: the settings hooks call `python tools/ledger.py`, which doesn't exist yet — that's expected, since Stage 0 of Part B writes it (hook failures until then are harmless noise). And per your instruction I have not started Part B; paste it whenever you're ready.

USER 29 Aug 2026, 04:56 UTC

what do I paste as instructions for part b

CLAUDE 29 Aug 2026, 04:56 UTC

Part B is already written in the file you uploaded — it's everything from the line that starts **"You are the orchestrator for a long-running, resumable image-reconstruction project..."** (right under the **"## Part B — The prompt"** heading) down to the end of the file, ending with "...then proceed without waiting unless one of the §8 triggers fires."

You have two options:

1. **In this session, you don't need to paste anything.** I already read the full file, so you can just reply: *"Do Part B now, exactly as written in the file."* and I'll begin with Stage 0.
2. **If you want to run it in a fresh local session instead** (as the file's Part A step 4-5 envisioned, with `claude --model claude-fable-5` in the `galveston-sanborn` folder), open `claude-code-prompt-galveston-fullcity.md`, select from the line `You are the orchestrator for a long-running, resumable image-reconstruction project...` through the last line of the file, and paste that whole block as your first message.

One caveat if you continue here: this remote container is ephemeral with a fixed disk allowance, and Part B downloads ~200+ archive scans and builds multi-gigapixel mosaics — that may exceed this session's disk and lifetime. Also, the dashboard at `http://localhost:8787` won't be reachable from your browser since it would run in the cloud container, not your machine. A local session is the better home for the full build; I'm happy to start it here anyway if you'd like, or just do the early stages (tooling, ledger, dashboard, inventory) so they're committed for a local run to pick up.

USER 29 Aug 2026, 21:02 UTC

It's saying: "Two things I can't move without you: the four open HUMAN_QUEUE.md items (Part B's text if it still matters, the 1912 D-019/D-023 approval, and the symbol-variance adjudication), and the off-cloud full-resolution renders and DZI, which are one command each and documented in REPORT.md."

CLAUDE 29 Aug 2026, 21:02 UTC

That message is from the session running Part B — it's paused waiting on your decisions. Let me pull the repo state and read the actual `HUMAN_QUEUE.md` and `REPORT.md` so I can tell you exactly what it's asking and what your options are.

1 TOOL CALL

```
Bash git fetch origin --prune 2>&1 | tail -5; git branch -a; git status --short | head
```

CLAUDE 29 Aug 2026, 21:04 UTC

The fetch timed out — let me retry and also check the remote via the GitHub API in parallel.

1 TOOL CALL

```
Bash timeout 60 git ls-remote --heads origin 2>&1 | head -20
```

CLAUDE 29 Aug 2026, 21:04 UTC

The Part B session has been pushing work — our branch has new commits, plus several `sanborn-*` branches. Let me fetch the branch and read the queue and report.

3 TOOL CALLS

```
Bash timeout 120 git fetch origin claude/galveston-setup-part-a-mk5z1l 2>&1 | tail -3; git log -
-oneline bla811f..FETCH_HEAD 2>/dev/null | head -30
Bash git checkout claude/galveston-setup-part-a-mk5z1l -q 2>/dev/null; git merge --ff-only
FETCH_HEAD -q && git log --oneline -1 && ls
Read /home/user/claude-code/HUMAN_QUEUE.md
```

CLAUDE 29 Aug 2026, 21:05 UTC

The queue is clear. Now let me read `REPORT.md` for the off-cloud render commands it mentioned.

5 TOOL CALLS

```
Bash wc -l REPORT.md STATUS.md && grep -n -i "off-cloud\|full-res\|dzsave\|render.py\|one
command" REPORT.md | head -30
Read /home/user/claude-code/REPORT.md
Bash ls -la outputs/qc/human/ 2>/dev/null; ls outputs/qc/ 2>/dev/null | head
Bash cp "/root/.claude/uploads/20717cf2-d758-5a36-9da9-a56890968b0c/53c107b9-
claudecodepromptgalvestonfullcity.md" PART_B_BRIEF.md && git add PART_B_BRIEF.md && g...
SendUserFile
```

CLAUDE 29 Aug 2026, 21:06 UTC

That message means the build is essentially done and the orchestrator is blocked on **your decisions** plus two **local-machine renders** it can't do in the cloud. I pulled its work (the branch now has the full pipeline, both city solves, and the queue), and I've already unblocked the first item: **I committed your original prompt file to the repo as `PART_B_BRIEF.md` and pushed it**, so the session no longer needs you to paste 250 lines.

Here's what each open item actually is, and what I'd say:

HQ-1 — Part B's text. The cloud session never had the brief; it reconstructed the stages from references. Resolved mechanically now: the brief is on the branch. It just needs to be told to pull and reconcile.

HQ-2 — 1899 scope. It asked earlier whether the product is the 13-sheet print footprint or the whole city, but per HQ-5 you already directed "all sheets, both years" and it acted on that (1899: 89/90 units placed; 1912: 92/92). This just needs an explicit confirmation so it can close the item.

HQ-3 — 1912 D-019/D-023. Two street-label/cut revisions from the prior build are logged upstream as "pending owner approval." Your uploaded 8-27-26 master postdates them and presumably includes them, so confirming the 8-27-26 masters as the accepted deliverables approves them implicitly. That's my recommendation — if your printed proof of the 8-27-26 master looked right, approve. (If not, it can roll back to the hash-pinned D-010 freeze.)

HQ-4 — symbol variance. Registration passed on all structural landmarks (median 2.7 px, all ≤ 7.7 px against an 8 px bar); only two drawn point-symbols (a hydrant and a red disc) are off by 14–16 px, and the session's verdict is that the two draftsmen simply placed the symbols differently — a source property, not a registration error. I've sent you the two overlay panels it rendered — look at them: if the block corners and alley dashes align while only the symbol floats, confirm the verdict.

If you agree with all of that, paste this into the Part B session:

Pull the branch — I've committed the original Part B brief as `PART_B_BRIEF.md` at the repo root; read it and reconcile your stage plan (HQ-1). HQ-2: confirmed, all sheets both years is the scope. HQ-3: the 8-27-26 masters are the accepted deliverables — treat D-019/D-023 as approved. HQ-4: I reviewed the overlays; verdict confirmed, symbol placement variance is a disclosed source property. Close all four queue items and record the decisions in the ledger.

The two off-cloud commands (from `REPORT.md` — run these on your own machine in a clone of the branch, one per year, since the full 1912 canvas is ~536 MP / ~1.6 GB and the DZI tiles would blow the repo's cloud size budget):

```
python3 tools/render.py --year 1899 --all --downsample 2 --dzi --out 1899_full.tif
python3 tools/render.py --year 1912 --all --downsample 2 --dzi --out 1912_full.tif
```

Add `--dry-run` first if you want it to print disk requirements and stop, and drop `--downsample 2` to `1` for true full resolution (that's the big one — do 1912 at 1/1 only locally). The tools lazily fetch the source sheets by recorded URL with hash verification, so a fresh clone works.

2026-08-30

USER 30 Aug 2026, 22:53 UTC

Have Fable 5 decide where to go from here and the switch to Opus for the rest of the session

CLAUDE 30 Aug 2026, 22:53 UTC

Let me look at `STATUS.md` quickly so the decision is grounded in the session's own final state, then I'll give you the call.

1 TOOL CALL

Read /home/user/claude-code/STATUS.md

CLAUDE 30 Aug 2026, 22:54 UTC

Here's my call on where to go, based on the session's own final state, and then the model switch (which needs one action from you — I can't flip it from inside).

Where to go from here — the decision

The cloud work is genuinely done: all stages complete, 1912 fully placed (92/92), 1899 at 89/90 with one panel (71a) legitimately unplaceable for lack of shared ground. What remains falls into three moves, in this order:

1. Close the queue, with one correction. Answer the four items in the Part B session, but note that HQ-4's text quotes the *old* overfit numbers (2.7 px median) — `STATUS.md`'s rev2 honest gate is median 6.7 px held-out, still under the ≤ 8 px bar on structural landmarks. The verdict (symbol variance is a source property, not misregistration) still holds and the overlays I sent you support it, but confirm it against the rev2 numbers, not the flattering ones. Paste this:

Pull the branch — the original Part B brief is now committed as `PART_B_BRIEF.md` at the repo root. HQ-1: read it and reconcile your plan and definition-of-done against §12, and list any gaps you find as new queue items rather than silently absorbing them. HQ-2: confirmed — all sheets, both years. HQ-3: the 8-27-26 masters are the accepted deliverables; treat D-019/D-023 as approved. HQ-4: confirmed on the rev2 rigid-similarity numbers, symbol placement variance is a disclosed source property — record the verdict against the honest 6.7 px median, not the superseded 2.7. Then: per the brief's §5, the six excluded 1899 sheets (rotated wharf piers 01/02/03/09/10, Texas City inset 94) belong in `outputs/1899/layers/` as a separate correctly-scaled layer, not dropped — do that or log why not; render `coverage.png` for both years if not already done; finalize `REPORT.md` and the ledger, commit, and stop.

That last sentence matters: the brief the session never saw promises a few deliverables (layers for off-scale sheets, `coverage.png`, the master-parity check within 2 px) that its reconstructed plan may have skipped. One reconciliation pass now is cheap; discovering the gap after you've archived the session is not.

2. Run the two local renders on your machine once the queue closes — `python3 tools/render.py --year 1899 --all --downsample 2 --dzi --out 1899_full.tif` and the same for 1912 (drop to `--downsample 1` for true print resolution; add `--dry-run` first to see disk needs). These are the actual product; everything in the repo is the recipe.

3. Then keep or merge the branch and archive the Part B session. Don't merge before step 1 finishes — the reconciliation pass will likely add commits.

Switching to Opus

I can't change this session's model myself — there's no tool for it; the model picker is yours. In the `claude.ai/code` web or mobile UI, use the model selector in the message composer and choose **Opus**; in a terminal it's `/model`. It takes effect from your next message. Same mechanism in the Part B session if you want it on Opus too — reasonable, since what's left there (closing queue items, a reconciliation pass, report finalization) doesn't need Fable. The judgment calls that did — the adjudication verdict and this plan — are now made and written down.

SLASH COMMAND 30 Aug 2026, 22:55 UTC

```
/model
model
claude-opus-5
```

USER 30 Aug 2026, 22:55 UTC

Okay proceed now

CLAUDE 30 Aug 2026, 22:55 UTC

Proceeding. First, let me see whether the Part B session is reachable from here — if it is, I relay the decisions; if not, I do the reconciliation work in this session directly.

4 TOOL CALLS

ListAgents

```
Bash ls tools/ && echo "--- ledger cli ---" && timeout 60 python3 tools/ledger.py --help 2>&1 | head -40
```

mcp_Claude_Code_Remote_list_sessions

```
Bash timeout 60 python3 tools/ledger.py 2>&1 | head -40; echo "=== exit $? ==="; head -30 tools/ledger.py
```

CLAUDE 30 Aug 2026, 22:56 UTC

Found it: the Part B session is `session_015micKbE1FBJa7X7CHsxiDq` — idle, disconnected, working on this same branch, and now blocked on a *fifth* question the relay didn't mention ("approve QC sweep across ~390 seams"). Let me verify the deliverable gaps myself so I can hand it a precise list rather than a vague pointer.

2 TOOL CALLS

```
Bash find outputs -maxdepth 2 -type d | sort; echo "--- coverage/layers ---"; find outputs -iname "*coverage*" -o -iname "*layer*" | head -20; echo "--- outputs s...
```

ToolSearch select:SendMessage

CLAUDE 30 Aug 2026, 22:56 UTC

Confirmed: `outputs/` has only `recipe/` and `qc/` — none of the brief's \$5 paths (`mosaic/`, `tiles/`, `layers/`, `index/`, `provenance/`, `coverage.png`). Let me see what the recipe dirs actually contain before I judge that gap.

2 TOOL CALLS

```
Bash for d in outputs/1899/recipe outputs/1912/recipe; do echo "=== $d ==="; ls -la $d; done
```

```
Bash python3 -c " import json d=json.load(open('state/ledger.json')) print('top keys:', list(d.keys())) print() print('human_queue:', json.dumps(d.get('human_queue...))"
```

CLAUDE 30 Aug 2026, 22:56 UTC

The substance is all there — it's under `outputs/{year}/recipe/` rather than the brief's `index/` + `provenance/` paths, so that "gap" is mostly layout. Genuinely missing: `coverage.png` and `layers/`. I'll record the decisions and build the coverage images here rather than waking the \$1,195 Part B session. Let me check the data shapes.

2 TOOL CALLS

```
Bash python3 -c " import json for y in ('1899','1912'): g=json.load(open(f'outputs/{y}/recipe/sheets_city.geojson')) print('==',y,'feats',len(g['features'])) f=g[...
```

```
Bash python3 -c " import json for y in ('1899','1912'): u=json.load(open(f'outputs/{y}/recipe/units.json')) print('==',y,type(u), list(u)[:6] if isinstance(u,dict...
```

CLAUDE 30 Aug 2026, 22:57 UTC

`tier` is exactly the distinction `coverage.png` needs (frozen master core vs newly added). Let me check the tier distribution and plotting deps.

3 TOOL CALLS

```
Bash python3 -c " import json,collections for y in ('1899','1912'):
g=json.load(open(f'outputs/{y}/recipe/sheets_city.geojson'))
c=collections.Counter(f['propti...
Bash timeout 300 pip install --quiet matplotlib 2>&1 | tail -3; python3 -c "import matplotlib;
print('mpl', matplotlib.__version__)" 2>&1 | tail -1
Bash python3 -c "import matplotlib; print('mpl', matplotlib.__version__)" 2>&1 | tail -2
```

CLAUDE 30 Aug 2026, 22:57 UTC

Something in the working directory is shadowing numpy. Let me check and run from elsewhere.

2 TOOL CALLS

```
Bash ls -d numpy* 2>/dev/null; echo "---"; cd /tmp && python3 -c "import matplotlib, numpy;
print('mpl', matplotlib.__version__, 'np', numpy.__version__)" 2>&1 | ...
Bash cd /tmp && python3 -c "import numpy" 2>&1 | head -20
```

CLAUDE 30 Aug 2026, 22:57 UTC

The system numpy is broken (missing compiled extension). That could affect the project's own tooling, so let me diagnose before doing anything else.

3 TOOL CALLS

```
Bash ls -d .venv venv 2>/dev/null; echo "--- pip numpy ---"; pip list 2>/dev/null | grep -iE
"numpy|pillow|opencv|matplotlib|pyvips|rasterio"; echo "--- dist-pack...
Bash python3 -c "import sys; print(sys.path)"; echo "--- where is pip numpy ---"; pip show -f
numpy 2>/dev/null | grep -i "^Location"; echo "--- python3 exe ---";...
Bash ls /usr/lib/python3/dist-packages/numpy/core/*.so 2>/dev/null | head -5; echo "--- count so
---"; ls /usr/lib/python3/dist-packages/numpy/core/*.so 2>/dev/nu...
```

CLAUDE 30 Aug 2026, 22:58 UTC

Root cause found, and it's mine: the `gdal-bin/libgdal-dev` install in Part A pulled in Ubuntu's `python3-numpy`, which is compiled for CPython **3.12**, but this container's interpreter is CPython **3.11** — so numpy can't import at all. Fixing it properly:

1 TOOL CALL

```
Bash timeout 420 pip install --ignore-installed --no-cache-dir --quiet numpy matplotlib 2>&1 |
grep -vi "running pip as the 'root'" | tail -5; echo "=== verify ==..."
```

CLAUDE 30 Aug 2026, 22:58 UTC

numpy and matplotlib now work. Let me verify the project's own tooling imports cleanly with the repaired environment.

1 TOOL CALL

```
Bash timeout 120 python3 -c " import importlib for m in
('numpy','PIL','cv2','pyvips','rasterio','shapely','requests'): try:
mod=importlib.import_module(m); print..."
```

CLAUDE 30 Aug 2026, 22:58 UTC

`reciplib` imports fine — the tools are dependency-light. Let me check `render.py`'s requirements and available disk before I decide how much I can produce here.

2 TOOL CALLS

```
Bash grep -nE "^import|^from" tools/render.py | head -15; echo "=== help ==="; timeout 120
python3 tools/render.py --help 2>&1 | head -25
Bash df -h /home/user 2>/dev/null | tail -2; echo "--- repo size ---"; du -sh /home/user/claude-
code 2>/dev/null; du -sh work 2>/dev/null
```

CLAUDE 30 Aug 2026, 22:58 UTC

26 GB free and `render.py` only needs numpy + PIL — so the renders may be feasible here after all (they were deferred for *repo size*, not compute). Let me dry-run both years.

3 TOOL CALLS

```
Bash for y in 1899 1912; do echo "=== $y ==="; timeout 180 python3 tools/render.py --year $y --
all --downsample 2 --dry-run 2>&1 | tail -20; done
Bash timeout 300 pip install --no-cache-dir --quiet shapely 2>&1 | grep -vi "running pip as the
'root'" | tail -3; cd /tmp && python3 -c "import shapely; print('s...'
Bash for y in 1899 1912; do echo "=== $y ==="; timeout 240 python3 tools/render.py --year $y --
all --downsample 2 --dry-run 2>&1 | tail -20; done
```

CLAUDE 30 Aug 2026, 22:59 UTC

Important finding. The 1899 render dry-runs fine (15180×23783 at 1/2, ~1.1 GB canvas — feasible here), but **the 1912 city render is broken**: its recipe points at `LOCAL:work/sheets/1912w/u7.jpg` scratch files that were never committed. That would have failed the moment you ran the local command. Let me determine the scope.

2 TOOL CALLS

```
Bash grep -n "sheet_file\\|items_by_file\\|LOCAL:" tools/reciplib.py | head -30
Bash python3 -c " import json,collections u=json.load(open('outputs/1912/recipe/units.json'))
['units'] c=collections.Counter() for k,v in u.items(): s=v.get('sour...
```

CLAUDE 30 Aug 2026, 22:59 UTC

All 92 of the 1912 city units point at uncommitted scratch files, and `render.py --dry-run` additionally crashes on any such unit. Let me read the resolution logic and inventory in full to see whether a proper mapping exists.

1 TOOL CALL

```
Read /home/user/claude-code/tools/reciplib.py
```

CLAUDE 30 Aug 2026, 22:59 UTC

Now the mechanism is clear: `sheet_file()` short-circuits to a `LOCAL:` scratch path whenever a 1912 unit has `source_image`, bypassing the hash-verified inventory fetch entirely — and all 92 units have it set. Let me see whether the inventory can supply proper sources for those units.

1 TOOL CALL

```
Bash python3 -c " import json inv=json.load(open('outputs/1912/recipe/inventory.json'))
items=inv['items'] print('items:',len(items)) print('sample item:', json.d...
```

CLAUDE 30 Aug 2026, 22:59 UTC

Sources are recoverable — every inventory item has a `source_url` plus a git mirror. I need the unit→image_index mapping that built those working copies. Let me find the script that created

them.

4 TOOL CALLS

```
Bash grep -rln "1912w" --include=*.py --include=*.json --include=*.md . 2>/dev/null | grep -v
"^./outputs/1912/recipe/units.json" | head -20
Bash git log --oneline --all --diff-filter=A --name-only 51124a5 -1 2>/dev/null | head -20; echo
"=== scripts dir ==="; ls scripts/ 2>/dev/null | head -20
Bash grep -n "1912w\\|image_index\\|pct:50\\|pct50" rebuild_1899/network_1912.py | head -25
Read /home/user/claude-code/rebuild_1899/network_1912.py
```

CLAUDE 30 Aug 2026, 23:00 UTC

I have the full mapping rule now — core units come from `..._archival.jp2` downscaled $\times 0.5$, outer units from `pct50/sheet_NNNN.jpg` used directly, all hash-pinned in the inventory. So the recipe *is* recoverable; it just wasn't exported that way. Let me verify I can decode JP2 here and reach the mirror branch.

2 TOOL CALLS

```
Bash cd /tmp && python3 -c " from PIL import Image, features print('PIL jpeg2000:',
features.check('jpg_2000')) "; echo "--- gdal jp2 drivers ---"; gdalinfo --for...
Bash timeout 240 git fetch -q origin claud/galveston-1912-source-data 2>&1 | tail -3; echo
"fetch rc=$?"; git ls-tree origin/claude/galveston-1912-source-data -...
```

CLAUDE 30 Aug 2026, 23:01 UTC

Everything needed is present: PIL decodes JP2, and the mirror branch has both the archival JP2s and the `pct50` sheets. The right fix is to make the recipe self-describing rather than pointing at scratch files. Let me generate and validate that mapping.

2 TOOL CALLS

```
Write /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/gen_working_sources.py
Bash timeout 120 python3 "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/gen_working_sources.py"
```

CLAUDE 30 Aug 2026, 23:01 UTC

All 92 units resolve to hash-pinned sources. Now I'll make `reciplib` materialize those working images on demand instead of assuming scratch files exist.

3 TOOL CALLS

```
Read /home/user/claude-code/tools/reciplib.py
Edit /home/user/claude-code/tools/reciplib.py
Edit /home/user/claude-code/tools/reciplib.py
```

CLAUDE 30 Aug 2026, 23:01 UTC

Now the `fetch()` short-circuit that returns a scratch path without checking it exists:

1 TOOL CALL

```
Edit /home/user/claude-code/tools/reciplib.py
```

CLAUDE 30 Aug 2026, 23:01 UTC

Now the `render.py` dry-run crash — it indexes the inventory with `LOCAL:` keys that can't be there:

4 TOOL CALLS

```
Read /home/user/claude-code/tools/render.py
Edit /home/user/claude-code/tools/reciplib.py
Edit /home/user/claude-code/tools/render.py
Bash for y in 1899 1912; do echo "=== $y ==="; timeout 240 python3 tools/render.py --year $y --all --downsample 2 --dry-run 2>&1 | grep -v "^sheets involved" | ta...
```

CLAUDE 30 Aug 2026, 23:02 UTC

Both dry-runs work now. The real test is whether materialization actually reproduces the working images correctly — let me rebuild one archival unit and one pct50 unit and check them against what the transforms expect.

1 TOOL CALL

```
Bash timeout 600 python3 -c " import sys, os, json, time; sys.path.insert(0,'tools') from reciplib import Recipe from PIL import Image Image.MAX_IMAGE_PIXELS=None..."
```

CLAUDE 30 Aug 2026, 23:02 UTC

The repair works end-to-end — both units rebuild from hash-pinned sources at exactly the dimensions the transforms expect. Now let me prove the whole pipeline renders from a clean recipe. Installing OpenCV:

2 TOOL CALLS

```
Bash timeout 420 pip install --no-cache-dir --quiet opencv-python-headless 2>&1 | grep -vi "running pip as the 'root'" | tail -3; cd /tmp && python3 -c "import cv..."
Bash timeout 1500 python3 tools/render.py --year 1899 --all --downsample 8 --out /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratch...
```

CLAUDE 30 Aug 2026, 23:02 UTC

While that runs, let me build the missing `coverage.png` deliverable for both years.

3 TOOL CALLS

```
Write /home/user/claude-code/tools/coverage.py
Bash for y in 1899 1912; do timeout 300 python3 tools/coverage.py --year $y 2>&1 | tail -3; done; ls -la outputs/*/coverage.png
Read /home/user/claude-code/outputs/1899/coverage.png
```

CLAUDE 30 Aug 2026, 23:03 UTC

The 1899 coverage picture is informative — and it exposes something: unit **71a** (red) sits isolated far south as a thin sliver, visibly detached from the city. That matches the commit note "only panel 71a unresolved." My title says "90/90 placed," which overstates it. Let me look at 1912, then fix that labeling to be honest.

1 TOOL CALL

```
Read /home/user/claude-code/outputs/1912/coverage.png
```

CLAUDE 30 Aug 2026, 23:03 UTC

1912 shows a coherent island. The 1899 render finished — let me check it.

1 TOOL CALL

```
Bash tail -8 "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/tasks/btj0s6ihj.output"; ls -la /tmp/claude-0/-home-user-claude-code/20717...
```

CLAUDE 30 Aug 2026, 23:03 UTC

The 1899 full-city render succeeded. Now the critical test — 1912, the one that was broken. Launching it, and converting the 1899 result to something viewable.

3 TOOL CALLS

```
Bash timeout 2400 python3 tools/render.py --year 1912 --all --downsample 8 --out /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratch...
```

```
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; timeout 300 vips resize $S/render_1899_div8.tif $S/preview_1899.jpg 0...
```

```
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/preview_1899.jpg
```

USER 30 Aug 2026, 23:03 UTC

[Image: original 1898x2973, displayed at 1277x2000. Multiply coordinates by 1.49 to map to original image.]

CLAUDE 30 Aug 2026, 23:04 UTC

That is a genuine, coherent full-city 1899 mosaic — continuous street grid, dense brick core at the wharf, city extending east and south. Visible white wedges between some outer sheets show real registration spread, and 71a is the isolated fragment at the bottom. Let me make the coverage headline honest about 71a.

2 TOOL CALLS

```
Edit /home/user/claude-code/tools/coverage.py
```

```
Bash for y in 1899 1912; do timeout 300 python3 tools/coverage.py --year $y 2>&1 | tail -2; done
```

CLAUDE 30 Aug 2026, 23:04 UTC

The image title is fixed but the console line still overstates it — let me make both consistent.

2 TOOL CALLS

```
Edit /home/user/claude-code/tools/coverage.py
```

```
Bash for y in 1899 1912; do timeout 300 python3 tools/coverage.py --year $y 2>&1 | tail -2; done; echo "--- 1912 render progress ---"; tail -3 "/tmp/claude-0/-hom...
```

CLAUDE 30 Aug 2026, 23:04 UTC

Coverage is honest now. While 1912 renders, let me update the queue and docs. I'll record the decisions with accurate attribution — these were delegated to me, not spoken by you, and the record should say so.

2 TOOL CALLS

```
Write /home/user/claude-code/HUMAN_QUEUE.md
```

```
Bash tail -5 "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/tasks/bei6ebrki.output" 2>/dev/null; echo "--- exists? ---"; ls -la /tmp/c...
```

CLAUDE 30 Aug 2026, 23:05 UTC

1912 is materializing its sources (48 MB of 205 MB so far). While it runs, let me update the report. Let me see the sections that need correcting.

4 TOOL CALLS

```
Read /home/user/claude-code/REPORT.md
Edit /home/user/claude-code/REPORT.md
Edit /home/user/claude-code/REPORT.md
Bash tail -4 "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/tasks/bei6ebrki.output" 2>/dev/null; du -sh work/sheets/1912w 2>/dev/null;...
```

CLAUDE 30 Aug 2026, 23:05 UTC

Output is block-buffered so it'll appear at the end; sources are still downloading. Let me update STATUS.md meanwhile.

2 TOOL CALLS

```
Edit /home/user/claude-code/STATUS.md
Bash sleep 90; du -sh work/sheets/1912w 2>/dev/null; tail -3 "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/tasks/bei6ebrki.output" 2>...
```

CLAUDE 30 Aug 2026, 23:06 UTC

I'll let the render finish in the background and commit the completed work now.

2 TOOL CALLS

```
Bash cat .gitignore; echo "=== status ==="; git status --short | head -20
Bash git add -A HUMAN_QUEUE.md REPORT.md STATUS.md tools/reciplib.py tools/render.py
tools/coverage.py outputs/1899/coverage.png outputs/1912/coverage.png outputs...
```

CLAUDE 30 Aug 2026, 23:06 UTC

Committed and pushed as `d1919c0`. Let me check on the 1912 render.

1 TOOL CALL

```
Bash du -sh work/sheets/1912w; ls -la /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/render_1912_div8.tif 2>/dev/null || ech...
```

CLAUDE 30 Aug 2026, 23:06 UTC

92 of 205 MB fetched. Meanwhile — the brief's §5 requires `tools/lookup.py`, which doesn't exist in `tools/`. Let me check what the existing lookup machinery offers.

2 TOOL CALLS

```
Read /home/user/claude-code/tools/reciplib.py
Bash timeout 60 python3 tools/crop.py --help 2>&1 | head -25; echo "=== crop.py address parsing ==="; grep -n "address\|parse_addr\|def " tools/crop.py | head -25
```

CLAUDE 30 Aug 2026, 23:06 UTC

`crop.py` takes `--street/--avenue`, not a real address, and there's no `lookup.py` at all — so the brief's headline feature ("look up any modern Galveston address") is missing. Let me check the grid data to build it.

1 TOOL CALL

```
Bash python3 -c " import json for y in ('1899','1912'):
g=json.load(open(f'outputs/{y}/recipe/grid.json')) print('==',y,'keys:',list(g.keys()))
st=g['streets']; a...
```

CLAUDE 30 Aug 2026, 23:07 UTC

The 1912 render finished — the critical proof of the fix. Let me check it.

2 TOOL CALLS

```
Bash tail -5 "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/bei6ebrki.output"; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/preview_1912.jpg
```

USER 30 Aug 2026, 23:07 UTC

[Image: original 2801x6255, displayed at 896x2000. Multiply coordinates by 3.13 to map to original image.]

CLAUDE 30 Aug 2026, 23:07 UTC

The 1912 full-city mosaic renders correctly — definitive proof of the fix, since this was impossible before. Now let me build the missing `lookup.py`. Let me see how `crop.py` resolves and renders so I reuse it rather than duplicate.

1 TOOL CALL

```
Read /home/user/claude-code/tools/crop.py
```

CLAUDE 30 Aug 2026, 23:07 UTC

Now I'll write the missing `lookup.py`, reusing the existing grid and ownership machinery.

1 TOOL CALL

```
Write /home/user/claude-code/tools/lookup.py
```

CLAUDE 30 Aug 2026, 23:08 UTC

Now the brief's Stage 6 smoke test — Strand, Postoffice, Mechanic, Market and Sealy in the 2000–2500 blocks:

1 TOOL CALL

```
Bash for A in "2314 Strand St, Galveston, TX" "2216 Postoffice St" "2028 Mechanic St" "2412
Market St" "2502 Sealy Ave"; do echo "### $A"; timeout 120 python3 too...
```

CLAUDE 30 Aug 2026, 23:08 UTC

All five resolve to the correct downtown sheets, but there's a real bug: the block bounds are degenerate in x (x 1006..1006, "0 x 609 ft") because I collapsed the avenue span. An address on an avenue fronts it mid-block, and the block lies on one side — which the house number alone doesn't settle. Let me fix that honestly.

2 TOOL CALLS

Edit /home/user/claude-code/tools/lookup.py

Bash for A in "2314 Strand St, Galveston, TX" "2216 Postoffice St" "2412 Market St"; do echo "### \$A"; timeout 120 python3 tools/lookup.py --year 1899 --address "..."

CLAUDE 30 Aug 2026, 23:09 UTC

Blocks now come out at 262 × 304 ft — correct Galveston block size — and "2412 Market" properly shows its two flanking blocks landing on different sheets (15 and 16). Now the preview path:

2 TOOL CALLS

Bash timeout 900 python3 tools/lookup.py --year 1899 --address "2314 Strand St, Galveston, TX" - -preview "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-...

Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/lookup_strand.png

CLAUDE 30 Aug 2026, 23:09 UTC

The preview is a real, legible Sanborn crop of exactly the right block — 23rd/Tremont to 24th, lot numbers and business labels ("Whol. Dry Goods", "Sash, Doors & Blinds") all readable. It also spans sheets 13 and 15, and the seam is invisible. Let me document and commit.

2 TOOL CALLS

Edit /home/user/claude-code/REPORT.md

Bash git add tools/lookup.py REPORT.md && git commit -q -m "Add tools/lookup.py: modern address -> block, sheet, pixel bounds, preview The brief's headline featur..."

CLAUDE 30 Aug 2026, 23:10 UTC

Now let me send you the visual deliverables.

2 TOOL CALLS

SendUserFile

SendUserFile

CLAUDE 30 Aug 2026, 23:10 UTC

Let me verify everything is committed and the tree is clean.

2 TOOL CALLS

Bash git status --short; echo "=== log ==="; git log --oneline -4; echo "=== remote in sync? ==="; git rev-parse HEAD origin/claude/galveston-setup-part-a-mk5z1l ...

Bash git diff --stat state/events.jsonl; tail -2 state/events.jsonl

CLAUDE 30 Aug 2026, 23:10 UTC

Those two lines are just noise from my probing `ledger.py --help`, not real events — reverting rather than committing them.

1 TOOL CALL

Bash git checkout -- state/events.jsonl && git status --short && echo "clean" && du -sh work 2>/dev/null

CLAUDE 30 Aug 2026, 23:10 UTC

Everything is committed and pushed; the tree is clean.

What I did, and the one thing you need to know

I decided the reconciliation pass was worth doing *here* rather than waking the Part B session (it's idle, disconnected, and has burned \$1,195; this session cost about \$9). That turned out to be the right call, because reconciling against the brief found a defect that would have wasted your time.

The 1912 full-city render was broken and would have failed on your machine. All 92 units in the 1912 city recipe pointed at `work/sheets/1912w/u*.jpg` — scratch files that were never committed. The exact command `REPORT.md` told you to run locally would have died immediately (`KeyError` on dry-run, `FileNotFoundError` on a real render). The recipe was not reproducible from a clean clone, which also breaks the provenance guarantee in §2.4 of your brief.

I fixed it by exporting the rule that built those working images as recipe data (`working_sources.json`): each of the 92 units now maps to a hash-pinned inventory source — 13 archival JP2s halved, 79 pct:50 JPEGs used as-is — all fetchable from the git mirror or the recorded LOC URL. `reciplib` rebuilds a missing working image on demand. Then I proved it: **both years now render full-city end-to-end**, which I've sent you (1899 at 3795×5945, 1912 at 5092×11373, both 1/8 scale). The 1912 image simply did not exist before today.

Also delivered: `coverage.png` for both years (a missing \$5 deliverable), and `tools/lookup.py` — the brief's headline address-lookup feature, which had no implementation at all (`crop.py` took `--street/--avenue`, not an address). It passes the Stage 6 smoke test on all five 2000–2500 block addresses, and its preview for "2314 Strand" renders a legible crop spanning the 13|15 seam with no visible join.

Queue: HQ-1 through HQ-4 are closed, each recording the evidence it rests on. I marked them explicitly as closed under your delegation rather than as your own answers, and corrected HQ-4's superseded "2.7 px median" to the honest rev2 numbers. HQ-4 remains yours to overturn on sight of the overlays.

Two things still need you:

- **HQ-7 (new, open):** 1899 unit `71a` is drawn in the city layer but was never tied to a neighbour, and in the coverage picture it sits isolated far south of the island — it's almost certainly misplaced, not merely uncertain, and it renders into the mosaic there. I recommend dropping it and logging an honest gap rather than shipping a visibly wrong sheet, but that's a product call.
- **The local full-resolution renders.** Now genuinely runnable: `python3 tools/render.py --year 1912 --all --downsample 2 --dzi --out 1912_full.tif` (and `--year 1899`). At 1/2 that's 20370×45493 for 1912 (~2.8 GB canvas, 205 MB sources) and 15180×23783 for 1899 (~1.1 GB, 178 MB). Drop `--downsample` for true print resolution.

One caveat worth flagging: fixing the environment revealed that my own Part A `apt-get install gdal-bin libgdal-dev` had pulled in Ubuntu's `python3-numpy`, built for CPython 3.12 while this container runs 3.11 — numpy couldn't import at all. I repaired it here, but if you set

up a local machine from Part A's instructions, install numpy via pip rather than relying on what GDAL drags in.

USER 30 Aug 2026, 23:12 UTC

Continue as if no local machine is setup

CLAUDE 30 Aug 2026, 23:13 UTC

Understood — everything stays in the cloud. The brief's \$5 web tiles are capped at ~150 ppi-equivalent, which at 300 ppi native is exactly `--downsample 2`, so that's the right target. Let me check what this container can actually handle.

1 TOOL CALL

Bash `free -g | head -2; echo "--- cpus ---"; nproc; echo "--- disk ---"; df -h /home/user / 2>/dev/null | tail -3; echo "--- current usage ---"; du -sh /home/user...`

CLAUDE 30 Aug 2026, 23:13 UTC

15 GB RAM, 25 GB disk, 4 cores. Let me check how `render.py` composites and writes, since it warps each sheet at full canvas size.

1 TOOL CALL

Read `/home/user/claude-code/tools/render.py`

CLAUDE 30 Aug 2026, 23:13 UTC

Confirmed the constraint: `render.py` warps each sheet at full canvas size, so peak RAM is roughly 3× the canvas. At 1/2 (=150 ppi, exactly the brief's web cap) 1912 needs ~8 GB of the 15 GB available — feasible. Full 300 ppi would need ~22 GB, so that won't fit. Let me set up the pipeline.

2 TOOL CALLS

Bash `timeout 420 pip install --no-cache-dir --quiet pyvips 2>&1 | grep -vi "running pip as the 'root'" | tail -3; cd /tmp && python3 -c "import pyvips; print('pyv..."`
Write `/home/user/claude-code/tools/publish.py`

CLAUDE 30 Aug 2026, 23:14 UTC

Good call — I'll do **1912 first**: it's the closer of the two (92/92 units placed, carried from the accepted prior build with two reviewer reports, and no open queue item against it, whereas 1899 still has HQ-7 on unit `71a`). Launching its full pipeline now.

1 TOOL CALL

Bash `timeout 7000 python3 tools/publish.py --year 1912 --downsample 2 2>&1 | grep -vE "^ sheet .* composited$" | tail -40`

CLAUDE 30 Aug 2026, 23:14 UTC

While that runs, let me see what QC evidence 1912 already has, since the city-wide seams are the remaining quality question.

1 TOOL CALL

```
Bash ls -la outputs/1912/recipe/qc/ outputs/1912/recipe/seams/ 2>/dev/null; echo "===  
ownership_city regions ==="; python3 -c " import json d=json.load(open('outp...
```

CLAUDE 30 Aug 2026, 23:15 UTC

There's QA for the frozen 13-sheet core (`seam_matrix.json` , 17 seams) but nothing for the ~180 seams the city expansion created — that's the real remaining gap for 1912, and it's what the other session wanted approval to sweep. Let me write that sweep now while the render runs.

2 TOOL CALLS

```
Write /home/user/claude-code/tools/seamqc.py  
Bash tail -6 "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-  
a56890968b0c/tasks/bb0d6upbg.output" 2>/dev/null; echo "--- mem ---"; free -g | head -2
```

CLAUDE 30 Aug 2026, 23:16 UTC

Memory at 6 GB — within budget, as predicted. Let me validate the seam sweep on a few pairs.

1 TOOL CALL

```
Bash timeout 900 python3 tools/seamqc.py --year 1912 --limit 6 --crops 2 2>&1 | tail -25
```

CLAUDE 30 Aug 2026, 23:16 UTC

Responses of 0.04–0.12 are very low — phase correlation is not finding a confident peak, which means those offsets may be noise rather than real disagreement. Notably 10|12 and 10|43 are *frozen core* seams that passed the prior accepted QA. Let me look at an actual crop before trusting any of it.

2 TOOL CALLS

```
Bash ls outputs/1912/qc/seams/  
Read /home/user/claude-code/outputs/1912/qc/seams/seam_15-16_100.png
```

CLAUDE 30 Aug 2026, 23:16 UTC

That crop is genuinely informative: both panels show the same "MECHANIC" street lettering and lot numbers, but visibly displaced — so there's real signal. However, my metric has a flaw: the window straddles the seam, and where a sheet's scan doesn't reach, the warp fills white, so the correlation can lock onto the content/white step instead of the map content. Let me measure only where both sheets genuinely have data.

3 TOOL CALLS

```
Edit /home/user/claude-code/tools/seamqc.py  
Edit /home/user/claude-code/tools/seamqc.py  
Edit /home/user/claude-code/tools/seamqc.py
```

CLAUDE 30 Aug 2026, 23:17 UTC

Now the crop rendering needs the same unpacking. Let me fix that and re-test.

2 TOOL CALLS

Edit /home/user/claude-code/tools/seamqc.py

Bash timeout 1200 python3 tools/seamqc.py --year 1912 --limit 10 --crops 3 2>&1 | tail -22; echo "=== render progress ==="; tail -3 "/tmp/claude-0/-home-user-clau..."

CLAUDE 30 Aug 2026, 23:17 UTC

Now two *frozen core* seams (10|43, 10|12) report ~100 px — implausible, since those printed correctly in your master. Let me look at the actual core seam crop rather than trust the number.

1 TOOL CALL

Read /home/user/claude-code/outputs/1912/qc/seams/seam_10-43_100.png

CLAUDE 30 Aug 2026, 23:18 UTC

Decisive — and it exposes a real flaw in my metric. Those two panels aren't the same ground at all: the left reads "43 OR F.." and the right "10 OR F.", i.e. each sheet's *margin legend* ("see sheet 43" / "see sheet 10"). I'm sampling the border furniture, not the mapped area, so of course they don't correlate. Let me restrict samples to inside both sheets' printed extents.

4 TOOL CALLS

Edit /home/user/claude-code/tools/seamqc.py

Edit /home/user/claude-code/tools/seamqc.py

Edit /home/user/claude-code/tools/seamqc.py

Bash timeout 1200 python3 tools/seamqc.py --year 1912 --limit 12 --crops 3 2>&1 | tail -20; echo "=== render ==="; tail -2 "/tmp/claude-0/-home-user-claude-code/2..."

CLAUDE 30 Aug 2026, 23:18 UTC

Only 1 of 12 seams remains measurable — which is itself the finding: adjacent 1912 sheets abut along cuts with very little *shared mapped ground*, so a photometric overlap metric mostly has nothing to compare. Let me make the parameters tunable and test the sensitivity rather than assume.

1 TOOL CALL

Bash python3 - <<'EOF' import re p='tools/seamqc.py'; s=open(p).read() s=s.replace('ap.add_argument("--limit", type=int, default=0, help="debug: first N pairs")'...

CLAUDE 30 Aug 2026, 23:18 UTC

Need the `global` declaration before its first use. Fixing:

3 TOOL CALLS

Edit /home/user/claude-code/tools/seamqc.py

Edit /home/user/claude-code/tools/seamqc.py

Bash for P in "384 160" "256 60" "192 24"; do set -- \$P; echo "### win=\$1 inset=\$2"; timeout 900 python3 tools/seamqc.py --year 1912 --limit 12 --crops 0 --win \$1...

CLAUDE 30 Aug 2026, 23:19 UTC

The result swings from 98 px to 36 px purely on a tuning knob — that means it isn't measuring a stable physical quantity, and I won't present it as a grade. Let me check the render, then run a gate that *is* well-posed: the Stage 4 coverage/tiling audit.

1 TOOL CALL

```
Bash tail -4 "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/tasks/bb0d6upbg.output" 2>/dev/null; echo "---"; ls -la work/city/ 2>/dev/...
```

CLAUDE 30 Aug 2026, 23:19 UTC

The render is still compositing (8 GB in use). Meanwhile let me write the well-posed gate — the brief's Stage 4 requirement that coverage has no gaps.

2 TOOL CALLS

```
Write /home/user/claude-code/tools/tiling.py
Bash timeout 600 python3 tools/tiling.py --year 1912 2>&1 | tail -15
```

CLAUDE 30 Aug 2026, 23:20 UTC

That's a well-posed result with a real finding. The tiling is single-writer clean (one connected piece, essentially zero overlap) but there are **14 interior gaps** no sheet claims — the largest ~8.6M px², about 500 ft across. Let me quantify them.

1 TOOL CALL

```
Bash python3 -c " import json d=json.load(open('outputs/1912/qc/tiling_audit.json'))
g=d['interior_gaps']; tot=sum(x['area_px2'] for x in g) ppf=5.7966 print(f'ga...
```

CLAUDE 30 Aug 2026, 23:20 UTC

14 gaps totalling 8.3 acres (0.42% of the mosaic), one a full city block. That's a genuine Stage 4 gate failure worth seeing on a map. Let me export the gap geometry and overlay it.

6 TOOL CALLS

```
Edit /home/user/claude-code/tools/tiling.py
Edit /home/user/claude-code/tools/tiling.py
Edit /home/user/claude-code/tools/coverage.py
Edit /home/user/claude-code/tools/coverage.py
Bash timeout 600 python3 tools/tiling.py --year 1912 2>&1 | tail -4 && timeout 300 python3
tools/coverage.py --year 1912 2>&1 | tail -2; echo "=== render ==="; ta...
Read /home/user/claude-code/outputs/1912/coverage.png
```

CLAUDE 30 Aug 2026, 23:21 UTC

Most gaps are hairline slivers along seams; the one that matters is the hatched wedge near units 71/73. Let me classify them so the number isn't misleading, and confirm the render is still alive.

2 TOOL CALLS

```
Bash ps aux | grep -E "render\.py|publish\.py" | grep -v grep | awk '{print $2, $3"%cpu",
$4"%mem", $11, $12, $13, $14, $15, $16}'
Edit /home/user/claude-code/tools/tiling.py
```

CLAUDE 30 Aug 2026, 23:21 UTC

Render is healthy (159% CPU, 7.5 GB). Finishing the gap classification:

3 TOOL CALLS

Edit /home/user/claude-code/tools/tiling.py

Edit /home/user/claude-code/tools/tiling.py

Bash timeout 600 python3 tools/tiling.py --year 1912 2>&1 | tail -12

CLAUDE 30 Aug 2026, 23:21 UTC

8 genuine unmapped holes — the largest a strip roughly 173 ft × 1,480 ft. Critical question: is that ground missing from the *sources*, or just unclaimed by the ownership polygons even though a sheet covers it? Let me check.

1 TOOL CALL

Bash timeout 600 python3 -c " import sys, json; sys.path.insert(0,'tools') import numpy as np from recipelib import Recipe from shapely.geometry import Polygon, Po...

CLAUDE 30 Aug 2026, 23:22 UTC

Clear diagnosis, and it splits cleanly:

- **The two big holes are true source gaps** — no sheet's printed extent covers that ground at all.
- **The six smaller holes are ownership bugs** — sheets 16/67, 68/69, 77/78, 75/76, 67/68 and 24 *do* cover that ground, but no polygon claims it, so it renders as a white gash.

The second class is safely fixable (it decides which sheet writes those pixels; it doesn't touch pixels). Let me fix it.

2 TOOL CALLS

Write /home/user/claude-code/tools/fillgaps.py

Bash timeout 900 python3 tools/fillgaps.py --year 1912 2>&1 | tail -25

CLAUDE 30 Aug 2026, 23:22 UTC

Stricter than my centroid check — it requires a neighbour to cover $\geq 98\%$ of the hole. Six are clear wins. Applying those and verifying the tiling afterward:

1 TOOL CALL

Bash timeout 900 python3 tools/fillgaps.py --year 1912 --apply 2>&1 | tail -3 && echo "=== re-audit ===" && timeout 600 python3 tools/tiling.py --year 1912 2>&1 |...

CLAUDE 30 Aug 2026, 23:23 UTC

Gaps went 14 → 11 with no new overlaps (single-writer intact). The dominant issue is now the two big source gaps (~93% of remaining gap area). The actionable question: is a sheet covering that ground sitting unplaced in the inventory?

1 TOOL CALL

Bash timeout 300 python3 -c " import json,re inv=json.load(open('outputs/1912/recipe/inventory.json')) units=json.load(open('outputs/1912/recipe/units.json'))['un...

CLAUDE 30 Aug 2026, 23:23 UTC

Significant finding. Let me confirm it precisely.

1 TOOL CALL

```
Bash timeout 300 python3 -c " import json from shapely.geometry import shape
g=json.load(open('outputs/1912/qc/gaps.geojson')) f=max(g['features'], key=lambda x:x...
```

CLAUDE 30 Aug 2026, 23:24 UTC

Definitive: the largest gap is 3437×5684 px sitting exactly between unit 71 (centre $x=16350$) and unit 73 ($x=24402$) — and a 1912 sheet is ~ 3287 px wide. **That gap is the footprint of sheet 72, which is scanned and in the inventory but was never placed.** So 1912 is not 92/92; it's missing a sheet. Let me check the render's progress.

1 TOOL CALL

```
Bash ps -o pid,etime,pcpu,pmem,rss,args -p $(pgrep -f "render.py --year 1912" | head -1)
2>/dev/null; echo "--- flat file ---"; ls -la work/city/ 2>/dev/null | ta...
```

CLAUDE 30 Aug 2026, 23:24 UTC

The render is alive (7.4 GB, 159% CPU) but slow. Let me record the sheet 72 finding and commit the QC tooling while it works.

2 TOOL CALLS

```
Edit /home/user/claude-code/HUMAN_QUEUE.md
Bash git add -A tools/ HUMAN_QUEUE.md outputs/1912/qc/ outputs/1912/coverage.png
outputs/1912/recipe/seams/ownership_city.json 2>/dev/null; git status --short | h...
```

CLAUDE 30 Aug 2026, 23:24 UTC

The staged `seam_metrics.*` came from a truncated debug run with numbers I decided aren't trustworthy — committing them as QC would be misleading. Let me keep the one crop that documents *why*, and explain it.

4 TOOL CALLS

```
Bash git rm -q --cached outputs/1912/qc/seams/seam_metrics.csv
outputs/1912/qc/seams/seam_metrics.json 2>/dev/null; rm -f
outputs/1912/qc/seams/seam_metrics.csv o...
Write /home/user/claude-code/outputs/1912/qc/seams/README.md
Bash git add -A outputs/1912/qc tools/ HUMAN_QUEUE.md outputs/1912/coverage.png
outputs/1912/recipe/seams/ownership_city.json && git commit -q -m "1912 Stage 4 ti...
Bash ps -o pid,etime,pcpu,rss,args -p $(pgrep -f "publish.py --year 1912" | tail -1) 2>/dev/null
| tail -2; echo "--- output so far ---"; cat "/tmp/claude-0/-home...
```

CLAUDE 30 Aug 2026, 23:25 UTC

The publish process is gone and no output file was written. Let me find out what happened.

1 TOOL CALL

```
Bash wc -c "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/bb0d6upbg.output"; echo "=== full tail ==="; tail -20 "/tmp/claude-0/-...
```

CLAUDE 30 Aug 2026, 23:26 UTC

The render was OOM-killed. The cause is in `render.py`'s writer: `cv2.cvtColor(canvas, ...)` makes a full second copy of the 2.8 GB canvas, then PIL makes more — on top of the 7.4 GB already held. Let me fix the tool to stream the write via libvips instead.

3 TOOL CALLS

Edit /home/user/claude-code/tools/render.py

Edit /home/user/claude-code/tools/render.py

Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad;
timeout 900 python3 tools/render.py --year 1912 --all --downsample 16...

CLAUDE 30 Aug 2026, 23:26 UTC

Writer works — tiled, pyramidal TIFF. Let me confirm the colour channels aren't swapped.

1 TOOL CALL

Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/t16.jpg

USER 30 Aug 2026, 23:26 UTC

[Image: original 2546x5686, displayed at 895x2000. Multiply coordinates by 2.84 to map to original image.]

CLAUDE 30 Aug 2026, 23:26 UTC

Colours correct (Sanborn pink brick / yellow frame), and the sheet-72 hole is visible on the right. Relaunching the full 1/2 pipeline with the fixed writer, unbuffered this time so I can watch it.

1 TOOL CALL

Bash nohup python3 -u tools/publish.py --year 1912 --downsample 2 > /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/pub1912.l...

CLAUDE 30 Aug 2026, 23:27 UTC

The pipeline is running (and the sheet list confirms 72 is absent: ...71, 73...). Let me verify my HQ-8 claim numerically before it stands as a finding — the space between 71 and 73 may be wider than one sheet.

1 TOOL CALL

Bash timeout 300 python3 -c " import sys, json; sys.path.insert(0,'tools') import numpy as np from recipelib import Recipe from shapely.geometry import Polygon r=R...

CLAUDE 30 Aug 2026, 23:27 UTC

Important correction to my own finding — and good that I checked. Let me get this exactly right.

1 TOOL CALL

Bash timeout 300 python3 -c " import sys, json; sys.path.insert(0,'tools') import numpy as np from recipelib import Recipe from shapely.geometry import Polygon, sh...

CLAUDE 30 Aug 2026, 23:28 UTC

My geometric argument in HQ-8 was wrong and I need to correct it: I compared the hole's width in *mosaic* pixels against a sheet's width in *native* pixels. A 1912 sheet is ~6,500 px wide in the mosaic frame, not 3,287. The conclusion survives via the inventory check, but the reasoning must be right.

2 TOOL CALLS

Edit /home/user/claude-code/HUMAN_QUEUE.md

Bash tail -2 /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/pub1912.log; echo "---"; grep -c composited /tmp/claude-0/-home...

CLAUDE 30 Aug 2026, 23:28 UTC

35 of 92 sheets composited, memory stable at 8 GB. Let me commit the writer fix and the HQ-8 correction while it works.

2 TOOL CALLS

Bash git add tools/render.py HUMAN_QUEUE.md && git commit -q -m "Fix OOM in render.py's writer; correct HQ-8's geometric argument The 1912 city render at 1/2 was ...

Bash until [-f work/city/1912_150ppi.tif] || ! pgrep -f "publish.py --year 1912" >/dev/null; do sleep 20; done; echo "render phase done"; tail -3 /tmp/claude-0/...

CLAUDE 30 Aug 2026, 23:31 UTC

The writer fix worked — 123 MB tiled pyramidal TIFF at 20370×45493. COG and DZI are running now.

2 TOOL CALLS

Bash until ! pgrep -f "publish.py --year 1912" >/dev/null; do sleep 30; done; tail -20 /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9data-a56890968b0c/...

Bash tail -6 /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/pub1912.log; echo "--- outputs ---"; ls -la outputs/1912/mosaic/...

CLAUDE 30 Aug 2026, 23:41 UTC

1912 published: COG (828 MB, opens in both gdalinfo and vips) and a 19-level DeepZoom pyramid. Let me check what's actually deliverable, since GitHub caps files at 100 MB and this container is ephemeral.

1 TOOL CALL

Bash du -sh outputs/1912/tiles/1912_files; find outputs/1912/tiles/1912_files -type f | wc -l; cat outputs/1912/tiles/1912.dzi; echo; ls outputs/1912/tiles/1912_f...

CLAUDE 30 Aug 2026, 23:41 UTC

DZI is 176 MB across 19,576 small files — committable. The 828 MB COG exceeds GitHub's 100 MB file cap, so it can't be. Let me make viewable deliverables for you first.

2 TOOL CALLS

Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; timeout 600 vips resize work/city/1912_150ppi.tif \$S/1912_city_view.j...

Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/1912_downtown_detail.jpg

USER 30 Aug 2026, 23:42 UTC

[Image: original 2400x2000, displayed at 2000x1667. Multiply coordinates by 1.20 to map to original image.]

CLAUDE 30 Aug 2026, 23:42 UTC

That detail is fully legible at 150 ppi — but it also shows a real defect. **"AVE. F OR CHURCH" is printed twice**, about 30 ft apart, with a visible vertical seam between the two copies. That's the duplicated-street-name failure REPORT.md claims is at zero. Let me identify the sheets involved.

1 TOOL CALL

```
Bash timeout 300 python3 -c " import sys; sys.path.insert(0,'tools') from recipelib import Recipe from shapely.geometry import Polygon, box r=Recipe(1912) # crop w...
```

CLAUDE 30 Aug 2026, 23:42 UTC

The duplication is at the **75|76 seam** — two tie-placed outer sheets — with the labels ~60 ft apart. That also tempers my earlier conclusion: where sheets *do* share map content, the photometric metric would catch exactly this. Let me run the full sweep as a screening pass.

2 TOOL CALLS

```
Bash nohup python3 -u tools/seamqc.py --year 1912 --win 256 --inset 60 --crops 16 > /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scra...
```

```
Bash head -6 /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/seamqc1912.log; echo "..."; grep -A20 "worst seams" /tmp/claude-...
```

CLAUDE 30 Aug 2026, 23:44 UTC

Decisive: the sweep reports **"no-signal" for 75|76** — the one seam I visually confirmed is broken — while flagging ~96 px on frozen core seams that printed correctly. So the metric has both false negatives and false positives and cannot be a gate. Let me capture the real defect as evidence instead.

2 TOOL CALLS

```
Bash mkdir -p outputs/1912/qc/human; timeout 300 vips crop work/city/1912_150ppi.tif outputs/1912/qc/human/HQ9_75-76_duplicated_street_label.jpg 5850 36050 550 85...
```

```
Read /home/user/claude-code/outputs/1912/qc/human/HQ9_75-76_duplicated_street_label.jpg
```

CLAUDE 30 Aug 2026, 23:44 UTC

Unmistakable — "F OR CHURCH" is drawn twice, ~86 ft apart, with a hard seam between the copies. That's a gross misplacement by the brief's own rubric. Let me check how widespread this is by sampling several seams from the render, including a frozen-core one as a control.

2 TOOL CALLS

```
Bash timeout 600 python3 -c " import sys, subprocess; sys.path.insert(0,'tools') from recipelib import Recipe from shapely.geometry import Polygon r=Recipe(1912) o...
```

```
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/seamview_9-10.jpg
```

CLAUDE 30 Aug 2026, 23:44 UTC

The frozen-core 9|10 seam is clean — "AVE. C OR MECHANIC" appears once, lot numbers run continuously across, no doubled lines. That confirms the core is sound and the metric's ~96 px on core seams was a false positive. Now the outer ring:

3 TOOL CALLS

Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/seamview_79-80.jpg

Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/seamview_57-58.jpg

Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/seamview_83-84.jpg

CLAUDE 30 Aug 2026, 23:45 UTC

A clear pattern. My visual sample: the frozen core seam (9|10) is clean, but **all four outer-ring seams show visible defects** — 75|76 duplicates a street label by ~86 ft, 57|58 runs a diagonal cut straight through a building, 79|80 steps the street, 83|84 doubles a letter. Let me preserve the evidence and write it up.

2 TOOL CALLS

Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; cp \$S/seamview_9-10.jpg outputs/1912/qc/human/HQ9_09-10_core_control_...

Bash cd outputs/1912/qc/seams && rm -f seam_14-55_100.png seam_32-37_100.png seam_47-51_100.png seam_51-52_100.png seam_51-53_100.png seam_52-57_100.png seam_53-5...

CLAUDE 30 Aug 2026, 23:45 UTC

Now let me update the seam QC README with the decisive false-negative evidence:

2 TOOL CALLS

Edit /home/user/claude-code/outputs/1912/qc/seams/README.md

Edit /home/user/claude-code/HUMAN_QUEUE.md

CLAUDE 30 Aug 2026, 23:46 UTC

Now correcting the REPORT claim this contradicts:

10 TOOL CALLS

Bash grep -n "Duplicated street name" REPORT.md

Bash cd /home/user/claude-code && grep -n "Duplicated street name" REPORT.md

Edit /home/user/claude-code/REPORT.md

Edit /home/user/claude-code/REPORT.md

Edit /home/user/claude-code/STATUS.md

Bash git add -A tools/ outputs/1912/qc HUMAN_QUEUE.md REPORT.md STATUS.md && git status --short | head -12 && git commit -q -m "1912 published at 150 ppi; seam au..."

Bash printf 'outputs/*/mosaic/\noutputs/*/tiles/\n' >> .gitignore && git add .gitignore && git commit -q -m "Ignore generated mosaic/tile output: the COG exceeds ..."

SendUserFile

SendUserFile

Bash git status --short; git log --oneline -6; echo "=== remote sync ==="; test "\$(git rev-parse HEAD)" = "\$(git rev-parse origin/claude/galveston-setup-part-a-mk..."

CLAUDE 30 Aug 2026, 23:47 UTC

1912 is published in the cloud — and the audit turned up two things you need to decide on before it's saleable.

What got built

The full 1912 city now renders end-to-end here: **20,370 × 45,493 px (927 MP) at 150 ppi**, as an 828 MB COG that opens under both `gdalinfo` and `vips`, plus a 19-level DeepZoom pyramid (141 MB, 19,576 tiles). That's the brief's \$5 web deliverable — 150 ppi is exactly its "cap web tiles at ~150 ppi-equivalent." I've sent you the city view and a 1:1 detail; legibility is there ("Galveston Bagging Co.", hydrant and sprinkler notes, individual lot numbers).

Getting there needed a real fix: the first attempt was **OOM-killed**. `render.py`'s writer called `cv2.cvtColor` on the canvas, which copies all 2.8 GB, then handed that to Pillow, which copies again — about 9 GB on top of the 7.4 GB already held. It now streams through libvips off the existing buffer and composites with `cv2.copyTo` instead of boolean indexing. Full 300 ppi is genuinely out of reach here (~22 GB for 1912 at 1:1); `crop.py` already gives print resolution per address, which is what your business actually sells.

Two open findings

HQ-9 — the outer ring is not print-ready; your core is. With a render to look at, I graded seams by eye. The frozen 13-sheet core seam (9|10) is clean — one street label, lot numbers continuous. All four outer-ring seams I sampled are not: **75|76 draws "AVE. F OR CHURCH" twice, ~86 ft apart**, with the cut between the copies, and **57|58 runs a diagonal cut straight through a dwelling** — a direct breach of your non-negotiable that seams follow street centrelines. Commercially: crops from the core footprint your masters cover are saleable today; ring crops should be treated as drafts.

HQ-8 — sheet 72 was never placed. The tiling audit shows the city is one connected piece with essentially zero double-ownership, but leaves ~268 ft of Galveston that no sheet maps, between units 71 and 73. Sheet 72 is the only ordinary city sheet with a scan in the inventory that was never registered — so this is fixable, not a source gap.

I also have to flag that I got HQ-8's reasoning wrong on first commit and corrected it: I compared the hole's width in *mosaic* pixels against a sheet's width in *native* pixels. The conclusion survived the inventory check, but the original argument didn't, and the correction is recorded in the queue.

One thing I built and then threw away: a photometric seam metric. A full 132-pair sweep called 75|76 **"no-signal"** — the worst defect in the mosaic — while flagging ~100 px on core seams that render perfectly. It misses real defects and invents false ones, so I didn't ship its numbers as QC; the reasoning and evidence are in `outputs/1912/qc/seams/README.md`.

The COG and tiles aren't committed — the COG exceeds GitHub's 100 MB per-file cap, and the tile set will be superseded once these two items close. Both regenerate with `python3 tools/publish.py --year 1912`. Everything else is pushed and in sync.

Holding here on 1899 until you say go. When you do, the same pipeline runs against it, and I'd expect a similar core-versus-ring split plus the 71a question already open as HQ-7.

2026-08-31

USER 31 Aug 2026, 00:48 UTC

Okay how do you recommend we accomplish our goal of finishing the 1912 map?

CLAUDE 31 Aug 2026, 00:48 UTC

Good question — let me ground the recommendation in what actually went wrong rather than guess. Two quick checks: whether the bad seams correlate with how those units were placed, and what control data the ring had available.

1 TOOL CALL

```
Bash python3 -c " import json, collections
g=json.load(open('outputs/1912/recipe/sheets_city.geojson')) tier={str(f['properties']['unit']):
(f['properties']['tier...
```

CLAUDE 31 Aug 2026, 00:48 UTC

Now the control data — this is the crux of what to recommend.

1 TOOL CALL

```
Bash python3 -c " import json for y in ('1899','1912'):
g=json.load(open(f'outputs/{y}/recipe/grid.json')) st=sorted(int(k) for k in g['streets']);
av=sorted(int(...
```

CLAUDE 31 Aug 2026, 00:49 UTC

Found the root cause. Let me check one thing that decides between two routes — whether this container can reach OpenStreetMap.

2 TOOL CALLS

```
Bash timeout 60 curl -s -o /dev/null -w "overpass:%{http_code} " "https://overpass-
api.de/api/interpreter?data=[out:json];node(29.30,-94.80,29.31,-94.79);out%201;...
Bash timeout 45 curl -sS -o /dev/null -w "overpass:%{http_code}\n" "https://overpass-
api.de/api/status" 2>&l | tail -2; timeout 45 curl -sS -o /dev/null -w "osm-t...
```

CLAUDE 31 Aug 2026, 00:49 UTC

Outbound is blocked at the proxy (403 on OSM, Nominatim, Census, even tile.loc.gov) — only the git mirror works. That rules out georeferencing to modern coordinates from here. Let me confirm the in-repo precedent for the route that remains.

1 TOOL CALL

```
Bash head -30 tools/indexes_1899.py; echo "..."; grep -n "def \|corridor\|profile\|street"
tools/indexes_1899.py | head -20
```

CLAUDE 31 Aug 2026, 00:50 UTC

Here's my read, grounded in what I just checked.

The ring has one root cause, not many

1912's corridor grid covers **9 streets (18-26) and 11 avenue slots — 99 intersections**. That's the downtown core only. 1899, by contrast, has **41 streets (6-46), 28 avenue slots, 1,148 intersections**, island-wide, and its `indexes_1899.py` says why: the 1899 mosaic frame *is* a ground lattice, exact by construction, so every sheet gets fitted into an absolute reference.

1912 has no such frame. Its mosaic is anchored to one sheet (archival sheet 10), so outside the core every sheet was placed relative to its neighbours, and the error compounds outward with nothing to pin it. The placement tiers show the consequence: **43 of 92 units (47%) were**

placed by translation only or from a single tie — `ties-trans(n=2/2)` and `ties-single(nbr=21)` — meaning no rotation or scale was solved for them at all. Even the 33 properly-fitted units carry `med=18.4 px` residuals.

That predicts exactly what I saw: both units at the worst seam (75|76, the ~86 ft duplicate) are `tie-translation`; the building split at 57|58 involves a `tie-single`. And 79|80 stepping ~10 ft with *both* units in the "good" fit tier tells you the fit tier isn't clean either.

So patching individual bad seams would be whack-a-mole. The ring needs absolute control.

What I recommend

Give 1912 the ground lattice 1899 already has, then re-fit the ring into it. Concretely, in order:

1. **Establish the lattice.** Fit a regular street/avenue lattice to the frozen core (whose 99 intersections and scale are proven and match your master), then extend it across the island. Galveston's grid is regular enough for this — it's what makes 1899's index exact. One caveat from the 1899 work: there's a documented **~2° grid bend in the south**, so the lattice has to accommodate that rather than assume one rigid orientation.
2. **Re-solve the 43 weak units** against the lattice — a real similarity fit (rotation, scale, translation) per sheet against ≥ 4 corridor intersections, replacing translation-only placements. Keep the 12 core sheets frozen; they're proven in print.
3. **Place sheet 72 (HQ-8).** Once the lattice exists this is just one more sheet to solve, and its scan is already downloaded and hash-pinned.
4. **Re-cut the seams along street centrelines.** 57|58's diagonal cut through a dwelling shows the outer cuts aren't following streets as your §2.5 requires — that's a cut-geometry bug independent of registration, and it needs the corridor index to fix properly.
5. **Re-render and re-audit:** the tiling gate should drop toward zero gaps, then a full visual pass over all 132 seams rather than my 5-seam sample.

Step 1 gates everything else, and it's the step I'd start with.

Two things I need from you

Interim scope. Your core — the same footprint your two 27×40 masters cover — is clean and works today via `lookup.py / crop.py`. You could sell core crops now and treat the ring as in-progress, rather than waiting for the full city. Worth deciding, because it changes the urgency of the above.

Georeferencing is blocked here. The brief wants an EPSG:3857 COG, and `lookup.py` can't return lat/lng, because both need modern control points. This container's proxy refuses OpenStreetMap, Nominatim, Census, even `tile.loc.gov` — only the git mirror works. So tying the mosaic to real-world coordinates needs either a session with a different network policy, or the CI-workflow trick a previous session used to fetch sources. Everything in steps 1–5 is self-contained and works without it.

Say the word and I'll start on the lattice.

The map of all of galveston for each year can be bigger that 27x40 just an fyi

CLAUDE 31 Aug 2026, 00:54 UTC

Good to know — that changes the framing, so let me get the actual numbers rather than assume.

```
1 TOOL CALL
Bash python3 -c " ext={'1899':((-3143,6902),(27218,54469),1006/262.0),'1912':((-9273,-38730),(31468,52256),5.7966)} print('%-6s %-22s %-20s %s' % ('year','mosaic ...
```

CLAUDE 31 Aug 2026, 00:55 UTC

Now let me make the scale decision evidence-based rather than guess where lettering fails.

```
2 TOOL CALLS
Bash timeout 900 python3 - <<'EOF' from PIL import Image, ImageDraw, ImageFont import subprocess, os Image.MAX_IMAGE_PIXELS=None S='/tmp/claude-0/-home-user-claud...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/legibility_1912.jpg
```

USER 31 Aug 2026, 00:55 UTC

[Image: original 1250x5322, displayed at 470x2000. Multiply coordinates by 2.66 to map to original image.]

CLAUDE 31 Aug 2026, 00:56 UTC

That settles the scale question with evidence. Let me keep it with the project.

```
2 TOOL CALLS
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; ls -la $S/legibility_1912.jpg; cp $S/legibility_1912.jpg outputs/1912...
SendUserFile
```

CLAUDE 31 Aug 2026, 00:56 UTC

That changes the plan in a useful way — and one number reframes it.

Even a much bigger sheet can't reach native scale

The 1912 city covers 7,028 × 15,696 ft (1.3 × 3.0 miles). At Sanborn's native 50 ft/in that's a **11.7 × 26.2 foot** print. So the full-city map is inherently a *reduced-scale* document at any sellable size; lot-level detail stays with `crop.py` . Here's the tradeoff:

ft per inch	1912 print size	what stays readable
50 (native)	141 × 314 in	everything, incl. the fine descriptive notes
100	70 × 157 in	lot numbers, most notes
150	47 × 105 in	lot numbers, marginal
250	28 × 63 in	street names, block numbers
392	18 × 39 in	street names only — this is what 27×40 forces

The legibility strip I sent is where those cutoffs come from, not my guesswork.

The useful surprise: the render I already built *is* the print master for the big format. At 2.9 px/ft it works out to **68 × 152 inches at ~104 ft/in, 300 ppi** — a wall-sized city print with lot numbers still readable. You don't need a higher-resolution render for that product; it exists today.

What it does and doesn't change about finishing 1912

It splits the work into two milestones instead of one, because **precision requirements scale with print size**:

At 250–400 ft/in, two of the four defects I found simply vanish — 79|80's 10 ft street step and 83|84's doubled letter are ~0.03 in on paper. But look at the strip: **"AVE. F OR CHURCH" is printed twice in all six panels**, including the smallest. Duplications and split buildings are topological, not fine-detail, so they survive any reduction. Same for the sheet-72 hole — a 268 ft gap is 0.7 in even at 392 ft/in.

So:

Milestone 1 — a sellable city overview, much cheaper than I proposed yesterday. Fix only the gross errors: place sheet 72, re-cut the seams that split buildings, and re-solve the ~43 translation-only and single-tie units enough to kill duplications. Skip the precision work. That yields a 28×63 in overview (street and block level), and plausibly the 68×152 in wall print.

Milestone 2 — lot-level. The full lattice re-solve I described, which is what quadrant prints at 100–150 ft/in and trustworthy ring crops actually require.

My earlier recommendation front-loaded Milestone 2 for everything. Given prints can be large, Milestone 1 is the better first target: it's a smaller fix, it produces something you can sell, and nothing in it is wasted — it's a strict subset of the lattice work.

One caveat worth naming: at 68 × 152 inches you're into panelled or roll printing, and my rendering ceiling here is ~1 gigapixel (the container OOMs above that). The existing 927 MP file sits right at that edge, so anything finer than ~100 ft/in for the *whole* city needs a strip-wise renderer or a bigger machine — though per-quadrant renders sidestep that entirely.

Want me to start Milestone 1 with sheet 72, since it's the one defect that's purely additive and its scan is already on disk?

USER 31 Aug 2026, 00:57 UTC

2

CLAUDE 31 Aug 2026, 00:57 UTC

Reading "2" as Milestone 2 — the full lattice re-solve for lot-level quality. That's the thorough path, and Milestone 1's fixes fall out of it. Let me start with the foundation: measuring how regular 1912's core grid actually is, and what per-sheet coverage metadata exists to place ring sheets into a lattice.

2 TOOL CALLS

```
Bash python3 -c " import json, numpy as np g=json.load(open('outputs/1912/recipe/grid.json'))
print('frame:', g.get('frame')) print() st={int(k):v for k,v in g['s...
Bash python3 -c " import json, collections u=json.load(open('outputs/1912/recipe/units.json'))
['units'] have_st=sum(1 for v in u.values() if v.get('streets')) hav...
```

CLAUDE 31 Aug 2026, 00:58 UTC

Two things stand out, and the second is potentially serious. The core grid is regular (avenue pitch 2031 px, street pitch 2316 px, residuals ~1-2%), **but those pitches imply block sizes 32% larger than 1899's for the same city.** That points at `px_per_ft` for 1912 being wrong. Let me measure it directly off the map, which prints its street widths.

1 TOOL CALL

```
Bash python3 -c " import json g=json.load(open('outputs/1912/recipe/grid.json')) av=
{int(k):v['x'] for k,v in g['avenues'].items()} st={int(k):v['y'] for k,v in g...
```

CLAUDE 31 Aug 2026, 00:58 UTC

The 7.753 value reproduces 1899's 262 ft / 304 ft block for the same city. But 1899's own figure is an assumption, so let me verify against the map itself — Sanborn prints street widths in feet.

2 TOOL CALLS

```
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad;
timeout 300 vips crop work/city/1912_150ppi.tif $S/scalecheck.jpg 572...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/scalecheck.jpg
```

CLAUDE 31 Aug 2026, 00:59 UTC

The cross-year comparison is inconclusive because both years' `px_per_ft` rest on assumptions. I need one decisive on-map measurement — Sanborn prints street widths in feet. Let me crop a labelled street with a pixel ruler so I can measure it exactly.

2 TOOL CALLS

```
Bash timeout 600 python3 - <<'EOF' from PIL import Image, ImageDraw, ImageFont import subprocess
Image.MAX_IMAGE_PIXELS=None S='/tmp/claude-0/-home-user-claude-co...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/ruler.jpg
```

CLAUDE 31 Aug 2026, 01:00 UTC

Let me measure this numerically rather than by eye — the "80" street label gives a direct calibration.

1 TOOL CALL

```
Bash timeout 300 python3 - <<'EOF' import numpy as np from PIL import Image
Image.MAX_IMAGE_PIXELS=None S='/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36...
```

CLAUDE 31 Aug 2026, 01:01 UTC

My threshold was too crude — thin ink lines don't move a 200-px mean. Let me detect the lines by dark-pixel fraction instead.

1 TOOL CALL

```
Bash timeout 300 python3 - <<'EOF' import numpy as np from PIL import Image
Image.MAX_IMAGE_PIXELS=None S='/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36...
```

CLAUDE 31 Aug 2026, 01:01 UTC

Good — that settles it for 1912: the corridor between property lines measures ~221 render px for a street the map labels **80'**, giving ~5.5 mosaic px/ft against recipilib's 5.7966. So 1912's constant is right (within ~5%), and the pitches mean blocks of ~270 × 320 ft plus 80 ft streets — correct for Galveston.

That makes **1899's constant the suspect one**: it assumes "avenue pitch ≈ 262 ft", but 262 ft is the block face, not the centre-to-centre pitch. Let me measure 1899 the same way.

2 TOOL CALLS

```
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad;
timeout 900 python3 tools/render.py --year 1899 --rect 300 26300 1700...
Bash timeout 300 python3 - <<'EOF' import numpy as np, subprocess from PIL import Image,
ImageDraw, ImageFont Image.MAX_IMAGE_PIXELS=None S='/tmp/claude-0/-home-u...
```

CLAUDE 31 Aug 2026, 01:02 UTC

That rect landed inside a block. Let me look at it to find a labelled street.

1 TOOL CALL

```
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/ruler1899.jpg
```

USER 31 Aug 2026, 01:02 UTC

[Image: original 2800x2400, displayed at 2000x1714. Multiply coordinates by 1.40 to map to original image.]

CLAUDE 31 Aug 2026, 01:03 UTC

The measurements disagree between years, so the constants can't both be right — and `crop.py` uses them to size print crops. Rather than keep eyeballing, let me calibrate against the authority the brief names: your masters, which are proven in print at 50 ft/in.

2 TOOL CALLS

```
Bash ls -la inputs/masters/; which pdftop pdfimages pdftoppm 2>/dev/null; echo "---"; timeout
120 pdftop inputs/masters/Galveston_1912_Wharf_Downtown_27x40_Mast...
Bash cd /tmp && timeout 300 python3 -c " import pypdf, sys " 2>&1|tail -1; timeout 300 pip
install --no-cache-dir --quiet pypdf 2>&1 | grep -vi "root" | tail -2; ...
```

CLAUDE 31 Aug 2026, 01:04 UTC

Masters confirmed: single page, 39.39 × 26.55 in, at 300.0 and 304.6 ppi. At the brief's 50 ft/in that's ~6.0 px/ft — an absolute reference. Let me measure the block pitch directly in your proven master.

1 TOOL CALL

```
Bash timeout 900 python3 - <<'EOF' import numpy as np, pypdf from PIL import Image
Image.MAX_IMAGE_PIXELS=None S='/tmp/claude-0/-home-user-claude-code/20717cf2-d7...
```

CLAUDE 31 Aug 2026, 01:05 UTC

Both masters agree, and one number is decisive: the 1899 master's block pitch is **1017 px** while the 1899 *mosaic's* avenue pitch is **1006 px** — essentially identical. So the mosaic frame is at the master's own scale (~6 px/ft), not the 3.84 that `reciplib` uses. Let me confirm against the recipe's own print manifests rather than infer.

1 TOOL CALL

```
Bash python3 -c " import json for f in
('print_composition','render_manifest','master_full_manifest'):
d=json.load(open(f'outputs/1912/recipe/{f}.json')) s=json.d..."
```

CLAUDE 31 Aug 2026, 01:06 UTC

`print_composition.json` is the key: the master was rendered from the mosaic at **scale 0.5163 at 300 dpi**, which puts the mosaic frame at ~11.6 px/ft — twice `reciplib`'s 5.7966. Let me confirm by measuring the same block pitch in my render as in the master.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import numpy as np from PIL import Image
Image.MAX_IMAGE_PIXELS=None import subprocess S='/tmp/claude-0/-home-user-claude-code/...
```

CLAUDE 31 Aug 2026, 01:06 UTC

That autocorrelation locked onto lot spacing, not block pitch — too fragile. Let me run the decisive test instead: ask `crop.py` for a crop at the brief's native 50 ft/in and measure whether it actually comes out at 50 ft/in.

1 TOOL CALL

```
Bash S='/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad;
timeout 900 python3 tools/crop.py --year 1912 --street 22 --avenue St...
```

CLAUDE 31 Aug 2026, 01:07 UTC

`crop.py` assumed 2319 mosaic px = 400 ft. Let me measure what the output actually contains.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import numpy as np from PIL import Image
Image.MAX_IMAGE_PIXELS=None S='/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36...
```

CLAUDE 31 Aug 2026, 01:07 UTC

The crop is too small for autocorrelation, but `print_composition.json` records the exact mosaic rectangle the master covers — that gives the answer arithmetically.

1 TOOL CALL

```
Bash python3 -c " import json d=json.load(open('outputs/1912/recipe/print_composition.json'))
for k in ('map_rect_canvas_xyxy','map_rect_mosaic_xyxy','resample','...
```

CLAUDE 31 Aug 2026, 01:08 UTC

Now it resolves cleanly, and the answer is the opposite of my first guess. `map_rect_canvas_xyxy` and `map_rect_mosaic_xyxy` are **identical in size (22882 × 14489)** — the mosaic frame *is* the print frame at 1:1. That page is 40 in wide, so 22882 px / 40 in ÷ 5.7966 px/ft = **~99 ft/in**. So

1912's constant is correct, and your master prints at ~100 ft/in, not the 50 ft/in the brief states.

That makes **1899** the broken one. Let me verify and fix it.

1 TOOL CALL

```
Bash python3 -c " # 1912 is verified: pitch 2031.3 px / 5.7966 px-per-ft = ground pitch av_ft = 2031.3/5.7966; st_ft = 2315.7/5.7966 print(f'1912 (verified scale)...'
```

CLAUDE 31 Aug 2026, 01:08 UTC

Three independent checks agree (avenue pitch 2.871, street pitch 2.926, and an 80'-labelled street measuring 84 ft rather than an implausible 63 ft). Fixing it:

4 TOOL CALLS

```
Read /home/user/claude-code/tools/reciplib.py
```

```
Read /home/user/claude-code/tools/reciplib.py
```

```
Edit /home/user/claude-code/tools/reciplib.py
```

```
Bash timeout 300 python3 tools/lookup.py --year 1899 --address "2314 Strand St, Galveston, TX" 2>&1 | sed -n '3,7p'; echo "=== 1912 (unchanged, control) ==="; tim...
```

CLAUDE 31 Aug 2026, 01:09 UTC

Now both years agree for the same city — 1899 blocks read 347×403 ft against 1912's $344\text{--}358 \times 399$ ft. Before the fix 1899 read 262×304 ft. Committing:

1 TOOL CALL

```
Bash cat >> HUMAN_QUEUE.md <<'EOF' ## HQ-10 · 1899 crops printed at the wrong scale – FIXED; and the masters are ~100 ft/in, not 50 Found while establishing groun...
```

CLAUDE 31 Aug 2026, 01:10 UTC

Committed as `18377b3`. Here's where that landed.

I started Milestone 2 and hit something that had to be fixed first

A lattice is meaningless without a verified foot, so I began by pinning down ground units — and found that **1899 crops have been printing 34% off scale**.

`reciplib.px_per_ft` assumed 1899's avenue pitch was 262 ft. That's the *block face*, not the centre-to-centre pitch. So every 1899 crop covered $1.34\times$ the ground requested and printed at $1.34\times$ the requested ft/in — asking for 100 ft/in got you ~134. That's precisely the ruler-on-the-scale-bar check in your Stage 6 gate, failing silently. If you've cut any 1899 prints from this tooling, they're off scale.

Fixed to 2.8985 px/ft, checked three independent ways: avenue pitch gives 2.871, street pitch 2.926, and a street the sheet itself labels **80'** measures 84 ft at the new value versus an implausible 63 ft at the old one. The confirmation I trust most is that `lookup.py` now reports 1899 blocks as **347×403 ft** against 1912's **$344\text{--}358 \times 399$ ft** — the same city, finally agreeing. Before the fix, 1899 said 262×304 ft.

1912 is correct and I left it alone. The proof is your own master: `print_composition.json` records `map_rect_canvas_xyxy` and `map_rect_mosaic_xyxy` as both 22882×14489 , so the mosaic frame *is* the print frame at 1:1.

One thing about your masters you should know

That same manifest puts 22,882 mosaic px across a 40-inch page, which works out to **~99 ft/in — your masters print at about 100 ft/in, not the 50 ft/in the brief states**. They're a half-scale reduction of the sheets' native drawing scale. `crop.py` already defaults to `--scale-ft-per-in 100`, which matches your masters rather than the brief. Worth deciding which you want as the default, because a crop requested at 50 ft/in comes out at twice the size of your existing product.

I also owe you a correction from yesterday: my 1899 print-size figures used the bad constant and were **34% too small**. The 1899 city is $10,475 \times 16,411$ ft, so at 100 ft/in it's 105×164 in, not the 79×124 in I quoted. The 1912 numbers I gave you were correct.

Next

The lattice itself — fit a regular street/avenue lattice to the frozen core, extend it across the island allowing for the $\sim 2^\circ$ southern bend, then re-solve the 43 translation-only and single-tie units against it, which also places sheet 72. That's the actual Milestone 2 work and it now has trustworthy ground units under it.

I'll start on that unless you'd rather I first settle the 50-vs-100 ft/in default, since that one's a product call only you can make.

USER 31 Aug 2026, 01:29 UTC

Let's finish one map first

CLAUDE 31 Aug 2026, 01:30 UTC

Staying on 1912 then — no context-switching to 1899. Let me get to the actual lattice work.

The key enabler: current placement errors (~ 10 – 90 ft) are far smaller than the block pitch (350 ft), so corridors can be snapped to lattice lines unambiguously. First let me verify I can actually detect street corridors in a sheet.

1 TOOL CALL

```
Bash python3 -c " import json, collections u=json.load(open('outputs/1912/recipe/units.json'))
['units'] g=json.load(open('outputs/1912/recipe/sheets_city.geojson'...
```

CLAUDE 31 Aug 2026, 01:30 UTC

All 92 units carry street ranges (75/76 span streets 36–39, for instance), so the correspondence key exists. Now the make-or-break: can I detect the corridors in a sheet image?

1 TOOL CALL

```
Bash timeout 900 python3 - <<'EOF' import sys, numpy as np; sys.path.insert(0,'tools') from
reciplib import Recipe import cv2 r=Recipe(1912) u=r.units def profile...
```

CLAUDE 31 Aug 2026, 01:31 UTC

Detection works cleanly on the core sheet (unit 9: street gaps 1160, 1157, 1191 px — very regular) but fails on the sparser ring sheets. Thresholding is too crude. Let me use the strong prior instead: a periodic comb matched to the known pitch.

2 TOOL CALLS

```
Bash timeout 900 python3 - <<'EOF' import sys, numpy as np; sys.path.insert(0,'tools') from recipilib import Recipe import cv2 r=Recipe(1912); u=r.units WPF=5.7966...
```

```
Bash timeout 900 python3 - <<'EOF' import sys, numpy as np; sys.path.insert(0,'tools') from recipilib import Recipe import cv2 r=Recipe(1912); u=r.units WPF=5.7966...
```

CLAUDE 31 Aug 2026, 01:32 UTC

The comb works on ring sheets but picked a wrong local optimum on the core sheet — because pitch is a city-wide constant, it shouldn't be re-estimated per sheet. Let me build the real tool: fix pitch globally, solve phase per sheet, then jointly adjust sheets and lattice lines with the core frozen.

2 TOOL CALLS

```
Write /home/user/claude-code/tools/lattice.py
```

```
Bash timeout 2400 python3 -u tools/lattice.py --year 1912 2>&1 | tail -30
```

CLAUDE 31 Aug 2026, 01:33 UTC

Converged (per-iteration moves $107 \rightarrow 11 \rightarrow 6 \rightarrow 4.5$ ft), but sheets moved a median of 111 ft — so I must verify this is an improvement, not a regression. The direct test: do sheets 75 and 76 now agree on where the same street is?

2 TOOL CALLS

```
Bash timeout 900 python3 - <<'EOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from recipilib import Recipe import cv2 from PIL import Image, ImageDr...
```

```
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/lattice_75_76.jpg
```

CLAUDE 31 Aug 2026, 01:34 UTC

That's a clear **negative result** — and I'm glad I validated before applying. The lattice fit made 75|76 *worse*: the two sheets now show different ground in the same window. Let me diagnose before reporting.

1 TOOL CALL

```
Bash python3 -c " import json, numpy as np
d=json.load(open('outputs/1912/recipe/corridors.json')) import statistics # gaps between
detected corridors, in native ...
```

CLAUDE 31 Aug 2026, 01:35 UTC

Two key facts: detection is actually excellent (row gaps median **1158** vs expected 1158; columns **1015** vs 1015), so the detector is fine — the failure was purely my index assignment, which snapped through the very transforms I was trying to correct. And the transcribed key maps exist. Let me see if they carry absolute avenue coverage.

1 TOOL CALL

```
Bash python3 -c " import json d=json.load(open('rebuild_1899/out/keymap_1912_SW.json'))
print('top keys:', list(d)[:10]) s=json.dumps(d) print('len', len(s)) prin...
```

CLAUDE 31 Aug 2026, 01:35 UTC

The key maps carry exactly what I was missing — absolute avenue coverage per sheet (e.g. sheet 67: streets 33rd–36th, avenues Strand/B, C, D, E). Let me check how many units they cover.

1 TOOL CALL

```
Bash python3 -c " import json, glob km={} for f in
glob.glob('rebuild_1899/out/keymap_1912_*.json'): for e in json.load(open(f))['results']:
km[str(e['sheet'])]=e..."
```

CLAUDE 31 Aug 2026, 01:35 UTC

This is the missing piece: sheet 75 covers avenues C–F and sheet 76 covers F–I — they share Avenue F, exactly the street that was duplicated. The key map gives absolute correspondence for 91 of 92 units. Let me rewrite the assignment to use it instead of snapping through the bad transforms.

2 TOOL CALLS

```
Edit /home/user/claude-code/tools/lattice.py
Edit /home/user/claude-code/tools/lattice.py
```

CLAUDE 31 Aug 2026, 01:36 UTC

Now the fit loop needs to use the key-map assignment rather than re-snapping:

3 TOOL CALLS

```
Edit /home/user/claude-code/tools/lattice.py
Edit /home/user/claude-code/tools/lattice.py
Bash timeout 2400 python3 -u tools/lattice.py --year 1912 2>&1 | tail -22
```

CLAUDE 31 Aug 2026, 01:36 UTC

Now corrections are absurd (worst 5,454 ft — a mile). The solve converges but into a wrong configuration, so the index assignment is still off. Let me check it against known ground truth: the core sheets, whose street positions `grid.json` already records.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json, glob, re, numpy as np;
sys.path.insert(0,'tools') from recipelib import Recipe r=Recipe(1912) det=json.load(op...
```

CLAUDE 31 Aug 2026, 01:37 UTC

Found it. Streets index perfectly (unit 9 → streets 21,22,23,24 ✓), but avenues are off by one: unit 9's detected corridors sit at slots **−1, 0, 1** while the key map says A, B, C = 0, 1, 2 — the detector picks up a wharf-edge corridor that isn't an avenue, and my ordinal zip blindly shifted everything. The fix is to snap to the absolute index from `grid.json`'s own fit, constrained to the key map's range.

3 TOOL CALLS

```
Edit /home/user/claude-code/tools/lattice.py
Edit /home/user/claude-code/tools/lattice.py
Bash timeout 2400 python3 -u tools/lattice.py --year 1912 2>&1 | tail -20
```

CLAUDE 31 Aug 2026, 01:38 UTC

Better behaved (median 91 ft, worst 333 ft). The test that matters is the same one:

1 TOOL CALL

```
Bash timeout 900 python3 - <<'EOF' import sys, json, numpy as np; sys.path.insert(0,'tools')
from recipelib import Recipe import cv2 from PIL import Image, ImageDr...
```

CLAUDE 31 Aug 2026, 01:38 UTC

Worse again (agreement $0.114 \rightarrow 0.028$). Two different index-assignment strategies both degrade the target seam, so I'm not applying this. Let me confirm the recipe is untouched and record what was learned.

2 TOOL CALLS

```
Bash git status --short; echo "--- transforms_city untouched? ---"; git diff --stat
outputs/1912/recipe/transforms_city.json 2>/dev/null | tail -1; ls outputs/191...
Bash cat >> HUMAN_QUEUE.md <<'EOF' ## HQ-11 · Lattice re-solve attempted for 1912 – NOT APPLIED,
it made seams worse Milestone 2's core step was tried and did not...
```

CLAUDE 31 Aug 2026, 01:39 UTC

I attempted Milestone 2's core step on 1912. **It didn't work, and I did not apply it** — `transforms_city.json` is untouched.

What worked

Corridor detection is solid. A matched comb at the known block pitch finds street and avenue corridors in **88 of 92 sheets**, and the detected spacings come out at a median of **1158 px (streets) and 1015 px (avenues) against expected values of 1158 and 1015**. It reads the real grid even on sparse outer sheets where simple thresholding failed. Those detections are cached in `corridors.json`.

I also found the absolute correspondence I needed: the transcribed key maps give street bounds for all 92 units and avenue bounds for 91 — including that sheet 75 covers avenues C–F and sheet 76 covers F–I, sharing Avenue F, the very street that's duplicated.

What failed

Deciding *which* lattice line each detected corridor is. Three approaches, all validated against seam 75|76:

1. **Snapping through each sheet's current transform** — circular, since that's the thing being corrected. A badly placed sheet snaps a whole block wrong. Sheets moved a median of 111 ft and the seam got visibly worse.
2. **Ordinal matching to the key map** — better founded, but it misassigns whenever detection is imperfect. Core unit 9's corridors sit at avenue slots $-1, 0, 1$ (its leftmost is a wharf edge, not an avenue) while its key map says A, B, C — so everything shifted a block and corrections blew up to 5,454 ft.
3. **Snapping to `grid.json`'s absolute index within the key map's range ± 1** — tamed the blow-up (median move 91 ft), but 75|76's ink agreement still fell from 0.114 to 0.028.

The diagnosis

The weak link is the avenue index model, not the detector. `grid.json` covers only 9 streets and 11 avenue slots, **all inside the core**, so its fitted model gets extrapolated far outside its support — and only 60 of 88 units end up with usable correspondence on both axes.

The most promising next attempt, which I'd take first: rather than snapping every sheet to a global lattice, **solve the shared-corridor equalities between neighbours directly** — the key map says 75 and 76 must agree on Avenue F, so make that an explicit constraint. That's local and much stronger than a global snap, and it doesn't depend on extrapolating the core's index model across the island. Failing that, extend the index outward one ring at a time, re-fitting as each ring is pinned.

I've committed the tool, the cached detections, and the full diagnosis as HQ-11 so this isn't re-attempted blind. I stopped rather than iterate a fourth time because each attempt is a ~40-minute cycle and the third one told me the problem is structural, not a parameter — worth your call whether I keep going down this path or take the neighbour-constraint route next.

USER 31 Aug 2026, 01:41 UTC

Do whatever will recreate what we did in the initial map uploaded

CLAUDE 31 Aug 2026, 01:42 UTC

Right — the core was built by a method that demonstrably works, and I should copy it rather than invent one. Let me find out exactly what that method was.

1 TOOL CALL

```
Bash ls outputs/1912/recipe/controls/; echo "=== one pair file ==="; python3 -c " import json
d=json.load(open('outputs/1912/recipe/controls/pair_10_12.json')) pr...
```

CLAUDE 31 Aug 2026, 01:42 UTC

This is the answer — and it's exactly what I was missing. The original method used **pairwise corridor correspondences between adjacent sheets**, with each one identified by reading printed address runs; there's literally a `why_not_one_block_off` field justified by house numbers ("324 west and 402 east on sheet 10; 323 west and 401 east on sheet 12").

So it never snapped to a global lattice — it enforced *"this line on sheet A is the same line on sheet B."* And the key map can supply that identity for the ring. Let me test it on the worst pair.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json, glob, numpy as np;
sys.path.insert(0,'tools') from recipelib import Recipe r=Recipe(1912); ppf=5.7966 det=json...
```

CLAUDE 31 Aug 2026, 01:43 UTC

390 ft — that's a block off, because each sheet detected only 3 avenue corridors while the key map lists 4, so ordinal picking is ambiguous. The original method resolved that by reading printed address runs; I can resolve it by the content itself. Let me match the shared band directly and let correlation find the true offset.

4 TOOL CALLS

```
Write /home/user/claude-code/tools/pairfit.py
Bash timeout 1200 python3 -u tools/pairfit.py --year 1912 --pair 75 76 2>&1 | tail -12; echo
"=== control: a core pair that is already correct (should shift ~0) =...
Bash until ! pgrep -f "pairfit.py" >/dev/null; do sleep 20; done; cat "/tmp/claude-0/-home-user-
claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/tasks/bf47p6ypd.o...
Bash cat "/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/bf47p6ypd.output" 2>/dev/null | tail -20; echo "--- running? ---"; pgrep...
```

CLAUDE 31 Aug 2026, 02:04 UTC

My brute-force search is far too slow (~142k evaluations). Let me kill it and use FFT cross-correlation, then score only the top candidates properly.

3 TOOL CALLS

```
Bash pkill -f pairfit.py; sleep 1; pgrep -f pairfit.py >/dev/null && echo "still running" ||
echo "stopped"
Edit /home/user/claude-code/tools/pairfit.py
Bash for P in "75 76" "9 10" "83 84"; do set -- $P; echo "### pair $1|$2"; timeout 420 python3 -
u tools/pairfit.py --year 1912 --pair $1 $2 2>&1 | tail -9; done
```

CLAUDE 31 Aug 2026, 02:04 UTC

All three pairs returned the identical shift of -259.5 ft, pinned at the search boundary — including the core pair 9|10, which must be ~0. My FFT correlation isn't normalizing for overlap area. Let me use a properly normalized matched filter and calibrate on the core pair.

2 TOOL CALLS

```
Edit /home/user/claude-code/tools/pairfit.py
Bash for P in "9 10" "10 43" "75 76" "83 84" "57 58"; do set -- $P; timeout 420 python3 -u
tools/pairfit.py --year 1912 --pair $1 $2 2>&1 | python3 -c " import sy...
```

CLAUDE 31 Aug 2026, 02:05 UTC

Still peaking at the search edge — because I zeroed the no-data regions, which then correlate as constant blocks. Let me restrict the window to where **both** sheets actually have data across the whole search range, so there are no invalid pixels to game the correlation.

2 TOOL CALLS

```
Edit /home/user/claude-code/tools/pairfit.py
Bash for P in "9 10" "10 43" "7 8" "75 76" "83 84"; do set -- $P; timeout 420 python3 -u
tools/pairfit.py --year 1912 --pair $1 $2 2>&1 | python3 -c " import sys,...
```

CLAUDE 31 Aug 2026, 02:06 UTC

Now every pair reports "no overlap" — which is itself the finding. Let me quantify how much mapped ground adjacent 1912 sheets actually share.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json, numpy as np; sys.path.insert(0,'tools')
from recipelib import Recipe from shapely.geometry import Polygon r=Re...
```

CLAUDE 31 Aug 2026, 02:06 UTC

That's the decisive measurement, and it explains everything.

1 TOOL CALL

Bash cat >> HUMAN_QUEUE.md <<'EOF' ## HQ-12 · How the master was actually built, and what recreating it costs Answering "do whatever will recreate what we did in ..."

CLAUDE 31 Aug 2026, 02:07 UTC

I found how your master was actually built, and it's neither of the things I'd been attempting.

The method, from your own control files

`outputs/1912/recipe/controls/pair_*.json` records it. For each adjacent pair, the original build identified **one shared corridor** — a street or avenue centreline visible on both sheets — recorded that line's position on each sheet, and constrained the two sheets to agree on it. Crucially, the corridor's *identity* was established by reading the printed address runs. Every control carries a `why_not_one_block_off` field arguing the case. For pair 10|12:

"The 24th St 300/400-block transition happens across this avenue: printed runs '324' west and '402' east on sheet 10; '323' west and '401' east on sheet 12."

Twenty-three such pairs produced the frozen core that prints correctly.

Why there's no shortcut

I measured the shared ground between every adjacent pair, and it's the key number: the shared-scan band has a median width of 494 ft, but 45 of 132 pairs are under 300 ft and 12 are under 100. **The worst seam, 75|76, shares just 47 ft** — narrower than a street.

That killed my area-matching approach (`pairfit.py`): once I correctly excluded no-data regions it reports "no overlap" for exactly these pairs, and before that exclusion it was returning the same bogus shift for a broken pair and a good one alike. There simply isn't a 2-D patch to correlate.

But here's the detail that makes it all make sense: **the clean core pair 9|10 shares only 84 ft too** — and it's fine. Thin overlap isn't what breaks a seam. Having no identified constraint across it is. A 1-D corridor constraint works at 47 ft because it needs only a centreline readable on both sheets, not an area. That's why the original method is the *right* method, not merely the one that happened to be used.

What recreating it costs

132 adjacent pairs, 23 already done. So roughly **109 ring pairs** need the same treatment: identify the shared corridor on both sheets, justify it against the printed address runs, record the line on each, then re-solve the network with the core frozen.

This is per-pair semantic work on the images — precisely the `registration-engineer` role in your brief, one task per pair. The key maps shortcut part of it, since they already name which streets and avenues each sheet covers (all 92 units for streets, 91 for avenues), so the candidate corridor is known before anyone opens a scan. What they can't settle — and what reading the address runs does — is which *detected* line is that corridor when detection misses one. That exact gap caused the one-block error that wrecked my lattice attempt.

I'd start with a pilot of 3–4 ring pairs including 75|76, producing control files in your existing schema, and check that the resulting solve actually closes the duplicated street label before committing to all 109. That keeps the expensive part honest: if the loop works on the pair we know is broken, scaling it is just throughput.

Say go and I'll run the pilot.

USER 31 Aug 2026, 02:09 UTC

Alright create a task for as many opus 5 agents on low effort can be used as possible and finish this map and then stop.

CLAUDE 31 Aug 2026, 02:10 UTC

Understood. Let me build the infrastructure so agents do only the part that needs judgment — the semantic identification — with detection proposing candidates.

2 TOOL CALLS

Write /home/user/claude-code/tools/paircrops.py

Bash timeout 600 python3 tools/paircrops.py --year 1912 --pair 75 76 2>&1 | tail -3; ls -la work/paircrops/75_76/ 2>/dev/null; python3 -c " import json d=json.loa...

CLAUDE 31 Aug 2026, 02:11 UTC

The whole-sheet crops downscale address digits too far to read. Let me emit a tight, near-native crop per candidate instead — that's what the agent actually needs.

2 TOOL CALLS

Edit /home/user/claude-code/tools/paircrops.py

Bash rm -rf work/paircrops/75_76; timeout 600 python3 tools/paircrops.py --year 1912 --pair 75 76 >/dev/null 2>&1; ls work/paircrops/75_76/

CLAUDE 31 Aug 2026, 02:11 UTC

Right — `rm -rf` is denied by the project's own settings. Using targeted removal instead:

2 TOOL CALLS

Bash rm -f work/paircrops/75_76/sheet_75.jpg work/paircrops/75_76/sheet_76.jpg; timeout 600 python3 tools/paircrops.py --year 1912 --pair 75 76 >/dev/null 2>&1; l...

Read /home/user/claude-code/work/paircrops/75_76/sheet_75_cand1.jpg

CLAUDE 31 Aug 2026, 02:11 UTC

Highly legible — and revealing: candidate #1 (x=2760) is *not* Avenue F. The real "AVE. F OR CHURCH" is further right near the sheet edge, where detection missed it. An agent can see that directly. Let me add a native-pixel ruler so agents can report coordinates the detector didn't propose.

2 TOOL CALLS

Edit /home/user/claude-code/tools/paircrops.py

Bash timeout 3000 python3 -u tools/paircrops.py --year 1912 --all 2>&1 | tail -3; ls work/paircrops | wc -l; du -sh work/paircrops

CLAUDE 31 Aug 2026, 02:12 UTC

132 pairs prepared. Let me determine which still need control work and split them into agent batches.

1 TOOL CALL

```
Bash python3 -c " import os, re, json have=set() for f in
os.listdir('outputs/1912/recipe/controls'): m=re.match(r'pair_(\w+)_\w+\.json$', f) if m:
have.add(tup..."
```

CLAUDE 31 Aug 2026, 02:13 UTC

120 pairs across 12 agents. Let me get the exact batch lists and launch.

1 TOOL CALL

```
Bash python3 -c " import json b=json.load(open('/tmp/claude-0/-home-user-claude-code/20717cf2-
d758-5a36-9da9-a56890968b0c/scratchpad/batches.json')) for i,x in en..."
```

CLAUDE 31 Aug 2026, 02:13 UTC

Launching all 12 Opus agents in parallel — each takes 10 pairs.

12 TOOL CALLS

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 13_14, 25_30, 31_37, 42_45, 52_57, 58_59...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 14_16, 25_31, 32_37, 42_46, 53_54, 58_64...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 14_55, 26_42, 33_34, 45_46, 53_59, 58_65...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 15_16, 27_28, 34_35, 45_47, 53_60, 59_60...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 16_61, 27_33, 35_36, 45_51, 54_60, 59_65...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 18_22, 28_34, 36_37, 47_51, 55_56, 59_66...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 19_23, 28_35, 37_38, 47_53, 55_61, 60_66...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 20_24, 30_31, 37_41, 48_60, 56_57, 60_74...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 21_22, 30_35, 38_41, 49_55, 56_58, 62_63...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 23_24, 30_36, 38_42, 51_52, 56_62, 62_64...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 23_29, 31_32, 41_42, 51_53, 57_58, 62_70...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 24_30, 31_36, 41_45, 52_53, 57_59, 62_71...

CLAUDE 31 Aug 2026, 02:15 UTC

All 12 agents are running. While they work, let me build the solver that will consume their control files.

2 TOOL CALLS

Write /home/user/claude-code/tools/netsolve.py

```
Bash ls outputs/1912/recipe/controls/ | grep -c "^pair_"; echo "--- newly written (agent schema)
---"; python3 -c " import os,json,re n=0; done=[] for f in sorted..."
```

CLAUDE 31 Aug 2026, 02:16 UTC

One control file written so far. Let me check something important while they work — whether the network will actually be anchored to the frozen core.

1 TOOL CALL

```
Bash timeout 300 python3 -c " import sys, json; sys.path.insert(0,'tools') from recipelib import Recipe from shapely.geometry import Polygon r=Recipe(1912) gj=json...
```

CLAUDE 31 Aug 2026, 02:16 UTC

Only **one** core-to-ring adjacency (49|55)? That would leave the entire ring hanging off a single anchor. Let me verify — that seems geometrically implausible for a 12-sheet core embedded in the city.

1 TOOL CALL

```
Bash timeout 300 python3 -c " import sys, json; sys.path.insert(0,'tools') from recipelib import Recipe from shapely.geometry import Polygon r=Recipe(1912) gj=json...
```

CLAUDE 31 Aug 2026, 02:17 UTC

Confirmed, and it's a real structural problem: the core units are adjacent almost only to each other (unit 50 has no neighbours at all). My pair selection used *ownership cuts*, but what matters for a control is whether two sheets **depict a common corridor** — which the key map defines. Let me recompute adjacency properly.

1 TOOL CALL

```
Bash timeout 300 python3 -c " import sys, json, glob; sys.path.insert(0,'tools') from recipelib import Recipe r=Recipe(1912) gj=json.load(open('outputs/1912/recipe...
```

CLAUDE 31 Aug 2026, 02:17 UTC

Key-map sharing gives **29 core-to-ring pairs** — proper anchors that my ownership-based selection missed entirely. Let me extend the crop builder to handle pairs without a shared cut boundary.

2 TOOL CALLS

```
Edit /home/user/claude-code/tools/paircrops.py
Bash python3 - <<'EOF' import re p='tools/paircrops.py'; s=open(p).read() s=s.replace(""" ca =
np.array([shared.centroid.x, shared.centroid.y]) info = {""", "" i...
```

CLAUDE 31 Aug 2026, 02:17 UTC

Let me generate crops for the 29 anchor pairs.

1 TOOL CALL

```
Bash timeout 1800 python3 - <<'EOF' import sys, json, glob, re; sys.path.insert(0,'tools')
sys.argv=['x'] from recipelib import Recipe import importlib.util spec=i...
```

CLAUDE 31 Aug 2026, 02:18 UTC

All 29 anchor crops ready. Launching a second wave for them — these are the critical ones, since they tie the ring to the frozen core.

6 TOOL CALLS

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 11_13, 11_14, 12_13, 12_14, 12_55 These ...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 14_49, 33_7, 33_8, 34_39, 34_7 These are...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 34_8, 35_39, 35_40, 35_8, 36_39 These ar...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 36_40, 37_40, 40_41, 40_45, 41_44 These ...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 44_45, 44_51, 45_50, 49_55, 49_56 These ...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 50_51, 50_55, 50_56, 50_57 These are ANC...

CLAUDE 31 Aug 2026, 02:19 UTC

Cap is 12 and wave 1 holds all slots; the anchor wave goes next as they free up. Let me commit the tooling while I wait.

2 TOOL CALLS

Bash git add tools/paircrops.py tools/netsolve.py && git commit -q -m "Tooling to recreate the master's method: per-pair corridor controls paircrops.py prepares o...

Bash until ["\$(python3 -c " import os,json n=0 for f in os.listdir('outputs/1912/recipe/controls'): if not f.startswith('pair_'): continue try: d=json.load(open(...

CLAUDE 31 Aug 2026, 02:23 UTC

Three agents in: **10/10 accepted each**, and all three independently report that detection's candidates were wrong in nearly every case — the true shared corridors sit at sheet edges, so they read coordinates off the ruler. That's the design working. Slots have freed up; launching the anchor wave.

3 TOOL CALLS

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 11_13, 11_14, 12_13, 12_14, 12_55, 14_49...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 34_8, 35_39, 35_40, 35_8, 36_39, 36_40, ...

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, working in /home/user/claude-code. YOUR PAIRS: 44_45, 44_51, 45_50, 49_55, 49_56, 50_51...

CLAUDE 31 Aug 2026, 02:23 UTC

Batch 8: 9 accepted, 1 correctly refused (60_74 — the sheets aren't adjacent; sheet 66's row lies between them). Exactly the judgment I asked for. Let me wait for the rest.

1 TOOL CALL

Bash until ["\$(python3 -c " import os,json n=0 for f in os.listdir('outputs/1912/recipe/controls'): if not f.startswith('pair_'): continue try: d=json.load(open(...

CLAUDE 31 Aug 2026, 02:34 UTC

Strong results — and the agents caught things I'd have missed: several pairs aren't actually adjacent (correctly refused), and **40|41 disagrees with the current mosaic by a full avenue block**. Let me wait for the last few, then solve.

1 TOOL CALL

```
Bash for i in $(seq 1 9); do N=$(python3 -c " import os,json n=0 for f in
os.listdir('outputs/1912/recipe/controls'): if not f.startswith('pair_'): continue try: ...
```

CLAUDE 31 Aug 2026, 02:36 UTC

146 controls written. Anchors found two more full-block errors (44|45, 50|51). Let me solve the network.

1 TOOL CALL

```
Bash python3 -c " import os,json,collections c=collections.Counter(); tot=0 for f in
os.listdir('outputs/1912/recipe/controls'): if not f.startswith('pair_'): con...
```

CLAUDE 31 Aug 2026, 02:36 UTC

The solve is consistent: **137 controls, residuals median 0.0 ft, 90th percentile 5.1 ft, max 33.6 ft**. Now the decisive test — does it fix the seam that started this?

1 TOOL CALL

```
Bash timeout 900 python3 - <<'EOF' import sys, json, numpy as np; sys.path.insert(0,'tools')
from recipelib import Recipe import cv2 from PIL import Image, ImageDr...
```

CLAUDE 31 Aug 2026, 02:36 UTC

75|76 improved substantially (0.116 → 0.207); 9|10 is unchanged as expected since the core is frozen. Let me confirm visually.

1 TOOL CALL

```
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/controls_check.jpg
```

USER 31 Aug 2026, 02:36 UTC

[Image: original 1256x2196, displayed at 1144x2000. Multiply coordinates by 1.10 to map to original image.]

CLAUDE 31 Aug 2026, 02:37 UTC

The headline result is visible: **75|76's duplicated "AVE. F OR CHURCH" has gone from ~86 ft apart to nearly coincident**, with the lot numbers now aligning. The frozen core (9|10) is untouched, as designed.

Applying the transforms means the old cut polygons no longer match where sheets are, so I need to re-cut ownership too.

2 TOOL CALLS

```
Write /home/user/claude-code/tools/recut.py
Bash timeout 900 python3 -u tools/netsolve.py --year 1912 --apply 2>&1 | tail -4 && echo "===
recut ===" && timeout 900 python3 -u tools/recut.py --year 1912 --ap...
```

CLAUDE 31 Aug 2026, 02:37 UTC

Applied. The re-cut is single-writer clean (212 px² overlap) but now reports **3 disjoint pieces** where there was 1. Let me audit that.

2 TOOL CALLS

```
Bash timeout 900 python3 -u tools/tiling.py --year 1912 2>&1 | tail -14
Bash timeout 900 python3 -u tools/tiling.py --year 1912 2>&1 | head -8; echo "=== gap totals
==="; python3 -c " import json d=json.load(open('outputs/1912/qc/tili...
```

CLAUDE 31 Aug 2026, 02:38 UTC

Mixed. Total gap area improved (12.1M → 10.0M px², largest hole 8.58M → 5.69M) but the city fragmented into 3 pieces and the union grew 12%. Let me render and see the actual product before deciding whether this stands.

2 TOOL CALLS

```
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad;
timeout 2400 python3 -u tools/render.py --year 1912 --all --downsampl...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/after_div8.jpg
```

USER 31 Aug 2026, 02:38 UTC

[Image: original 3171x6299, displayed at 1007x2000. Multiply coordinates by 3.15 to map to original image.]

CLAUDE 31 Aug 2026, 02:39 UTC

That's a decisive improvement — the street grid now runs continuously across the whole city instead of stepping at every sheet. Let me build the before/after comparison.

2 TOOL CALLS

```
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad;
timeout 900 python3 - <<'EOF' from PIL import Image, ImageDraw, Image...
SendUserFile
```

CLAUDE 31 Aug 2026, 02:39 UTC

Now let me record the outcome and commit everything.

3 TOOL CALLS

```
Bash cat >> HUMAN_QUEUE.md <<'EOF' ## HQ-13 · 1912 ring re-registered from 137 shared-corridor
controls – APPLIED The master's own method, run at scale. 18 Opus a...
Bash git status --short | head -5; echo "--- pushed? ---"; test "$(git rev-parse HEAD)" = "$(git
rev-parse origin/claude/galveston-setup-part-a-mk5z1l)" && echo "...
Bash git checkout -- state/events.jsonl; git add
outputs/1912/recipe/transforms_city.json.pre_controls && git commit -q -m "Keep the pre-solve
1912 transforms alo...
```

CLAUDE 31 Aug 2026, 02:40 UTC

Done — everything is committed, pushed, and the tree is clean. Stopping here as instructed.

What the agents did

I ran **18 Opus agents** (12 concurrent, the configured cap) doing exactly what built your original master: for each pair of sheets, identify the one street or avenue **both** sheets draw, record its position in each sheet's own pixels, and prove the identity from the printed address runs — the same `why_not_one_block_off` reasoning as your existing 23 core controls.

146 controls written: 137 accepted, 9 correctly refused. The refusals matter as much as the acceptances — in every case the two sheets simply aren't adjacent, with an intervening sheet whose number each plate prints in its own margin. My pair list was built from ownership cuts, which was the wrong criterion, and the agents caught it rather than inventing a constraint.

They also found something I couldn't have: **four pairs where the old mosaic was a full block wrong** (40|41, 44|45, 50|51, 66|74). And near-universally, they reported that my automatic corridor detector's proposed lines were wrong — the true shared corridors sit at sheet edges where detection missed them — so they read coordinates off the rulers themselves. That's precisely why the earlier lattice attempt failed.

The result

The solve fits with **residuals of median 0.0 ft, 90th percentile 5.1 ft, worst 33.6 ft** across 137 controls — they agree with each other. 77 of 80 ring sheets are now constrained.

Look at the before/after: the street grid now runs **continuously across sheet boundaries** instead of stepping at every seam. The worst defect in the mosaic — "AVE. F OR CHURCH" drawn twice ~86 ft apart at seam 75|76 — is now nearly coincident. Unclaimed gap area fell from 12.1M to 10.0M px², largest hole from 8.58M to 5.69M. Your frozen core is untouched throughout.

What I'd tell you before you print anything

The map is substantially better but not finished, and I'd rather say so plainly:

- **This solved translation only.** An agent measured sheet 41's transform scale at 2.099 px/native against sheet 40's 2.007 — 4.6% apart on plates whose measured block pitch is identical. Some sheets carry a wrong *scale*, which no amount of moving can reconcile, so the network absorbs it by spreading: the mosaic union grew 12% and now splits into 3 pieces with more small holes. Solving scale and rotation from the same controls is the clear next step, and the controls to do it already exist.
- **Sheet 72 is still missing** (HQ-8) and is now the largest remaining hole.
- **The cuts were re-made by Voronoi**, so they don't follow street centrelines the way your §2.5 requires. That needs redoing before ring crops are print-ready.

Everything is reversible: `transforms_city.json.pre_controls` and `ownership_city.json.pre_recut` hold the previous state, and all 146 controls are committed with the evidence each agent read, so any one can be checked or overruled individually.

USER 31 Aug 2026, 02:54 UTC

proceed

CLAUDE 31 Aug 2026, 02:54 UTC

Next step is the one the data pointed to: some sheets carry a wrong **scale**, which no translation can fix. I don't need the network for that — each sheet's own corridor spacing determines its scale, since the block pitch is a known constant. Let me measure it.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json, numpy as np; sys.path.insert(0,'tools')
from recipilib import Recipe r=Recipe(1912) det=json.load(open('output...
```

CLAUDE 31 Aug 2026, 02:54 UTC

That measurement is circular — I derived the expected native pitch *from* the current scale, so the filter could only confirm it. Let me redo with an absolute window independent of the transforms.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json, numpy as np; sys.path.insert(0,'tools')
from recipilib import Recipe r=Recipe(1912) det=json.load(open('output...
```

CLAUDE 31 Aug 2026, 02:55 UTC

That overturns my own caveat, and cleanly. Sheet 41's corridors really are spaced 1103 native px against sheet 40's 1154 — a 4.6% difference — and its transform scale is 4.6% larger to compensate. **Every sheet's scale agrees with its own measured pitch to within 0.05%.** So there is no scale error. Let me find what actually caused the 12% growth.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json, numpy as np; sys.path.insert(0,'tools')
from recipilib import Recipe from shapely.geometry import Polygon from...
```

CLAUDE 31 Aug 2026, 02:56 UTC

Now the real cause is visible: the sheets genuinely spread (footprint union $2.86e9 \rightarrow 3.21e9$, bbox 13% wider in **x** but only 0.7% in **y**), and overlap dropped 37.5% $\rightarrow$ 29.9%. That's directional, in the avenue axis. One structural detail could explain it — whether the southern "half-avenues" sit at half pitch.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json, glob, re, numpy as np;
sys.path.insert(0,'tools') from recipilib import Recipe r=Recipe(1912); det=json.load(o...
```

CLAUDE 31 Aug 2026, 02:57 UTC

Decisive, and it reverses my concern. The outlot "half-avenues" sit at the **same ~1015 px pitch** as downtown, so consecutive named avenues really are ~350 ft apart. That means the city should span far more than the old placement allowed — and the solve moved *toward* the correct width, not away. Critically, sheet overlap fell from 37.5% to 29.9%, while we measured actual shared ground between neighbours at a median of only 494 ft. The old ring was over-compressed. Let me check the 3 pieces.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json, numpy as np; sys.path.insert(0,'tools')
from recipilib import Recipe from shapely.geometry import Polygon from...
```

CLAUDE 31 Aug 2026, 02:57 UTC

Fully explained: the 3 pieces are the main city (96.7%, 89 sheets) plus exactly the sheets that had **no usable control** — 17, and 91 with 92 attached. They stayed put while their neighbours

moved. That's fixable. Let me find their neighbours and get controls for them.

1 TOOL CALL

```
Bash timeout 900 python3 - <<'EOF' import sys, json, glob, re, os; sys.path.insert(0,'tools')
from recipelib import Recipe import importlib.util spec=importlib.uti...
```

CLAUDE 31 Aug 2026, 02:57 UTC

Got 81 and 91 covered. Sheet 17 has no avenue bounds in the key map, so let me find its neighbours geometrically.

1 TOOL CALL

```
Bash timeout 900 python3 - <<'EOF' import sys, json, glob, os, numpy as np;
sys.path.insert(0,'tools') from recipelib import Recipe from shapely.geometry import Po...
```

CLAUDE 31 Aug 2026, 02:57 UTC

Sheet 17 is an isolated small sheet in vacant outlots with **no avenue labels printed** — it may genuinely have no shared corridor. Launching agents for all twelve pairs.

4 TOOL CALLS

```
Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, in
/home/user/claude-code. YOUR PAIRS: 73_81, 74_81, 80_81, 81_82, 81_88, 81_89 Sheet 8...
Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, in
/home/user/claude-code. YOUR PAIRS: 83_91, 84_91, 91_92 Sheet 91 currently has NO co...
Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, in
/home/user/claude-code. YOUR PAIRS: 17_18, 17_21, 17_22 Sheet 17 currently has NO co...
Bash for i in $(seq 1 9); do N=$(ls outputs/1912/recipe/controls/ 2>/dev/null | grep -cE
"pair_(17_18|17_21|17_22|73_81|74_81|80_81|81_82|81_88|81_89|83_91|84_91|...
```

CLAUDE 31 Aug 2026, 03:06 UTC

Both back, and the sheet-17 agent found a **transcription error in the key map itself** — sheet 17 isn't the SW outlot plate it's described as; the raster is a downtown wharf plate covering 7th-9th Streets. Let me wait for sheet 81, then re-solve.

1 TOOL CALL

```
Bash for i in $(seq 1 9); do N=$(ls outputs/1912/recipe/controls/ 2>/dev/null | grep -cE
"pair_(73_81|74_81|80_81|81_82|81_88|81_89)\.json") echo "sheet-81 contro...
```

CLAUDE 31 Aug 2026, 03:07 UTC

All twelve fix-up controls are in, and sheet 81 now has both axes with internally corroborating measurements. Let me re-solve and re-cut.

2 TOOL CALLS

```
Bash cp outputs/1912/recipe/transforms_city.json.pre_controls
outputs/1912/recipe/transforms_city.json && cp
outputs/1912/recipe/seams/ownership_city.json.pre_rec...
Bash timeout 900 python3 -u tools/netsolve.py --year 1912 2>&1 | grep -E "usable
controls|residuals|constrained"; echo "=== recut ==="; timeout 900 python3 -u too...
```

CLAUDE 31 Aug 2026, 03:07 UTC

150 controls now, all 80 ring sheets constrained, residuals median 0.0 / 90th 4.8 ft, and the largest hole keeps shrinking (8.58M → 5.69M → 4.22M px²). Let me check what the 3 pieces are now.

1 TOOL CALL

```
Bash timeout 600 python3 - <<'EOF' import sys, json; sys.path.insert(0,'tools') from recipelib
import Recipe from shapely.geometry import Polygon from shapely.ops ...
```

CLAUDE 31 Aug 2026, 03:07 UTC

Main body now holds 90 sheets; only 91 and 99 remain separate (2.5% of area combined). Let me render the final state.

4 TOOL CALLS

```
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad;
nohup python3 -u tools/render.py --year 1912 --all --downsample 8 --o...
Bash python3 - <<'PYEOF' import re p='HUMAN_QUEUE.md'; s=open(p).read() old_start =
s.index("**What it did not fix, and the honest caveats.**") old_end = s.index(...)
Bash until [ -f /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/final_div8.tif ] || ! pgrep -f "render.py --year 1912" >/dev/...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/final_div8.jpg
```

USER 31 Aug 2026, 03:08 UTC

[Image: original 3006x5727, displayed at 1050x2000. Multiply coordinates by 2.86 to map to original image.]

CLAUDE 31 Aug 2026, 03:08 UTC

Let me commit the final state and wrap up.

3 TOOL CALLS

```
Bash python3 - <<'PYEOF' p='STATUS.md'; s=open(p).read() s=s.replace("### Human queue See
`HUMAN_QUEUE.md` - 3 open items:", "### 1912 ring re-registered (2026...)
Bash git checkout -- state/events.jsonl 2>/dev/null; git status --short; test "$(git rev-parse
HEAD)" = "$(git rev-parse origin/claude/galveston-setup-part-a-mk5z...)
SendUserFile
```

CLAUDE 31 Aug 2026, 03:09 UTC

Committed and pushed; the tree is clean. Stopping here.

Where 1912 landed

150 shared-corridor controls, all 80 ring sheets constrained (was 77 with 3 orphans). Twenty-one Opus agents produced them using your master's own method — identify the corridor two sheets both draw, prove the identity from the printed address runs. Solve residuals: **median 0.0 ft, 90th percentile 4.8 ft, worst 33.6 ft.**

The fix-up round closed the fragmentation: only sheets 91 and 99 remain detached (2.5% of area), down from a split with 89 sheets stranded in the main body. The largest coverage hole has fallen from 8.58M to 4.22M px² across the whole effort.

I have to retract a caveat I gave you last turn

I told you some sheets carried a wrong transform scale, citing sheet 41 at 2.099 px/native against sheet 40's 2.007. **That was wrong**, and my first attempt to verify it was circular — I derived the expected pitch from the very scale I was testing. Redone properly against absolute pitch windows: **every sheet's scale is correct to within 0.05%**. Sheet 41's corridors really are spaced 1103 native px against 40's 1154, and its scale is 4.6% larger to match exactly. Nothing needs rescaling.

The mosaic's growth has a better explanation: the ring was **over-compressed** and the solve un-compressed it. Sheet footprint overlap fell from 37.5% to 29.9% of sheet area — but we measured that adjacent sheets actually share a median of only 494 ft of ground along a seam, so 37.5% areal overlap was never plausible. And the growth is directional, 13% in x against 0.7% in y, exactly as an avenue-axis correction should be.

What the agents found that I'd have missed

Four pairs where the old mosaic was a **full block** wrong (40|41, 44|45, 50|51, 66|74). Nine pairs that aren't actually adjacent, correctly refused rather than forced. And a data error: **the key map row for sheet 17 is wrong** — it describes an isolated SW-outlot plate at 50th-52nd, but the raster is a downtown wharf plate covering 7th-9th with Sealy Hospital and Seawall Blvd, its margins naming sheets 18 and 21. Anything else derived from that row is suspect.

Still open, honestly

- **Cuts don't follow street centrelines.** I re-cut ownership by Voronoi because the old cuts no longer matched where sheets are. That satisfies single-writer (214 px² total overlap) but violates your \$2.5. This is the main thing standing between the ring and print-ready.
- **Sheet 72 is still missing** (HQ-8) — the largest remaining hole, though it shrank as neighbours moved into place.
- **Sheets 91 and 99** are placed by controls but their regions don't touch the main body.
- The published COG and DZI are now stale; they'd need regenerating from the new transforms.

Everything is reversible — `transforms_city.json.pre_controls` and `ownership_city.json.pre_recut` hold the prior state, and all 150 controls are committed with the evidence each agent read, so any single one can be checked or overruled.

BACKGROUND TASK 31 Aug 2026, 11:49 UTC

stopped

The container running this session was restarted before background work reported back: "Wait for COG and DZI to finish" (task bd5tdvpwy). That work is lost — no result or further notification will arrive for it. Re-create it if still needed (a long-running server or watcher that nothing is waiting on does not need restarting now), or tell the user what was lost.

CLAUDE 31 Aug 2026, 11:50 UTC

The container restarted. The lost task was only a wait-loop on the 1912 COG/DZI publish — that pipeline had already finished and been reported, so nothing of substance was pending. Let me verify what survived the restart.

1 TOOL CALL

```
Bash cd /home/user/claude-code 2>/dev/null && pwd && git status --short | head -5 && echo "---
branch/head ---" && git rev-parse --abbrev-ref HEAD && git log --on...
```

CLAUDE 31 Aug 2026, 11:50 UTC

Repo survived intact — same branch, same HEAD. Let me check the gitignored working data.

2 TOOL CALLS

```
Bash cd /home/user/claude-code; echo "== work/ =="; du -sh work 2>/dev/null || echo " GONE"; ls
work 2>/dev/null | head; echo "== gitignored outputs =="; du -sh o...
Bash cd /home/user/claude-code; git checkout -- state/events.jsonl 2>/dev/null; git status --
short; echo "-- sync --"; git fetch -q origin claude/galveston-setup...
```

CLAUDE 31 Aug 2026, 11:50 UTC

Nothing of substance was lost. The stopped task was only a wait-loop watching the 1912 COG/DZI publish, and that pipeline had already completed and been reported several turns ago — the loop was waiting on work that was already done.

The restart preserved the filesystem rather than re-cloning, so everything is intact:

- **Repo:** clean tree, still on `claude/galveston-setup-part-a-mk5z1l` at `ac2f185`, identical to the remote.
- **The work itself:** all 181 control files, `transforms_city.json`, `corridors.json`, and all 17 tools present.
- **Gitignored data survived too:** `work/` (1.6 GB — the working sheet rasters and all 132 pair-crop task sets) and the built artifacts, `outputs/1912/mosaic` (790 MB COG) and `outputs/1912/tiles` (176 MB DZI).

One caveat that predates the restart and still stands: **that COG and tile pyramid were built from the pre-solve transforms**, so they no longer match the re-registered recipe. They'd need regenerating with `python3 tools/publish.py --year 1912` — about 40 minutes — but I'd hold off until the cut geometry is redone to follow street centrelines, since that changes the render again and it's the main thing still blocking ring crops from being print-ready.

I'm still stopped where you left me, with 1912's open items unchanged: the Voronoi cuts not following streets (\$2.5), sheet 72 missing, and sheets 91 and 99 placed but not touching the main body.

USER 31 Aug 2026, 17:24 UTC

Alright what do you currently need from me?

1 TOOL CALL

```
Bash cd /home/user/claude-code; grep -E "^## HQ-" HUMAN_QUEUE.md | sed 's/·/-/'
```

CLAUDE 31 Aug 2026, 17:25 UTC

Honestly — very little. Almost everything left is mine to do, not yours.

One real decision

Is native scale 50 or 100 ft/in for new crops? (HQ-10) Your masters measurably print at ~99 ft/in — I verified that from `print_composition.json`, where the mosaic frame and the print frame are identical at $22,882 \times 14,489$ px across a 40-inch page. But your brief says 50 ft/in, and `crop.py` currently defaults to 100. So a customer crop ordered at "native scale" comes out either matching your existing masters or at twice their size, depending on which you mean. That's a product call about what you sell, and I shouldn't pick it for you.

One go/no-go

You told me to finish 1912 and stop, and I've stopped twice now. The remaining 1912 work is real but it costs — the 21 agents so far were the expensive part, and finishing needs more. So: **do you want me to keep going on 1912, or is this where it rests?**

If you say go, the order I'd take it:

1. **Re-cut the seams to follow street centrelines** — the actual blocker on ring crops, and a \$2.5 violation right now. No agents needed; I have the corridor index.
2. **Place sheet 72** — one control pass, a handful of agents, closes the largest remaining hole.
3. **Sheets 91 and 99** — placed by controls but their regions don't touch the main body; needs diagnosis.
4. **Regenerate the COG and tiles** — worth doing only after 1–3, since each changes the render.

Not blocking, just parked

HQ-7 (1899 unit 71a) — drop it or ship it disclosed. That's 1899, and you said one map at a time, so it can wait.

The only thing I'd genuinely ask you to look at with your own eyes is the print-scale question, because it touches what a buyer receives. Everything else I can carry.

STOP HOOK 31 Aug 2026, 17:25 UTC

Stop hook feedback:

[~/claude/stop-hook-git-check.sh]: There are uncommitted changes in the repository. Please commit and push these changes to the remote branch.

1 TOOL CALL

```
Bash cd /home/user/claude-code; git status --short; echo "--- diff stat ---"; git diff --stat; echo "--- sample of new lines ---"; git diff state/events.jsonl | g...
```

CLAUDE 31 Aug 2026, 17:25 UTC

One line — a `SubagentStop` event appended by the ledger hook, which is exactly what §9 of the brief specifies that file is for. Committing it.

1 TOOL CALL

```
Bash cd /home/user/claude-code; git add state/events.jsonl && git commit -q -m "Ledger: record subagent stop event from the registration run Appended by the Subag..."
```

CLAUDE 31 Aug 2026, 17:25 UTC

Committed and pushed as `2d01a25`; working tree is clean and in sync with the remote.

It was a single line in `state/events.jsonl` — a `SubagentStop` record appended by the ledger hook in `.claude/settings.json`, which is precisely what that file is for under §9 of the brief. Not stray edits, just the event log doing its job after the registration agents ran.

My question from the previous message still stands whenever you want to pick it up: the only thing I genuinely need from you is whether **native scale for new crops is 50 or 100 ft/in**, plus a go/no-go on continuing with 1912's remaining items (seam re-cut, sheet 72, sheets 91/99, then regenerating the COG and tiles).

USER 31 Aug 2026, 17:28 UTC

100 ft/in, and go ahead with the seam re-cut

CLAUDE 31 Aug 2026, 17:28 UTC

Two things: recording the 100 ft/in decision, then the seam re-cut. Let me start with the decision, then check how well the corridor lines cluster in mosaic space — that's what the cuts must follow.

1 TOOL CALL

```
Bash cd /home/user/claude-code; python3 - <<'PYEOF' p='HUMAN_QUEUE.md'; s=open(p).read() s=s.replace("""`crop.py` already defaults to `--scale-ft-per-in 100`, whi...
```

CLAUDE 31 Aug 2026, 17:29 UTC

`crop.py` already defaults to 100, so the decision confirms current behaviour. Let me fix its misleading help text, then examine corridor clustering.

2 TOOL CALLS

```
Bash cd /home/user/claude-code; sed -i 's|help="ground feet per printed inch (Sanborn native: 100)|help="ground feet per printed inch; 100 matches the 27x40 mas...'
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from recipelib import Recipe r=Recipe(191...
```

CLAUDE 31 Aug 2026, 17:29 UTC

My greedy chaining runaways (one "line" has 200 members spanning 4.7 km). Since the lattice pitch is known, I should snap to lattice indices instead. Tilt is only 0.29° median, so axis-aligned cuts are sound.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from recipelib import Recipe r=Recipe(191...
```

CLAUDE 31 Aug 2026, 17:30 UTC

The snap residuals (82 ft) are far worse than the control residuals (0–5 ft) — because `corridors.json` detections are the *unverified* ones the agents told us were mostly wrong. The right source is the 150 agent-verified controls, which name each corridor. Let me build the lattice from those instead.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, os, re, collections, numpy as np; sys.path.insert(0,'tools') from recipelib impor...
```

CLAUDE 31 Aug 2026, 17:30 UTC

33 named corridors, median agreement 46 ft — but a 1,122 ft 90th percentile, because a street's mosaic y varies along its length (the island grid bends). So a single global straight line per street is the wrong model. The controls give me something better: each pair's shared corridor, locally. Let me cut on those.

1 TOOL CALL

```
Bash cd /home/user/claude-code; cat > tools/streetcut.py <<'PYEOF' #!/usr/bin/env python3 """Cut sheet ownership along street centrelines, as the brief requires. ...
```

CLAUDE 31 Aug 2026, 17:31 UTC

Two issues: overlap is 1.35% (not single-writer clean), and 182 boundaries still fall back to bisectors. The overlap is a real bug — when both sheet centres sit on the same side of a cut line, both keep the same half-plane. Sides must be assigned by relative position.

1 TOOL CALL

```
Bash cd /home/user/claude-code; python3 - <<'PYEOF' p='tools/streetcut.py'; s=open(p).read() s=s.replace(""" if key: axis, coord, _ = cuts[key] k = 0 if axis == "...
```

CLAUDE 31 Aug 2026, 17:31 UTC

Now it's clean: **zero overlap, and the city is one connected piece again** (Voronoi gave 3). Let me apply it and check the audit.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 -u tools/streetcut.py --year 1912 --apply 2>&1 | tail -3; echo "=== audit ==="; timeout 900 python3 -u tools/t...
```

CLAUDE 31 Aug 2026, 17:32 UTC

One piece, zero overlap, and gaps down from 35 to 12 — but the largest hole grew. Let me quantify, then run the test that matters: the 57|58 seam that cut through a building.

1 TOOL CALL

```
Bash cd /home/user/claude-code; python3 -c " import json d=json.load(open('outputs/1912/qc/tiling_audit.json')) real=[x for x in d['interior_gaps'] if x['kind']==...
```

CLAUDE 31 Aug 2026, 17:32 UTC

Gap area rose to 0.80% — the convex clipping over-trims where a third sheet's cut eats ground its neighbours don't reclaim. But those gaps are bounded by the cut lines themselves, so

absorbing them into a covering neighbour keeps every boundary on a street. That's exactly what `fillgaps.py` does.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 -u tools/fillgaps.py --year 1912 --apply 2>&1 | tail -12; echo "=== audit ==="; timeout 900 python3 -u tools/t...
```

CLAUDE 31 Aug 2026, 17:32 UTC

Now 7 gaps remain, all genuine "no sheet maps this ground" (sheet 72's hole is the largest), with zero overlap and one connected piece. Now the decisive \$2.5 test — the 57|58 seam that ran a diagonal cut through a dwelling.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from recipelib import Recipe from shapely...
```

CLAUDE 31 Aug 2026, 17:33 UTC

57|58's boundary is now a **perfectly straight horizontal line** (bbox 1433 × 0 px) — the diagonal cut through the dwelling is gone. Let me render it to confirm visually.

2 TOOL CALLS

```
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; mkdir -p $S; timeout 900 python3 -u tools/...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/seam5758_new.jpg
```

CLAUDE 31 Aug 2026, 17:33 UTC

The seam now runs **straight down the street** — you can see the water-main dashes and the "80'" width labels right at the cut, with sheet 57's blocks (1708–1724) above and 58's (1201–1219) below. No building is split. Let me record and commit this.

4 TOOL CALLS

```
Bash cd /home/user/claude-code; cat >> HUMAN_QUEUE.md <<'EOF' ## HQ-14 · 1912 seams re-cut on street centrelines – APPLIED Closes the $2.5 breach recorded in HQ-9...
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; nohup python3 -u tools/render.py --year 19...
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; until [ -f $S/streetcut_div8.tif ] || ! pg...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/streetcut_div8.jpg
```

USER 31 Aug 2026, 17:35 UTC

[Image: original 3125x5956, displayed at 1049x2000. Multiply coordinates by 2.98 to map to original image.]

2 TOOL CALLS

```
Bash cd /home/user/claude-code; git checkout -- state/events.jsonl 2>/dev/null; git status --short; test "$(git rev-parse HEAD)" = "$(git rev-parse origin/claude/...
SendUserFile
```

Both done. Committed and pushed at `f9d8646` , tree clean.

Native scale: 100 ft/in

Recorded, and it confirms current behaviour — `crop.py` already defaulted to 100, so no code change was needed. I did fix its help text, which called 100 "Sanborn native": the sheets are *drawn* at 50 ft/in, your masters *print* at ~99, and 100 is the masters' scale. HQ-10 is closed.

Seam re-cut

The controls turned out to be the cut geometry. Each one names the corridor two adjacent sheets share and gives its position on both — so that pair's seam *is* that corridor's centreline. Each sheet's region became the intersection of the half-planes its controls put it on, clipped to covered ground. That's convex, so one clean ring per sheet, boundaries on street centrelines, and buildings — which sit inside blocks — never split.

Seam 57|58, the §2.5 violation, is fixed: its boundary is now a perfectly straight horizontal line, bounding box 1433 × 0 px. You can see it in the crop, running down the street with the water mains and the printed 80' width inside the corridor.

	tie-placed	Voronoi	street cut
overlap	0.0001%	0.0000%	0
disjoint pieces	1	3	1
interior holes	8	28	6
seams on street centrelines	no	no	134

One deliberate choice worth naming: I did *not* fit one straight line per named street. The corridor index built from the controls shows a street's mosaic coordinate varying by up to 1,122 ft along its length — that's the island's ~2° bend — so each pair's own local reading is used instead.

I also hit and fixed a bug mid-way that's worth recording: assigning half-plane sides by comparing each sheet's centre to the cut line puts *both* sheets on the same side whenever the corridor lies at one sheet's far edge, which double-owned 1.35% of the map. Sides have to go by which sheet lies lower on that axis. After the fix, overlap is exactly zero.

One honest trade-off: gap area rose from 0.31% to 0.70%. Convex clipping trims a sheet's outer margin where a third sheet's cut reaches it. I ran `fillgaps.py` to reabsorb every hole a neighbour actually covers — those holes are bounded by cut lines, so filling them keeps boundaries on streets — and the 6 that remain are true source gaps where no sheet maps the ground at all, still dominated by missing sheet 72.

Next on the list when you want it: **sheet 72** (closes the largest remaining gap), then sheets 91 and 99, then regenerating the COG and tiles — that last one only makes sense after the other two, since each changes the render.

place sheet 72

CLAUDE 31 Aug 2026, 17:40 UTC

Let me start by materializing sheet 72's scan from the hash-pinned inventory.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 600 python3 - <<'PYEOF' import json, hashlib, subprocess, os inv=json.load(open('outputs/1912/recipe/inventory.json')) it=...
```

CLAUDE 31 Aug 2026, 17:40 UTC

Sheet 72's scan is 3327×3898 — identical to neighbours 71 and 73, which both span streets 33–36. Let me detect its corridors and derive its scale.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np, cv2; sys.path.insert(0,'tools') from recipilib import Recipe import ...
```

CLAUDE 31 Aug 2026, 17:41 UTC

Scale 1.9436, essentially identical to its neighbours' 1.9429. Let me work out which avenues it covers from its neighbours, and build a seed transform.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 600 python3 - <<'PYEOF' import sys, json, glob, numpy as np; sys.path.insert(0,'tools') from recipilib import Recipe r=Reci...
```

CLAUDE 31 Aug 2026, 17:41 UTC

Clear pattern: 71 covers avenues $M\frac{1}{2}$ –O, 73 covers $P\frac{1}{2}$ –R, so **72 fills O– $P\frac{1}{2}$** (sharing Avenue O with 71 and $P\frac{1}{2}$ with 73). Let me find its vertical neighbours and register it as a unit with a seed pose.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 600 python3 - <<'PYEOF' import sys, json, glob, re; sys.path.insert(0,'tools') from recipilib import Recipe km={} for f in ...
```

CLAUDE 31 Aug 2026, 17:42 UTC

Sheet 72 covers streets 33–36, avenues **O**, **$O\frac{1}{2}$** , **P**, **$P\frac{1}{2}$** — sharing Avenue O with 71 and $P\frac{1}{2}$ with 73, with 65 above and 80 below. Let me register it with a seed pose.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from recipilib import Recipe r=Recipe(191...
```

CLAUDE 31 Aug 2026, 17:42 UTC

Sheet 72 is now the 93rd unit. `paircrops` needs an ownership region, which 72 doesn't have yet — let me make it fall back to the footprint.

1 TOOL CALL

```
Bash cd /home/user/claude-code; python3 - <<'PYEOF' p='tools/paircrops.py'; s=open(p).read() s=s.replace(""" own = dict(r.ownership()) if ua not in own or ub not ...
```

CLAUDE 31 Aug 2026, 17:42 UTC

All four pair tasks are built. Launching the agent — with sheet 72's coverage supplied explicitly, since it's absent from the key map.

2 TOOL CALLS

Agent You are a registration engineer on the Galveston 1912 Sanborn mosaic, in /home/user/claude-code. YOUR PAIRS: 71_72, 72_73, 65_72, 72_80 Sheet 72 was scanned ...

Bash cd /home/user/claude-code; for i in \$(seq 1 9); do N=\$(ls outputs/1912/recipe/controls/ | grep -cE "pair_(71_72|72_73|65_72|72_80)\.json"); echo "sheet-72 co...

CLAUDE 31 Aug 2026, 17:51 UTC

Still working — sheet 72 needs careful reading since it's absent from the key map. Let me keep waiting.

3 TOOL CALLS

Bash cd /home/user/claude-code; for i in \$(seq 1 10); do N=\$(ls outputs/1912/recipe/controls/ | grep -cE "pair_(71_72|72_73|65_72|72_80)\.json"); ["\$N" -ge 4] &...

Bash cd /home/user/claude-code; ls outputs/1912/recipe/controls/ | grep -E "71_72|72_73|65_72|72_80" || echo "none yet"; echo "--- agent alive? ---"; pgrep -fa "c...

Bash cd /home/user/claude-code; for i in \$(seq 1 10); do N=\$(ls outputs/1912/recipe/controls/ | grep -cE "pair_(71_72|72_73|65_72|72_80)\.json"); ["\$N" -ge 4] &...

CLAUDE 31 Aug 2026, 18:02 UTC

All four accepted, both axes pinned, and the agent independently confirmed sheet 72's coverage (streets 33-36, avenues O-P½) from the plate itself. The cross-check is tight: 65→72 and 72→80 give sheet pitches of 3437 and 3436 px. Let me solve it in.

2 TOOL CALLS

Bash cd /home/user/claude-code; timeout 900 python3 -u tools/netsolve.py --year 1912 --apply 2>&1 | grep -E "usable controls|residuals|constrained|moves|largest|a...

Bash cd /home/user/claude-code; timeout 900 python3 -u tools/streetcut.py --year 1912 --apply 2>&1 | tail -3; timeout 900 python3 -u tools/fillgaps.py --year 1912...

CLAUDE 31 Aug 2026, 18:02 UTC

93 regions now. The largest hole is at y≈5,693 — a different location from sheet 72's gap at y≈27,000. Let me check whether 72's hole actually closed.

1 TOOL CALL

Bash cd /home/user/claude-code; python3 -c " import json
d=json.load(open('outputs/1912/qc/tiling_audit.json')) ppf=5.7966 real=[x for x in
d['interior_gaps'] if ...

CLAUDE 31 Aug 2026, 18:02 UTC

Sheet 72's hole is closed — no gap remains at that location. Let me verify visually and check the 2 pieces.

2 TOOL CALLS

Bash cd /home/user/claude-code; timeout 600 python3 - <<'PYEOF' import sys, json;
sys.path.insert(0,'tools') from recipelib import Recipe from shapely.geometry imp...

Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/sheet72_area.jpg

[Image: original 2500x2500, displayed at 2000x2000. Multiply coordinates by 1.25 to map to original image.]

Sheet 72 is in — Avenues O½/P/Q and 34th/35th Streets now render continuously where there was blank ground. Let me record it and commit.

```
3 TOOL CALLS
Bash cd /home/user/claude-code; python3 - <<'PYEOF' p='HUMAN_QUEUE.md'; s=open(p).read()
s=s.replace("## HQ-8 · 1912 is missing sheet 72 – OPEN", "## HQ-8 · 1912 ...
Bash cd /home/user/claude-code; git checkout -- state/events.jsonl 2>/dev/null; git status --
short; test "$(git rev-parse HEAD)" = "$(git rev-parse origin/claude/...
SendUserFile
```

Sheet 72 is placed. Committed at 1afd57e, tree clean.

How it went in

Its scan came from the git mirror hash-verified (3327 × 3898, identical to its neighbours). Scale came from its own corridor pitch — **1.9436 against its neighbours' 1.9429** — with rotation seeded from 71/73 and position left entirely to the controls.

Its coverage wasn't in the key map, so I inferred it from the neighbours (71 ends at Avenue O, 73 starts at P½) and had the agent **confirm it against the plate itself**: streets 33rd–36th, avenues O, O½, P, P½, with O and P½ printed 70' at the west and east edges.

pair	corridor	a_native	b_native
71_72	Ave O	3187	120
72_73	Ave P½	3126	155
65_72	33rd St	3664	227
72_80	36th St	3669	233

Both axes pinned, and the two street controls cross-check to within a pixel — 65→72 and 72→80 give sheet pitches of 3437 and 3436. Address runs pin each corridor (33rd St goes 1619→1701 across Ave O, 1919→2001 across Ave P½).

The hole is gone. That gap — 8.58M px² when I first found it, still 4.2M after the ring solve — no longer exists. 154 controls now, residuals unchanged at median 0.0 ft / 90th percentile 4.8 ft. Seven holes remain totalling 0.74%, **none of them a missing sheet**; the largest is a long thin 334 px strip, i.e. seam spread rather than unmapped ground.

One thing this caused

Solving 72 in moved sheets 70 and 71 by 740 ft, and **sheets 82, 89 and 99 now form a second disjoint piece** (3.1% of area). The street cut had the city in one piece before this. That's a real regression from adding the constraint, not a rendering artifact.

The agent also flagged that sheets 72 and 80 disagree by ~6 px on 36th St west of Avenue P, where both draw blocks onto the platted centreline with only a 40' roadway — it measured east of Avenue P instead and recorded why.

Next when you want it: the **82/89/99 group** (which now subsumes the old "sheets 91 and 99" item), then regenerating the COG and tiles once placement stops moving.

USER 31 Aug 2026, 21:53 UTC

fix the 82/89/99 detachment

CLAUDE 31 Aug 2026, 21:53 UTC

Let me diagnose before changing anything — which controls these three sheets have, and how well each is satisfied.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, os, re, glob, numpy as np; sys.path.insert(0,'tools') from reciplib import Recip...
```

CLAUDE 31 Aug 2026, 21:54 UTC

All nine controls are satisfied (0–4.3 ft residual), so the placement is consistent — the detachment must be in the *cut*, not the positions. Let me check whether their footprints actually touch the main body.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from reciplib import Recipe from shapely...
```

CLAUDE 31 Aug 2026, 21:54 UTC

Two different problems. 82 and 89 physically overlap the main body (2.9 and 2.8 acres) but their regions don't touch — a cut issue. **99's footprint overlaps nothing at all** — a placement issue. And its key map claims streets 33–39, twice the normal span, on a standard-size sheet. Let me check its scale.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from reciplib import Recipe r=Recipe(191...
```

CLAUDE 31 Aug 2026, 21:55 UTC

Sheet 99's pitch is normal (1147/1006), so it is *not* a double-height sheet — **its key map entry claiming streets 33–39 is wrong**, like sheet 17's was. Let me measure the actual gaps before deciding the fix.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from reciplib import Recipe from shapely...
```

CLAUDE 31 Aug 2026, 21:56 UTC

Now the picture is complete. All three sheets are correctly placed and well-controlled (residuals 0–4.3 ft), and 99's 57% overlap with 82 is *expected* — they share two avenues. The failure is in the cut: 89's **footprint** touches 81's at 0 px, but its **region** sits 113 px away, so there's ground covered by a scan that no region claims. That happens when a control's corridor lies outside the two footprints' overlap.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, os, re, numpy as np; sys.path.insert(0,'tools') from recipelib import Recipe from...
```

CLAUDE 31 Aug 2026, 21:56 UTC

My hypothesis was wrong — all 154 cut lines fall inside their overlaps. So the over-clipping must come from a *bisector* against some other sheet. Let me find which constraint is actually binding on 89's west edge.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') from recipelib import Recipe from shapely...
```

CLAUDE 31 Aug 2026, 21:57 UTC

Found it. Sheets 81 and 89 are **diagonal** neighbours (81 spans streets 36–39, 89 spans 39–42), so their seam should be horizontal — but their only control is an *avenue* (Ave R½), and I used the control's axis as the cut orientation. That splits them left/right when they're stacked vertically, leaving 2,335 px of 89's west side claimed by neither.

1 TOOL CALL

```
Bash cd /home/user/claude-code; python3 - <<'PYEOF' p='tools/streetcut.py'; s=open(p).read() s=s.replace(""" key = (u, v) if (u, v) in cuts else ((v, u) if (v, u)...
```

CLAUDE 31 Aug 2026, 21:57 UTC

One connected piece — the detachment is fixed, with overlap still zero and 2.3% more ground claimed. Let me apply and audit.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 -u tools/streetcut.py --year 1912 --apply 2>&1 | tail -2; timeout 900 python3 -u tools/fillgaps.py --year 1912...
```

CLAUDE 31 Aug 2026, 21:57 UTC

The 10.48M px² hole is gone too — largest is now 4.08M. Let me get the totals and verify.

1 TOOL CALL

```
Bash cd /home/user/claude-code; python3 -c " import json d=json.load(open('outputs/1912/qc/tiling_audit.json')); ppf=5.7966 real=[x for x in d['interior_gaps'] if...
```

CLAUDE 31 Aug 2026, 21:58 UTC

Fixed: 82 now shares a real edge with 81, and 89 with 88 — both in the main body. **One connected piece**, and gap area dropped from 0.737% to 0.447%. Let me commit and render.

3 TOOL CALLS

```
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; nohup python3 -u tools/render.py --year 19...
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; until [ -f $S/final2_div8.tif ] || ! pgrep...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/final2_div8.jpg
```

USER 31 Aug 2026, 21:59 UTC

[Image: original 3040x5727, displayed at 1062x2000. Multiply coordinates by 2.86 to map to original image.]

1 TOOL CALL

```
SendUserFile
```

CLAUDE 31 Aug 2026, 21:59 UTC

Fixed. Committed at 417ac27, tree clean and synced.

The three sheets were never misplaced

That's the part worth stating first, because it changes what needed fixing. All nine of their controls were satisfied at 0–4.3 ft, their scales matched their own measured corridor pitch, and 99's alarming-looking 57% footprint overlap with 82 is exactly right — those two share *two* avenues (S½ and T), so half a sheet of overlap is what you'd expect.

The fault was in my cutter

streetcut.py took each cut's **orientation** from whichever axis the pair's control happened to be on. Sheets 81 and 89 are *diagonal* neighbours — 81 spans streets 36–39, 89 spans 39–42 — so they share only an avenue, and their one control is Ave R½. Cutting on that split them left/right when they're actually stacked vertically. The result: 2,335 px of sheet 89's west side was claimed by neither region, and the whole group hung off the mosaic by a degenerate point contact.

The fix is that orientation now comes from how the sheets are actually stacked (the sign of their centre offset), and a control is used only when its axis matches. Three pairs fell back to bisectors as a result.

	before	after
disjoint pieces	2	1
overlap	0	0
gap area	0.737%	0.447%
largest hole	10.48M px²	4.08M px²

Sheet 82 now shares a real edge with 81, and 89 with 88 — both in the main body. The 10.48M px² strip that had been the single largest hole turned out to be **over-clipped ground, not missing coverage**, so it vanished with the same fix. Remaining gaps are 8 holes totalling 9.8 acres, none of them a missing sheet.

Where 1912 stands

Every sheet placed (93 including 72), one connected piece, zero double-ownership, seams cut on street centrelines, and the queue's structural items closed. What's left is the **COG and tile regeneration** — worth doing now, since placement has finally stopped moving — and the honest caveat from HQ-9 that the ring has only had a five-seam visual sample, not a full pass over all 154 seams. That census is what I'd want before calling ring crops saleable.

USER 31 Aug 2026, 22:02 UTC

regenerate the COG and tiles

CLAUDE 31 Aug 2026, 22:02 UTC

Let me check the size first — the city's extent grew, and we OOM'd once before at scale.

1 TOOL CALL

```
Bash cd /home/user/claude-code; df -h / | tail -1; free -g | sed -n 2p; echo "--- current artifacts ---"; du -sh outputs/1912/mosaic outputs/1912/tiles work 2>/dev/null
```

CLAUDE 31 Aug 2026, 22:03 UTC

1.11 gigapixels, 3.34 GB canvas — peak ~10 GB against 14 GB free, and 22 GB disk. Feasible. Let me clear the stale artifacts (they're from the pre-solve transforms) and launch.

2 TOOL CALLS

```
Bash cd /home/user/claude-code; find outputs/1912/tiles -type f -delete 2>/dev/null; find outputs/1912/tiles -depth -type d -empty -delete 2>/dev/null; rm -f outp...
```

```
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; nohup python3 -u tools/publish.py --year 1...
```

CLAUDE 31 Aug 2026, 22:03 UTC

Running with all 93 sheets. Let me monitor.

1 TOOL CALL

```
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; for i in $(seq 1 10); do N=$(grep -c compo...
```

CLAUDE 31 Aug 2026, 22:09 UTC

Complete. Let me verify the outputs.

1 TOOL CALL

```
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; tail -8 $S/publish2.log; echo "=== indepen...
```

CLAUDE 31 Aug 2026, 22:09 UTC

All verified: 1.114 gigapixels, DEFLATE, 512×512 tiles, 7 overview levels, 17 DZI levels / 23,233 tiles. Let me smoke-test the product-facing tools against the new recipe.

3 TOOL CALLS

```
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; echo "=== lookup (core) ==="; timeout 300 ...
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; timeout 300 vips resize $S/crop_test.png $...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/crop_test_view.jpg
```

CLAUDE 31 Aug 2026, 22:11 UTC

That's a genuinely saleable 16×20 at 100 ft/in — City Hall & Market, the Cotton Exchange, Grand Opera House, all legible with clean seams. Let me record the results and one limitation the smoke test exposed.

2 TOOL CALLS

```
Bash cd /home/user/claude-code; cat >> HUMAN_QUEUE.md <<'EOF' ## HQ-17 · COG and tiles regenerated — and address lookup still stops at downtown Rebuilt from the f...
SendUserFile
```

CLAUDE 31 Aug 2026, 22:11 UTC

Regenerated and verified. Committed at `b1741c8`, tree clean.

What was built

artifact	
COG	24,317 × 45,816 — 1.114 gigapixels , 915 MB
	DEFLATE, 512 × 512 tiles, 7 internal overview levels; opens under both <code>gdalinfo</code> and <code>vips</code>
DZI	17 levels, 23,233 tiles , 201 MB

Peak memory 10 GB of the 14 GB available — the writer fix from earlier held up at a third more pixels than last time. Neither artifact is committed (the COG is over GitHub's 100 MB file cap), but both rebuild with `python3 tools/publish.py --year 1912`.

I also smoke-tested `crop.py` at the 100 ft/in you confirmed, and it produced the 16×20 I've sent — City Hall & Market, the Cotton Exchange, the Grand Opera House, all legible with clean seams. That's a saleable print off the finished recipe.

One thing the smoke test caught

Address lookup still stops at downtown. A crop anywhere in the ring dies outright:

```
python3 tools/crop.py --year 1912 --street 34 --avenue P ...
KeyError: '18'
```

`grid.json` indexes **streets 18-26 and avenue slots 0-10** — the frozen core and nothing more. The city spans **streets 7-52 and slots 0-27**. So `lookup.py` and `crop.py` serve downtown addresses only, and sheet 72 — which we just placed — is unreachable by address even though its pixels are now in the mosaic.

That's your §1(a), "look up any modern Galveston address and find its lot", still unmet outside downtown. The fix material already exists: `corridor_index.json` holds 33 named corridors located in the mosaic frame, derived from the 154 verified controls. Extending the grid from it — allowing for the $\sim 2^\circ$ bend that makes a street's mosaic coordinate drift along its length — is what would open lookup to the whole city.

Minor note: `crop.py` prints "unowned-sliver fallback (disclosed)" where a crop crosses one of the remaining hairline gaps. It discloses rather than silently filling, which is the right behaviour, but it's a visible reminder those gaps are still there.

So 1912's pixels are done; its **address layer is the last piece**. Say the word and that's the next thing I'd take.

USER 31 Aug 2026, 22:54 UTC

extend the grid city-wide

CLAUDE 31 Aug 2026, 22:55 UTC

Let me see exactly what corridor observations the controls give me, and how well they cover the city.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, os, re, collections, numpy as np; sys.path.insert(0,'tools') from recipelib impor...
```

CLAUDE 31 Aug 2026, 22:55 UTC

Good central coverage — 15 streets (9–42) and 18 avenue slots (2–26), many with multi-thousand-foot spans. Let me fit a family model that captures the bend and fills the unobserved corridors, then check it against the trusted core grid.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, os, re, collections, numpy as np; sys.path.insert(0,'tools') from recipelib impor...
```

CLAUDE 31 Aug 2026, 22:56 UTC

Streets fit cleanly (399.6 ft pitch, 6 ft median against the trusted core), but avenues are badly off — pitch 262 ft where the core says 350. That smells like non-uniform slot spacing: full avenues downtown, but every *half*avenue named in the southern outlots. Let me test that.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, numpy as np; sys.path.insert(0,'tools') ppf=5.7966 g=json.load(open('outputs/191...
```

CLAUDE 31 Aug 2026, 22:56 UTC

The half-step model fixes the pitch (354 ft, matching the core's 350) but residuals stay at ~ 350 ft — one full block, which means some observations are assigned a slot that's off by one. Let me look at the raw sequence rather than fit a model.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, os, re, collections, numpy as np; sys.path.insert(0,'tools') from recipelib impor...
```

CLAUDE 31 Aug 2026, 22:57 UTC

The sequence is non-monotonic — $M^{1/2}$ appears *west* of L, O west of N, P west of O — which is physically impossible. Let me find which controls produce those, since that distinguishes a parsing bug from a real geometry problem.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, os, re, numpy as np; sys.path.insert(0,'tools') from recipelib import Recipe r=Re...
```

CLAUDE 31 Aug 2026, 22:58 UTC

This is a real geometry defect, not parsing. The same Avenue L is placed at $x \approx 15,000$ by sheets 30–57 but at $x \approx 5,800$ by sheets 62–78 — a **1,550 ft row-to-row drift**. Let me confirm directly.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, glob, re, numpy as np; sys.path.insert(0,'tools') from recipelib import Recipe r=...
```

CLAUDE 31 Aug 2026, 22:58 UTC

Decisive: sheets 57 and 63 cover the **identical avenue band** (L, M, $M^{1/2}$, N) in vertically adjacent rows. Their y differs by 1,225 ft — correctly one row — but their **x differs by 1,643 ft**, and there is **no control between them**. Let me get visual proof.

2 TOOL CALLS

```
Bash cd /home/user/claude-code; S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad; timeout 1200 python3 -u tools/render.py --...  
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/rowshear.jpg
```

USER 31 Aug 2026, 22:59 UTC

[Image: original 4000x3000, displayed at 2000x1500. Multiply coordinates by 2.00 to map to original image.]

CLAUDE 31 Aug 2026, 22:59 UTC

That's conclusive. Look at the horizontal band across the middle — **the same "AVENUE M" appears at two different x positions across the row boundary**, displaced by exactly the 1,643 ft I computed. Let me quantify the missing constraint.

1 TOOL CALL

```
Bash cd /home/user/claude-code; timeout 900 python3 - <<'PYEOF' import sys, json, glob, re, os, numpy as np; sys.path.insert(0,'tools') from recipelib import Recip...
```

STOP HOOK 31 Aug 2026, 23:01 UTC

This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation.

Summary:

1. Primary Request and Intent:

The user is building full-city Sanborn fire-insurance mosaics of Galveston, TX for 1899 and 1912, from a brief (PART_B_BRIEF.md) that specifies non-negotiables and deliverables. Requests in chronological order:

- **Part A setup only:** git init, create `inputs/masters/` and move the two master PDFs in, create empty `.claude/agents/`, install libvips and gdal if missing, write `.claude/settings.json` exactly as shown. Confirm each step with the command run. Do NOT start Part B.
- Explain what to paste for Part B.
- Unblock a separate Part B session stuck on four HUMAN_QUEUE items and off-cloud renders.
- "Have Fable 5 decide where to go from here and the switch to Opus for the rest of the session" → then "Okay proceed now."
- "Continue as if no local machine is setup" (all work must stay in the cloud).
- "we can focus on one map at a time... with the map that is more close to being completed first" → 1912 chosen.
- "Have you... what do you recommend we accomplish our goal of finishing the 1912 map?"
- "The map of all of galveston for each year can be bigger than 27x40" (print size constraint lifted).
- "2" (= choose Milestone 2, the full lattice re-solve for lot-level quality).
- "Let's finish one map first" (stay on 1912).
- "Do whatever will recreate what we did in the initial map uploaded" (replicate the master's own build method).
- "Alright create a task for as many opus 5 agents on low effort can be used as possible and finish this map and then stop."
- "proceed"
- "Alright what do you currently need from me?"
- "100 ft/in, and go ahead with the seam re-cut"
- "place sheet 72"
- "fix the 82/89/99 detachment"
- "regenerate the COG and tiles"
- "extend the grid city-wide" (current task)

2. Key Technical Concepts:

- Sanborn fire-insurance map mosaicking; Galveston's regular grid (numbered streets × lettered avenues A-T¹/₂)
- Mosaic frame coordinates, per-sheet similarity transforms (M, t), `p_mosaic = M @ p_native + t`
- **Verified ground scale:** 1912 `px_per_ft = 5.7966`; 1899 corrected to `2.8985`. Block pitch: streets 399.5 ft, avenues 350.4 ft
- Masters print at **~99 ft/in** (not the brief's 50 ft/in); native scale for new crops confirmed as **100 ft/in**
- Ownership/cut polygons (single-writer rule), Voronoi vs street-centreline cuts, half-plane intersection
- Control-based network registration: each control asserts one named corridor is the same line on two sheets, justified by printed address runs (`why_not_one_block_off`)
- COG (Cloud-Optimized GeoTIFF), DeepZoom (DZI) pyramids, pyvips streaming writes
- Opus subagents doing vision-based corridor identification (max 12 concurrent per `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS`)

3. Files and Code Sections:

- `.claude/settings.json` (created Part A, verbatim from brief) — permissions allow list, deny `Bash(rm -rf *)`, `Write(inputs/**)`, `Edit(inputs/**)`; env `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=0`, `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS=12`; SubagentStart/Stop hooks calling `python tools/ledger.py event start|stop`.
- `tools/reciplib.py` — shared recipe loader. Key edits:
 - `fetch()` now materializes missing `LOCAL`: working images from `working_sources.json`
 - Added `materialize(unit, path)` (archival JP2 halved via PIL BOX, or pct50 copy) and `source_bytes(sheet)`
 - `px_per_ft()` corrected:


```
python if recipe.year == 1912: return 5.7966 return 2.8985 # mean of the
avenue/street solves
```
- `tools/render.py` — fixed gigapixel OOM. Added `save()` streaming through `pyvips` (`new_from_memory` zero-copy, BGR→RGB lazily, `tiffsave` tiled/pyramid/bigtiff), and compositing now uses `cv2.copyTo(warped, mask, canvas)` instead of boolean fancy-indexing.
- `tools/lookup.py` (new) — modern address → block, sheet, pixel bounds, optional preview. Uses Galveston hundred-block convention. Reports both flanking blocks; marks numbered-street addresses LOW confidence; states lat/lng unavailable (no EPSG:3857 solve).
- `tools/coverage.py` (new) — writes `outputs/{year}/coverage.png`, coloured by tier, overlays gaps from `qc/gaps.geojson`, counts verified placements separately from `prior-unverified`.
- `tools/tiling.py` (new) — Stage 4 gate as exact geometry: connected pieces, overlap area, interior gaps classified `hole` vs `hairline` via `width = 2*area/perimeter`, `HOLE_WIDTH = 30.0`. Writes `tiling_audit.json` and `gaps.geojson`.
- `tools/fillgaps.py` (new) — assigns interior holes to a neighbour whose scan covers ≥98% of the hole; ownership only.
- `tools/publish.py` (new) — render → COG (`gdal_translate -of COG -co COMPRESS=DEFLATE -co PREDICTOR=2 -co BLOCKSIZE=512`) → `vips dsave`.
- `tools/paircrops.py` (new) — builds per-pair agent tasks: near-native crops per candidate corridor with a GREEN native-pixel ruler, plus `task.json` with axis/keymap/candidates. Falls back to footprints when a unit has no ownership region; handles pairs whose cut regions don't touch.
- `tools/netsolve.py` (new) — solves a translation per sheet by least squares from controls, core frozen; avenue control pins x, street control pins y. Writes `transforms_controls.json`, --apply updates `transforms_city.json` (backup `.pre_controls`).
- `tools/streetcut.py` (new) — cuts ownership on shared corridor centrelines. Region = footprint ∩ half-planes. Two critical fixes:


```
python # sides by which sheet lies lower on the axis, NOT by comparing each centre
to the line g = g.intersection(half_plane(axis, coord, cen[u][k] < cen[v][k]))
python # orientation from how sheets are stacked, not the control's axis d = cen[v]
- cen[u] want = "x" if abs(d[0]) >= abs(d[1]) else "y" if key and cuts[key][0] !=
want: key = None
```
- `tools/recut.py` (new) — Voronoi re-cut (superseded by `streetcut`).
- `tools/lattice.py`, `tools/seamqc.py`, `tools/pairfit.py` (new) — all documented negative results; kept with reasoning.
- `outputs/1912/recipe/` — `working_sources.json` (92→93 units → hash-pinned sources), `corridors.json`, `corridor_index.json` (33 named corridors), `controls/pair_*.json` (154

accepted), transforms_city.json, seams/ownership_city.json.

- HUMAN_QUEUE.md — HQ-1..HQ-17 recorded, including corrections to my own earlier claims.

4. Errors and fixes:

- **numpy broken:** my Part A `apt-get install gdal-bin libgdal-dev` pulled `python3-numpy` built for CPython 3.12 while interpreter is 3.11 → fixed with `pip install --ignore-installed numpy matplotlib`.
- **HQ-6, 1912 recipe not reproducible:** all 92 units pointed at uncommitted `work/sheets/1912w/u*.jpg` → exported `working_sources.json`, patched `reciplib.fetch()`.
- **HQ-10, 1899 crops printed 1.34× off scale:** `px_per_ft` assumed 262 ft avenue pitch (that's the block face, not the pitch) → corrected to 2.8985, verified 3 ways.
- **First scale measurement was circular** — I derived expected pitch from the scale being tested; redone with absolute windows.
- **render.py OOM** at 1/2 scale → `pyvips` streaming writer + `cv2.copyTo`.
- **HQ-11 lattice re-solve failed twice** — snapping through current transforms is circular; ordinal keymap matching misassigns when detection misses a line (core unit 9's corridors sit at slots -1,0,1). Not applied.
- **pairfit area correlation failed** — adjacent sheets share a median of only 494 ft; 75|76 shares just 47 ft. No 2-D patch to correlate.
- **Retracted my own wrong caveat:** I claimed some sheets had wrong transform scale (sheet 41 at 2.099 vs 40's 2.007). Measurement showed **every scale correct to 0.05%** — 41's corridors really are spaced 1103 px vs 40's 1154.
- **streetcut 1.35% overlap** → side assignment must go by relative sheet position.
- **82/89/99 detachment** → cut orientation must follow sheet stacking, not the control's axis.
- `rm -rf` **denied** by project settings (correctly) → used `find -delete / rm -f`.

5. Problem Solving:

Solved: recipe portability, ground-scale calibration for both years, gigapixel rendering, ring re-registration from 154 agent-identified controls (residuals median 0.0 ft / 90th 4.8 ft), street-centreline cuts, sheet 72 placement, group detachment, COG/DZI regeneration.

Currently unsolved and just discovered: the 1912 mosaic has a **~1,643 ft row-to-row east-west shear**. Sheets 57 and 63 cover the identical avenue band (L, M, M½, N) in adjacent street rows; y differs correctly by 1,225 ft but x differs by 1,643 ft, and there is no control between them. Visually confirmed — the same "AVENUE M" renders at two x positions across the row boundary. This explains the non-monotonic avenue index, the 13% x-growth, and why a city-wide avenue grid cannot be fitted.

6. All user messages:

- (with uploads) "Read claude-code-prompt-galveston-fullcity.md completely. Do Part A for me now: git init, create inputs/masters/ and move the two master PDFs into it, create an empty .claude/agents/ directory, install libvips and gdal if they're missing, and write .claude/settings.json exactly as shown in the file. Confirm each step with the command you ran. Do not start Part B yet."
- "/exit" (×2, local command)
- "what do I paste as instructions for part b"
- "It's saying: 'Two things I can't move without you: the four open HUMAN_QUEUE.md items (Part B's text if it still matters, the 1912 D-019/D-023 approval, and the symbol-variance adjudication), and the off-cloud full-resolution renders and DZI, which are one command each and documented in REPORT.md.'"
- "Have Fable 5 decide where to go from here and the switch to Opus for the rest of the session"

- "/model claude-opus-5" then "Okay proceed now"
- "Continue as if no local machine is setup"
- "Also we can focus on one map at a time if needed with the map that is more close to being completed first then continuing on the second map after I say so"
- "Okay how do you recommend we accomplish our goal of finishing the 1912 map?"
- "The map of all of galveston for each year can be bigger than 27x40 just an fyi"
- "2"
- "Let's finish one map first"
- "Do whatever will recreate what we did in the initial map uploaded"
- "Alright create a task for as many opus 5 agents on low effort can be used as possible and finish this map and then stop."
- "proceed"
- (system: container restart notice)
- "Alright what do you currently need from me?"
- "[Stop hook feedback: uncommitted changes...] 100 ft/in, and go ahead with the seam re-cut"
- "place sheet 72"
- "fix the 82/89/99 detachment"
- "regenerate the COG and tiles"
- "extend the grid city-wide"

7. Pending Tasks:

- **Extend the grid city-wide** (current, blocked by the shear discovery)
- Fix the row-to-row x shear by adding cross-row avenue controls (101 pairs identified)
- HQ-9: full visual seam census over all 154 seams (only 5 sampled)
- HQ-7 (1899 71a unverified/misplaced) — deferred, user said one map at a time
- Regenerate COG/DZI again after any further placement changes

8. Current Work:

Working on "extend the grid city-wide". `grid.json` covers only streets 18-26 and avenue slots 0-10 (the frozen core) while the city spans streets 7-52 and slots 0-27, so `crop.py --street 34 --avenue P` fails with `KeyError: '18'`.

I fitted a city-wide grid from the 154 controls:

- **Streets fit well:** pitch 399.6 ft, residual median 22.9 ft, and vs. the trusted core `grid.json` median 6 ft / max 18 ft.
- **Avenues fit badly:** pitch 262.6 ft (core says 350), residual median 370.9 ft. A half-step model for outlot avenues improved pitch to 354.2 ft but residuals stayed ~350 ft.

Direct inspection of the observed avenue sequence showed it is **non-monotonic** (M^{1/2} west of L, O west of N, P west of O) with large per-slot standard deviations (L sd 688 ft, N sd 768 ft). Tracing to source controls showed the split is by sheet row: Ave L is at x≈15,000 for pairs 30|31...56|57 but x≈5,800 for pairs 62|63, 77|78.

Confirmed numerically and visually. The last diagnostic run output:

```
existing controls: 56 avenue(x), 98 street(y) vertically-adjacent pairs sharing >=2
avenues: 106 already have an avenue(x) control: 5 MISSING an x control across the row: 101
The missing-pair list was saved to scratchpad/crossrow.json. Latest commit is b1741c8 ; branch
claude/galveston-setup-part-a-mk5z1l was in sync.
```

9. Optional Next Step:

Report the shear finding to the user and fix it by commissioning avenue (x) controls across the 101 missing vertically-adjacent pairs, using the same `paircrops.py` + Opus-agent + `netsolve.py`

workflow that produced the existing 154 controls — then re-solve, re-cut, and only then extend the grid.

This follows directly from the user's "extend the grid city-wide" request and the immediately preceding work; my last message to the user stated: *"That's conclusive. Look at the horizontal band across the middle — the same 'AVENUE M' appears at two different x positions across the row boundary, displaced by exactly the 1,643 ft I computed. Let me quantify the missing constraint."* The grid cannot be extended until the shear is fixed, since the avenue index is currently non-monotonic. Given the cost of another agent batch, I should confirm with the user before launching ~101 pair tasks.

If you need specific details from before compaction (like exact code snippets, error messages, or content you generated), read the full transcript at: `/root/.claude/projects/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c.jsonl`

Continue the conversation from where it left off without asking the user any further questions. Resume directly — do not acknowledge the summary, do not recap what was happening, do not preface with "I'll continue" or similar. Pick up the last task as if the break never happened.

15 TOOL CALLS

```
Bash git log --oneline -3 && git status --porcelain | head -20 && ls scratchpad/ 2>/dev/null; ls /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a568...
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad && python3 -c " import json d=json.load(open('$S/crossrow.json')) prin...
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad && ls -t $S/*.py | head -20
Bash S=/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad && cat $S/fit.py
Bash python3 - <<'EOF' import json,os,sys,re import numpy as np sys.path.insert(0,'tools') from recipelib import Recipe, px_per_ft from shapely.geometry import Pol...
Bash python3 - <<'EOF' import json,os,sys,collections import numpy as np sys.path.insert(0,'tools') from recipelib import Recipe, px_per_ft from netsolve import lo...
Bash python3 - <<'PY' import re p='tools/paircrops.py' s=open(p).read() s = s.replace( '''def build(year, ua, ub, outdir=None):''', '''def build(year, ua, ub, out...
Bash python3 - <<'PY' p='tools/paircrops.py' s=open(p).read() s = s.replace( ''' if force_axis in ("avenue", "street"): vertical = (force_axis == "avenue") axis =...
Bash python3 - <<'PY' p='tools/paircrops.py' s=open(p).read() s=s.replace(''' ap.add_argument("--pair", nargs=2)''', ''' ap.add_argument("--pair", nargs=2) ap.add...
Bash cat > tools/crossrow.py <<'PY' #!/usr/bin/env python3 """Find the pairs that would tie the sheet rows together in x. python3 tools/crossrow.py --year 1912 [-...
Bash timeout 3000 python3 tools/crossrow.py --year 1912 --prepare 2>&1 | tail -5
Bash sed -i 's/^ crossed = vertical != natural$/ crossed = bool(vertical != natural)/' tools/paircrops.py && grep -n "crossed = " tools/paircrops.py && timeout 30...
Bash ls work/paircrops/ | grep _x | wc -l; ls work/paircrops/25_31_x/; python3 -c " import json;d=json.load(open('work/paircrops/25_31_x/task.json'));print(json.d...
Bash python3 - <<'PY' p='tools/paircrops.py' s=open(p).read() s=s.replace(''' "seam_is": "vertical (sheets side by side)" if vertical else "horizontal (sheets one...
Read /home/user/claude-code/work/paircrops/25_31_x/sheet_25_cand1.jpg
```

CLAUDE 31 Aug 2026, 23:05 UTC

Crops look right — Avenue M and M½ are both legible with the native ruler. One thing they show: the detector's top candidate here is the mid-block **alley** (dashed water-pipe line, 20' label), not an avenue. I'll say so explicitly in the agent brief.

2 TOOL CALLS

```
Bash python3 - <<'PY' import json,os pairs=json.load(open('work/crossrow_pairs.json')) n=12
batches=[[ for _ in range(n)] for i,p in enumerate(pairs): batches[i%...
Bash mkdir -p work/agentprompts && python3 - <<'PY' import json
batches=json.load(open('work/crossrow_batches.json')) TMPL = ""You are a registration observer
fo...
```

CLAUDE 31 Aug 2026, 23:05 UTC

Launching the 12 agents now.

12 TOOL CALLS

```
Agent Read the file /home/user/claude-code/work/agentprompts/batch0.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch1.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch2.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch3.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch4.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch5.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch6.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch7.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch8.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch9.txt and follow it exactly as
your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch10.txt and follow it exactly
as your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/batch11.txt and follow it exactly
as your task instructions.
```

CLAUDE 31 Aug 2026, 23:06 UTC

While those run, let me commit the tooling and get the grid builder ready.

10 TOOL CALLS

```
Bash ls tools/ && echo --- && python3 -c " import
json;d=json.load(open('outputs/1912/recipe/grid.json'));print(json.dumps(d,indent=1)[:1200])"
Bash python3 -c " import json;d=json.load(open('outputs/1912/recipe/grid.json'))
print(list(d.keys())) print('streets',sorted(d['streets'],key=lambda s:int(s))) p...
Bash sed -n 1,90p tools/crop.py
Bash grep -n "def locate" -A 40 tools/reciplib.py
Bash python3 - <<'PY' import json,os,glob,collections c=collections.Counter() for f in
sorted(glob.glob('outputs/1912/recipe/controls/pair_*.json')): d=json.load(...
Bash grep -n "avenue_slot" -B 3 -A 25 tools/reciplib.py | head -60
Bash cat > tools/gridfit.py <<'PY' #!/usr/bin/env python3 ""Rebuild the corridor grid city-wide
from the named controls. python3 tools/gridfit.py --year 1912 [--...
Bash python3 tools/gridfit.py --year 1912 2>&1 | tail -20
Bash git add -A tools work/crossrow_pairs.json outputs/1912/recipe/grid_city.json && git status
--porcelain | head -20
Bash git add -A tools outputs/1912/recipe/grid_city.json state && git commit -q -m "Find and
tool the row-to-row x shear The 1912 ring is sheared east-west from o...
```

CLAUDE 31 Aug 2026, 23:07 UTC

Committed. Here's where things stand while the agents work:

Extending the grid uncovered why it couldn't be extended. The 1912 mosaic has a row-to-row east-west shear. Sheets 57 and 63 cover the identical avenue band (L, M, M½, N) one street-row apart: their y differs by the correct 1,225 ft, their x by 1,643 ft. Visually confirmed — "AVENUE M" renders at two different x positions across the row boundary.

The cause is structural, in the control network rather than the drawings:

- Sheets **side by side** share an avenue → 56 avenue(x) controls chain each row internally.
- Sheets **stacked** share a street → 98 street(y) controls chain each column.
- **Nothing ties one row to the next in x.** With the frozen core counted as one rigid body, the 56 avenue controls leave **26 independent x-components** floating against each other.

That is exactly what `gridfit.py` measures on the current transforms: streets fit a 399.5 ft pitch with a 28.8 ft residual (sound), avenues a 334.6 ft pitch with a **329.2 ft** residual (the shear). It also explains the non-monotonic avenue index I hit earlier — M½ appearing west of L.

The fix, now in flight. A stacked pair abuts along a street but *crosses* every avenue in its band, so the same pair can be asked the other question. `crossrow.py` picks a doubly-redundant spanning set — 45 pairs, so every component join is made twice and a bad call shows up as a contradiction rather than a shear. `paircrops.py` gained an `--axis` override that centres the crop on the shared seam and offers three candidates instead of two, because the detector often locks onto the mid-block alley (20 ft, dashed water-pipe line, no name) rather than the 70 ft named avenue — I verified that on sheet 25 and wrote the distinction into the agent brief.

12 Opus agents are working the 45 pairs now, same method that produced the existing 154 controls: identity decided from printed address runs, with `why_not_one_block_off` evidence required and an honest UNRESOLVED preferred over a guess. When they land: re-solve, re-cut seams, apply the city-wide grid, then regenerate the COG and tiles.

STOP HOOK 31 Aug 2026, 23:08 UTC

Stop hook feedback:

[~/claude/stop-hook-git-check.sh]: There are 1 unpushed commit(s) on branch 'claude/galveston-setup-part-a-mk5z1l'. Please push these changes to the remote repository.

1 TOOL CALL

Bash git push -u origin claude/galveston-setup-part-a-mk5z1l 2>&1 | tail -5

CLAUDE 31 Aug 2026, 23:08 UTC

Pushed — `b1741c8..777a442` on `claude/galveston-setup-part-a-mk5z1l`.

The 12 agents are still working the 45 cross-row avenue pairs. I'll pick up the re-solve, re-cut, grid apply, and tile regeneration when their controls land.

BACKGROUND TASK 31 Aug 2026, 23:16 UTC

a4d86fb51f0dc4b51

```
toolu_01UrGNffHtmD2FAjR4SHU6Ta
/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/a4d86fb51f0dc4b51.output
completed
```

Agent "Cross-row avenue controls b9" finished

A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once. All 12 crops now examined; every sheet's span matches its key map. Sanity checks I ran: the three files parse as JSON, their key sets match each `task.json` "return" schema exactly, `pair / axis / observer` match the spec, coordinates are 1-decimal or null, and I created only the three new `_x.json` files (the pre-existing `pair_37_41.json` street control was left untouched).

37_41 — ACCEPTED (Ave M 1/2). Both sheets letter the corridor "AVENUE M 1/2" inside its own roadway, and on both it is the 1300/1400 hundred-block boundary: sheet 37 has 1302-1324 (17th/18th St) on its west face and 1402-1412 on its east; sheet 41 has 1301-1323 (18th/19th St) west and 1401-1411 east. Ave M one block west is the 1200/1300 boundary (measured at 1145.0 / 1153.3), Ave N one block east is 80 ft wide and the 1400/1500 boundary. Independent validation: the Ave M → Ave M 1/2 spacing measures 1003.7 px on sheet 37 and 1004.5 px on sheet 41 (agreeing to 0.8 px), and both avenues give the same A→B offset (+9.1 vs +8.3 px). None of the six proposed candidates was an avenue — all six land mid-block, so both reported values are ruler-read roadway centres averaged over five disjoint scan bands per sheet.

87_98 and 70_76 — UNRESOLVED. These are not stacked pairs sharing an avenue; the sheets sit in different avenue columns. Sheet 87 runs Ave O→P 1/2 (hundreds 1700-1900, south margin plates 95/96) while sheet 98 runs Ave S→T 1/2 (hundreds 2500-2700, north margin plate 89) — Q, Q 1/2, R, R 1/2 lie between them and are on neither sheet. Sheet 70 runs Ave K→M 1/2 (hundreds 1100-1300, south margin plates 77/78) while sheet 76 runs Ave F→I (hundreds 600-800, north margin plate 68) — sheet 76's easternmost avenue is still well west of sheet 70's westernmost. No avenue crosses both plates in either pair, so any control would have been at least four avenues wrong.

Files written:

```
- /home/user/claude-code/outputs/1912/recipe/controls/pair_37_41_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_87_98_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_70_76_x.json
```

result:

```
37_41_x, Ave M 1/2, a_native 2148.7, b_native 2157.8, ACCEPTED | 87_98_x, none (sheets share no
avenue: O-P½ vs S-T½), null, null, UNRESOLVED | 70_76_x, none (sheets share no avenue: K-M½ vs
F-I), null, null, UNRESOLVED
12724738606312
```

CLAUDE 31 Aug 2026, 23:16 UTC

Important finding from the first agent: two of its three pairs share no avenue at all. My candidate selection used footprint overlap from the *current* transforms — which is circular, since those transforms are what's sheared. Let me redo selection from the key maps, which are transform-independent.

2 TOOL CALLS

```
Bash python3 - <<'PY' import json,glob,os,sys sys.path.insert(0,'tools') from paircrops import
keymap km=keymap('1912') print("keymap units:",len(km)) missing=[] ...
Bash python3 - <<'PY' p='tools/crossrow.py' s=open(p).read() s=s.replace(''from recipelib import
Recipe, px_per_ft # noqa: E402 from netsolve import load_controls...
```

CLAUDE 31 Aug 2026, 23:17 UTC

The key-map selection is right — every pair now genuinely shares printed avenues, including 57|63, the pair that exposed the shear. Let me also fix the crop placement, since the agent reported none of the six proposed candidates was even an avenue.

8 TOOL CALLS

```
Bash python3 - <<'PY' p='tools/paircrops.py' s=open(p).read() s=s.replace(''def build(year, ua,
ub, outdir=None, force_axis=None):'', ''def build(year, ua, ub,...
Bash timeout 600 python3 -c " import sys; sys.path.insert(0,'tools') import paircrops
print(paircrops.build('1912','57','63',force_axis='avenue',shared=['L','M'],'...
Bash python3 - <<'PY' p='tools/paircrops.py' s=open(p).read() s=s.replace(''def build(year, ua,
ub, outdir=None, force_axis=None, shared=None):'', ''def build(...
Bash grep -n "avenues_on_both_sheets\|info\[.crossed.\]" tools/paircrops.py
Bash sed -n 178,196p tools/paircrops.py
Bash python3 -c " import
json;d=json.load(open('work/paircrops/57_63_x/task.json'));print(list(d.keys()));print(d['axis']
,'|',d['seam_is'],'|',d.get('crossed'))"
Bash rm -rf tools/__pycache__ && python3 -c " import sys; sys.path.insert(0,'tools') import
paircrops, inspect print(inspect.signature(paircrops.build)) paircrops...
Bash find tools/__pycache__ -type f -delete; python3 -c " import sys; sys.path.insert(0,'tools')
import paircrops, inspect print(inspect.signature(paircrops.build...
```

CLAUDE 31 Aug 2026, 23:18 UTC

Found it — the parameter `shared` is shadowed by the seam-geometry variable of the same name.

6 TOOL CALLS

```
Bash python3 - <<'PY' p='tools/paircrops.py' s=open(p).read() s=s.replace("shared=None,\n
crossrow=False):","avenues=None,\n crossrow=False):") s=s.replace('' if...
Bash ls outputs/1912/recipe/controls/*_x.json 2>/dev/null | wc -l; python3 - <<'PY' import
glob,json ok=bad=0 for f in sorted(glob.glob('outputs/1912/recipe/contr...
Bash git add -A tools && git commit -q -m "Select cross-row pairs from the key maps, not the
sheared placement The first agent batch rejected two of its three pai...
Bash python3 - <<'PY' import json,glob,os run={f"{a}_{b}" for batch in
json.load(open('work/crossrow_batches.json')) for a,b,_ in batch} want=[(a,b,av) for a,b,av...
Bash timeout 2400 python3 - <<'PY' import json,sys sys.path.insert(0,'tools') import paircrops
todo=json.load(open('work/crossrow_wave2.json')) n=0 for a,b,av in ...
Read /home/user/claude-code/work/paircrops/57_63_x/sheet_57_cand2.jpg
```

CLAUDE 31 Aug 2026, 23:19 UTC

Crops now cover the full avenue band with both Avenue M and M½ legible, plus the plate number in the margin. Dispatching wave 2 into the freed slots.

7 TOOL CALLS

```
Bash python3 - <<'PY' import json todo=json.load(open('work/crossrow_wave2.json')) n=5 batches=
[[] for _ in range(n)] for i,p in enumerate(todo): batches[i%n].app...
Agent Read the file /home/user/claude-code/work/agentprompts/w2_batch0.txt and follow it exactly
as your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/w2_batch1.txt and follow it exactly
as your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/w2_batch2.txt and follow it exactly
as your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/w2_batch3.txt and follow it exactly
as your task instructions.
Agent Read the file /home/user/claude-code/work/agentprompts/w2_batch4.txt and follow it exactly
as your task instructions.
Bash echo '["16_67","49_55","76_84"] # w2_batch4.txt - awaiting a free subagent slot' >
work/pending_dispatch.txt && cat work/pending_dispatch.txt
```

CLAUDE 31 Aug 2026, 23:20 UTC

Progress on the shear fix — 12 accepted cross-row avenue controls so far, and the agents are catching real errors in my own selection:

What the first wave found. Seven pairs came back UNRESOLVED because the two sheets share no avenue at all — sheet 87 runs Ave O-P½ while 98 runs S-T½, with Q, Q½, R, R½ on neither. My selection had tested whether the sheets' *footprints* overlap in x, under the very transforms whose x is what this corrects. Circular. `crossrow.py` now reads coverage off the printed key maps instead: rows must touch but differ, and at least two avenues must be printed on both. All 40 pairs it now proposes share real ground, 57|63 included.

Two traps the agents documented. Every observer independently reported that the corridor detector's candidates were the mid-block 20 ft alley — dashed, labelled `6" W. PIPE`, no name, no hundred-block break — never the 70 ft named avenue. So the crops no longer follow detection at all: three views per sheet at 25/50/75% of its width, covering the whole avenue band, with the red line relabelled a position marker. The second trap is subtler: on 86|94, sheet 94's grid sits one full avenue-block west, so matching by near-equal native x (1548 vs 1552) would have tied Ave N to Ave M½. Every plate shares the same ~999 px lattice, so native proximity is as misleading as mosaic proximity. Both traps are now written into the brief.

Early confirmation the diagnosis is right. On 83|91 the true cross-sheet offset is 0.8 px against ~90 px of current separation — pure shear, exactly as predicted.

Wave 2 covers the key-map pairs; 4 batches are running, one more queued behind the 12-agent concurrency limit. Once they land: re-solve, re-cut, apply the city-wide grid, regenerate the COG and tiles.

STOP HOOK 31 Aug 2026, 23:20 UTC

Stop hook feedback:

[~/./claude/stop-hook-git-check.sh]: There are uncommitted changes in the repository. Please commit and push these changes to the remote branch.

2 TOOL CALLS

```
Bash git status --porcelain
Bash python3 - <<'PY' import glob,json,os acc=[];unr=[];bad=[] for f in
sorted(glob.glob('outputs/1912/recipe/controls/pair_*_x.json')): n=os.path.basename(f)[5:-...
```


CLAUDE 31 Aug 2026, 23:20 UTC

Strong internal cross-validation in these: independent agents reading the same corridor on the same sheet agree closely — Ave D on sheet 28 read as 1176.8 and 1176.6 by two different observers, Ave P on sheet 53 as 1162.0 twice, Ave O on 58 as 2117.8 and 2121.1, Ave M on 31 as 1152.6 and 1147.7. That's ~0–5 native px, under 2 ft on the ground.

4 TOOL CALLS

Bash git add -A outputs/1912/recipe/controls state && git commit -q -m "First 33 cross-row avenue controls from the agent batches 22 ACCEPTED, 11 UNRESOLVED. The ...

Bash python3 - <<'PY' p='work/agentprompts/w2_batch4.txt' s=open(p).read()
s=s.replace("RULES\n", "RULES\n- Other agents are running concurrently and the shared sc...

Agent Read the file /home/user/claude-code/work/agentprompts/w2_batch4.txt and follow it exactly as your task instructions.

Bash rm -f work/pending_dispatch.txt; git add -A outputs/1912/recipe/controls state && git commit -q -m "More cross-row controls; observers keep catching bad pair...

CLAUDE 31 Aug 2026, 23:21 UTC

Wave 2's final batch is dispatched, and the evidence keeps corroborating itself independently:

- Sheet 22 carries the identical Ave D reading (1191.4) in both pairs it appears in; sheet 53 the identical Ave P reading (1162.0); sheet 86's Ave N read 1169.2 and 1167.1 by observers on different batches.
- One observer measured every accepted line at 206–211 px between block faces — 70 ft at the plates' 2.97 px/ft — while every rejected detector candidate measured 60 px, the 20 ft alley. That's a clean quantitative separator, not a judgement call.
- Per-pair second-avenue checks agree on the inter-sheet offset to within a couple of pixels (22→28 gives +14.6 vs +15.9; 58→64 gives +51.5 vs +51.6), where a one-block error would show as ~1000 px.

Three more of my footprint-selected pairs were correctly rejected as sharing no avenue — 41|47, 86|93, 84|95 — each settled by the margin plate numbers (41's south margin reads 45, 47's north reads 26). The observers named the sheets that should have been paired instead, and those re-pairings were already in the key-map selection.

Running total: 25 accepted cross-row controls, 15 rejected pairings. Committed and pushed at 842c50e .

BACKGROUND TASK 31 Aug 2026, 23:21 UTC

a7fafbc39963c329f
toolu_01HZAcZHLp1CUymosYgH9eLb
/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/a7fafbc39963c329f.output
completed

Agent "Cross-row avenue controls b4" finished

A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once. Sanity checks done: all 24 crops read visually; every identification rests on the printed avenue lettering plus the hundred-block address runs on both faces; each accepted pair was cross-checked

with a *second* independently measured avenue, and the two give the same sheet-to-sheet offset (1.5 px, ~4 px, 2 px) where a one-block error would be ~1010 px; each avenue was measured on two separate crops per sheet. All four JSON files parse and carry the full schema.

Written to /home/user/claude-code/outputs/1912/recipe/controls/ : pair_19_23_x.json , pair_61_70_x.json , pair_38_42_x.json , pair_90_98_x.json .

```
19_23_x, Ave H (Ball), 2205.6, 2166.2, ACCEPTED
61_70_x, (none - sheets share no avenue), null, null, UNRESOLVED
38_42_x, Ave O,          2196.7, 2178.9, ACCEPTED
90_98_x, Ave T,          1171.5, 2173.3, ACCEPTED
```

Two things worth passing upstream:

61_70 is not a stacked pair. Sheet 61 runs Ave F (Church) to Ave I (Sealy) — west margin letters "AVE. F. OR CHURCH" next to adjoining plate 60, east margin "AVE. I. OR SEALY" next to plate 62, address runs 600/700/800. Sheet 70 runs Avenue K to Avenue M½ — west margin "AVENUE K" next to plate 69, east margin "AVENUE M½" next to plate 71, address runs 1100/1200/1300. Avenue J (Broadway) lies between them and is on neither sheet; sheet 61's south neighbour is plate 68 while sheet 70's north neighbours are 62 and 63, so the two are not even edge-adjacent. Their candidate lines are different physical streets, and pairing any of them would shear the map.

Detection never once landed on an avenue. On all eight sheets the proposed candidates sit inside the blocks, next to the 20 ft mid-block alley (printed "20'" with a 6" W.PIPE), roughly 400 native px east of the true avenue. Every coordinate I report is a ruler-read avenue centre, not a candidate — each measured as the midpoint of the two block-face lines, which came out 69–71 ft apart, matching the 70' printed beside each corridor. If the same offset shows up in other observers' pairs, the corridor detector is probably keying on the alley/rear-lot line rather than the roadway.

One consistency note for whoever assembles these: sibling pair_23_29_x.json reads sheet 23's Ave H centre as 2163.1 where I read 2166.2. The 3 px gap is in locating the west block face; it does not affect registration, since I used the identical method on both sheets of each pair so any bias cancels in the offset.

result: 4 cross-row x-controls written to outputs/1912/recipe/controls/ — Ave H (Ball) for 19_23, Ave O for 38_42, Ave T for 90_98 all ACCEPTED on printed lettering plus hundred-block address runs; 61_70 UNRESOLVED because sheets 61 (Ave F–I) and 70 (Ave K–M½) share no avenue.
16403055929315

CLAUDE 31 Aug 2026, 23:21 UTC

A systematic finding worth recording: the corridor detector never once landed on an avenue across eight sheets — it sits ~400 native px east of the roadway, on the alley/rear-lot line.

3 TOOL CALLS

```
Bash tail -30 HUMAN_QUEUE.md
```

```
Bash grep -n "^## HQ-" HUMAN_QUEUE.md | tail -6
```

```
Bash cat >> HUMAN_QUEUE.md <<'MD' --- ## HQ-18 · The 1912 rows are sheared east-west, and the corridor detector never finds avenues Two findings, both from extend...
```

CLAUDE 31 Aug 2026, 23:22 UTC

Recorded as HQ-18 and pushed. The detector caveat matters beyond this pass: the existing 154 controls were proposed by the same detector, so a control that took the alley on one sheet and the avenue on the other would be off by ~70 ft and would *not* show in the residuals. A

uniform bias cancels in the offset — which is all `netsolve` uses — so registration isn't necessarily wrong, but the accepted controls deserve a pass against the 70 ft width test. I've flagged it as not yet done rather than assuming it's fine.

Nine agents are still working. Once they land I'll re-solve, re-cut, apply the city-wide grid, and regenerate the COG and tiles.

STOP HOOK 31 Aug 2026, 23:22 UTC

Stop hook feedback:

[~/./claude/stop-hook-git-check.sh]: There are uncommitted changes in the repository. Please commit and push these changes to the remote branch.

3 TOOL CALLS

Bash git status --porcelain

Bash git diff --stat state/events.jsonl && tail -2 state/events.jsonl

Bash git add state/events.jsonl && git commit -q -m "Ledger: subagent start/stop events for the cross-row batches Co-Author: Claude Opus 5 <noreply@anthropic..."

CLAUDE 31 Aug 2026, 23:22 UTC

Tree is clean and pushed at `07c2772` — that last change was only the subagent start/stop ledger, which the hooks append as agents come and go.

One correction to something I said earlier: you asked for the agents to run at **low** effort, and the ledger shows they're recording `"effort":{"level":"high"}`. The `Agent` tool in this build exposes only a model override, not an effort setting — that comes from the agent definition — so I couldn't honour that part of the instruction. It's why each batch is running ~15 minutes and ~140k tokens rather than something much cheaper. If you want them genuinely low-effort for any future batch, that needs an agent definition in `./claude/agents/` with the effort set in its frontmatter, which I can write.

Nine agents are still working the remaining pairs.

BACKGROUND TASK 31 Aug 2026, 23:24 UTC

```
a421d0fe55531388d
toolu_01HKwVbBFr6rjMiHkmk2qmrn
/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/a421d0fe55531388d.output
completed
```

Agent "Cross-row avenue controls b8" finished

A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once. Sanity checks complete: all four files parse, carry every schema key, use observer `crossrow-agent-8` (and are the only files that do), coordinates are rounded to 1 decimal, `inputs/` is untouched, and I ran `no netsolve/render/git`.

What I did for each pair: read both sheets' crops, identified the avenue from its printed name plus the hundred-block address runs on the flanking block faces, then read the corridor's two block faces off the green native-pixel ruler and took the centre. Detection's candidates were useless on every

sheet — all twelve fall inside blocks on lot lines, roughly a third of a block east of a real avenue — so none of my reported coordinates is one of the proposed candidates.

Key findings:

- **50/56** — Broadway is unique on the island, so there is no one-block-off risk at all. It carries the 900→1000 block change on both sheets (918-924 west, 1002-1024 east), with its own frontage in the 2500 block on sheet 50 and the 2700 block on sheet 56 — the same avenue seen at two different street ranges, exactly as a stacked pair must show.
- **59/65** — Ave P½, lettered and marked 70' on both, carries the 1900→2000 change on both. Ave P measured independently gives the same sheet-to-sheet offset to 1.3 px (16.7 vs 15.4), against ~1007 px for a one-block error.
- **20/25** — Avenue L is the only avenue the two sheets share, and each sheet is trimmed *inside* it: sheet 20 shows only its west face, sheet 25 only its east face. I reconstructed the centre on each sheet from its one visible face plus the avenue width measured on that same sheet (Ave K on 20, Ave M on 25); both reconstructed centres land inside their own sheet's printed area, so the sheets genuinely overlap across the line. Flagged in the file as carrying ~±5 px extra uncertainty.
- **80/90** — the pair's premise is false. Sheet 80 ends at Avenue Q and sheet 90 begins at Avenue S½; the 2100-2500 blocks lie between and appear on neither. The sheets' own continuation numerals settle it: sheet 80 prints "88" in its south margin (its southern neighbour is 88, not 90) and sheet 90 prints "82" north and "89" west. They are diagonal neighbours about two sheet-widths apart, so no line on one is the same physical line on the other. Reported UNRESOLVED with null coordinates rather than a guess.

Files written (absolute paths):

- /home/user/claude-code/outputs/1912/recipe/controls/pair_50_56_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_59_65_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_20_25_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_80_90_x.json

One line per pair:

```
50_56_x, Ave J (Broadway), a_native 1153.5, b_native 1154.5, ACCEPTED
80_90_x, (none - sheets share no avenue), a_native null, b_native null, UNRESOLVED
59_65_x, Ave P 1/2,          a_native 2153.4, b_native 2168.8, ACCEPTED
20_25_x, Ave L,             a_native 3291.7, b_native 111.8, ACCEPTED
```

result: 3 of 4 cross-row x-controls ACCEPTED from printed evidence (50/56 Broadway 1153.5/1154.5; 59/65 Ave P½ 2153.4/2168.8; 20/25 Ave L 3291.7/111.8); 80/90 UNRESOLVED because sheets 80 and 90 do not adjoin — sheet 80 ends at Ave Q, sheet 90 begins at Ave S½, and sheet 80's printed south neighbour is sheet 88.
170586471076907

1 TOOL CALL

Bash git add -A outputs/1912/recipe/controls state && git commit -q -m "Cross-row batch 8: Broadway, Ave P 1/2, and an edge-trimmed Ave L 50|56 is the strongest c..."

CLAUDE 31 Aug 2026, 23:24 UTC

Batch 8 in. Two notable ones:

50|56 is the strongest control in the whole set — Broadway is unique on the island, so the one-block-off failure mode doesn't exist for it. It carries the 900→1000 break on both sheets,

and its own frontage reads 2500 on sheet 50 and 2700 on sheet 56: the same avenue seen at two different street ranges, which is exactly what a genuine stacked pair must show.

20|25 is the weakest, and is marked so in the file. Avenue L is the only avenue those two share, and each plate is trimmed *inside* it — sheet 20 prints only its west face, sheet 25 only its east face. The observer reconstructed each centreline from its one visible face plus the avenue width measured on that same sheet, and recorded the extra ~5 px of uncertainty. The near-edge coordinates (3291.7 and 111.8) are correct for that geometry rather than a transcription slip, which is worth knowing when the residuals come out.

Eight agents still running.

BACKGROUND TASK 31 Aug 2026, 23:26 UTC

```
a68c26053d2ee2f2e
toolu_01VXetKDFyiQ37CcdEqFSGvT
/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/a68c26053d2ee2f2e.output
completed
```

Agent "Cross-row avenue controls b11" finished

A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once.

Verified: all three files parse, match the required schema exactly, and I wrote only those three (the other recent files in that directory are sibling agents' work). A sibling independently placed sheet 84's Ave I centre at 2173.4 vs my 2168.6 — consistent within the plate skew I documented.

What I did: I confirmed the supplied `sheet_<UNIT>_cand<N>.jpg` crops are exact views of `work/sheets/1912w/u{unit}.jpg` (the red line at native $x=1445$ reproduced to 0.1 px off the green ruler), so I read the corridors directly off the native plates, measuring each avenue as the midpoint of its two drawn block-face lines rather than eyeballing the ruler.

Key findings

- **14/16** — Both plates letter `AVE. D. OR MARKET` in the roadway; the 300-block (319 321 323) lies west and the 400-block (401 403 405...) east on both. Roadway widths 211.8 / 211.6 px = the printed 70' (the mid-block alleys measure ~58 px). Four independent corridors give the same offset (+24.6, +25.8, +27.7, +25.4 px), vs ~1011 px for a one-block error. None of the six proposed candidates is a corridor at all.
- **84/93** — These are **diagonal** neighbours (84's east margin prints 85, 93's west margin prints 92); Broadway/Ave J is their only shared corridor, as the 900/1000 block boundary. Critically, **neither sheet draws all of Broadway** — 84 shows only its west face (3060.4), 93 only its east face (279.9) — so I report the centreline. Broadway is drawn *compressed*: annotated "150 ft wide" but drawn ~301 px where a 70 ft avenue is ~209 px. I measured its drawn width on the three plates showing both faces (69, 77, 56): mean 301.2 px, sd 1.1 → half-width 150.6 ± 0.6 . The derived centres sit 1042.4 px from Ave I on 84 and 1042.2 px from Ave K on 93 — an independent agreement. Full derivation is disclosed in the file.
- **69/78** — The pair's premise fails. Sheet 69's easternmost corridor is Avenue K (1000-block to its west); sheet 78's westernmost is Avenue L (1200-block to its east). The intervening **1100 block appears on neither sheet** — it is on plates 70 and 77. No shared line exists, so I reported nothing rather than shear the map.

result:

14_16_x → Ave D (Market), a=1134.1, b=1159.9, ACCEPTED | 84_93_x → Ave J (Broadway),
a=3211.0, b=129.3, ACCEPTED (centreline derived from measured faces + Broadway's 301.2 px
drawn width; derivation documented) | 69_78_x → none, a=null, b=null, UNRESOLVED (sheets share
no avenue; the 1100 block between Ave K and Ave L is on neither plate)
184926611196059

1 TOOL CALL

Bash git add -A outputs/1912/recipe/controls state && git commit -q -m "Cross-row batch 11: Ave
D, and Broadway derived from its drawn width 84|93 needed a deriva..."

CLAUDE 31 Aug 2026, 23:26 UTC

Batch 11 in, and it surfaced a drafting fact worth carrying forward.

84|93 needed a derivation, not a read. Neither plate draws all of Broadway — 84 shows only its west face, 93 only its east — so the observer measured Broadway's drawn width on the three plates that *do* show both faces (69, 77, 56): 301.2 px, sd 1.1, giving a half-width of 150.6 ± 0.6. The two derived centrelines then land 1042.4 px from Ave I on sheet 84 and 1042.2 px from Ave K on sheet 93 — an agreement the derivation wasn't fitted to, so it's a genuine check. The working is disclosed in the control file.

The side finding matters more. Broadway is annotated 150 ft but drawn at 301 px, where a 70 ft avenue is drawn at ~209 px. At the plates' 2.97 px/ft that's 101 ft, not 150 — Broadway is drafted compressed. The controls themselves are unaffected, since they compare centreline to centreline, but any block pitch measured *across* Broadway will read short. I've flagged avenue slot 9 to watch when the grid fit runs.

Seven agents still working.

BACKGROUND TASK 31 Aug 2026, 23:26 UTC

a490ec58f6396a90c
toolu_01PvavKc8LCZoh98Jy9iAbNo
/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/a490ec58f6396a90c.output
completed

Agent "Cross-row avenue controls b5" finished

A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once. Sanity check passed: all four files are valid JSON with exactly the schema keys from each `task.json`, coordinates rounded to 1 decimal, `inputs/` untouched, and no `netsolve/render/git` run. I only wrote the four prescribed output files.

Method. For each pair I calibrated every crop from its own green ruler (a clean 68.0 image-px per 100 native-px, anchored on the red line), then identified the avenue from the printed lettering and the hundred-block address runs, never from ordinal position or current mosaic proximity. I took each avenue's native x as the midpoint of its two facing block lines, measured in the block band immediately adjacent to the shared seam and averaged over the two crops that show the corridor.

The candidates were almost all wrong. On every sheet in all four pairs, corridor detection had locked onto mid-block lines near the 20-ft alleys, not avenues — so none of my reported coordinates

is a proposed candidate. Each avenue I report measures ~212 native px between faces, matching the 70' printed in the roadway.

68/77 is genuinely unresolvable. The two plates share no avenue: sheet 68 letters Ave E-H and ends just east of Ave H (native ~3280); sheet 77 letters Ave I-L and begins inside Ave I. Three independent confirmations — the Galveston block numbers (block = C - 60·avenue + street, which I validated against pair 23/29's blocks 310-313), the printed margin indices (sheet 68's south neighbour is 76, not 77; sheet 77's north is 69), and the physical plate edges. They are diagonal neighbours, so an x-tie must chain 68→69→77 or 68→76→77.

Two accepted controls carry a recorded plate-scale disagreement (23/29: ±2 px of corridor-choice sensitivity; 27/33: Water gives an 8.2 px shift where the Strand gives 3.9), documented in the files rather than reconciled. My sheet-23 and sheet-29 readings are independently corroborated by agents 4 and 10, who measured the same Ave H at the opposite seams of those sheets — their values differ from mine by exactly the rotation drift I measured down each plate.

result:

- 23_29_x, Ave H (Ball), a=2158.8, b=2144.8, ACCEPTED
- 68_77_x, none — plates share no avenue (68 covers Ave E-H, 77 covers Ave I-L), a=null, b=null, UNRESOLVED
- 51_57_x, Ave M, a=1133.8, b=1191.7, ACCEPTED
- 27_33_x, Ave A (Water), a=1165.8, b=1157.6, ACCEPTED

191046501237691

1 TOOL CALL

```
Bash git add -A outputs/1912/recipe/controls state && git commit -q -m "Cross-row batch 5: offsets agree even where absolute reads differ This batch revised two f...
```

CLAUDE 31 Aug 2026, 23:27 UTC

Batch 5 in, with a result that directly validates the method's error model. The agent revised two of its own earlier files — sheet 23's Ave H from 2163.1 to 2158.8, sheet 29's from 2149.4 to 2144.8 — but the **offset** held at -13.7 vs -14.0 px. Same on 51|57: absolute reads shifted by 1.2 px each, offset identical at +57.9. Since `netsolve` uses only the offset, a shared bias in locating a block face cancels exactly. That's the drift down each plate the observers keep reporting, and it's harmless.

68|77 was rejected three independent ways: lettered coverage (E-H vs I-L), printed margin indices (68's south neighbour is 76, not 77), and Galveston's block-numbering formula, which the agent validated against pair 23|29's known blocks before trusting it. An x-tie there has to chain 68→69→77.

Six agents still running.

BACKGROUND TASK 31 Aug 2026, 23:33 UTC

```
a1a29407f3c4d4953
toolu_0127Gj2JrWXrVb3HLsFMkFz8
/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/a1a29407f3c4d4953.output
completed

Agent "Cross-row avenue controls b6" finished
```


A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once. Sanity checks done: all four JSON files parse and carry every key in task.json's `return` schema; every reported coordinate was re-measured independently in the native sheet raster (`work/sheets/1912w/u<unit>.jpg`, 3327×3898) as well as off the crop rulers, and each agrees with existing controls in the network — `pair_21_27_x` reads sheet 21's Strand at its opposite (12th St) seam as 2156.7 where my full-height profile of the same corridor gives 2157.2–2157.6; `pair_20_24_x` reads sheet 24's Ave K at its opposite (9th St) seam as 2194.0, continuous with my 2209.8 at 12th St; `pair_55_56` and `pair_16_61` corroborate sheets 55 and 61 to ~6 px. Scratch scripts were deleted; only the four control files were written.

Key findings worth flagging:

- **Every one of the 24 proposed candidate lines is wrong.** All of them land on mid-block lot lines or beside the 20-ft alley, with the hundred-block run continuing unbroken across them. On 17/21 the detector even returned the identical triple (1763/748/2778) for both sheets. All four reported corridors are ruler-read avenues, not candidates.
- **task.json's keymap for sheet 17 is attached to the wrong unit.** It describes an isolated SW outlot plate at 50th–52nd Sts with no avenues; the actual u17 raster shows Seawall Blvd, 8th/9th Sts, John Sealy Hospital, letters Water/Strand/Mechanic, and prints "21" in its bottom margin. I went with the printed evidence, so this pair resolved.
- **77/83 is a spurious pair.** Sheet 77 spans Ave I→L, sheet 83 spans Ave D→G; Avenue H and the whole 800-block column lie between them and appear on neither plate. The margins agree: 77's bottom margin prints 84/85, 83's top margin prints 75/76. No shared avenue exists, so no coordinate is given.
- **24/30 carries a recorded disagreement.** The two plates differ ~1.3% in drafted scale across this span, so Broadway gives $b-a = +17.6$ while Ave K gives $+4.0$. I reported Ave K (crisper 70-ft casings, and it is the corridor sheet 24's other x-control already uses) and wrote the discrepancy and a ± 7 px residual into the file rather than hiding it.

Per pair — PAIR, corridor, a_native, b_native, status:

- 17_21_x, Ave B (Strand), 2147.7, 2160.5, ACCEPTED
- 77_83_x, none — sheets share no avenue, null, null, UNRESOLVED
- 55_61_x, Ave G (Winnie), 1143.4, 1156.2, ACCEPTED
- 24_30_x, Avenue K, 2209.8, 2213.8, ACCEPTED

Files written:

```
- /home/user/claude-code/outputs/1912/recipe/controls/pair_17_21_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_77_83_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_55_61_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_24_30_x.json
```

result: 3 of 4 stacked pairs tied in x from printed avenue names and hundred-block address runs (17/21 Strand 2147.7→2160.5, 55/61 Winnie 1143.4→1156.2, 24/30 Avenue K 2209.8→2213.8); 77/83 reported UNRESOLVED because the sheets share no avenue at all (I-L vs D-G), and all 24 detector candidates were rejected as mid-block lot lines.

219165761645721

A keymap data error surfaced — sheet 17's entry describes the wrong plate. Let me verify that directly.

1 TOOL CALL

```
Bash grep -rl '"sheet": 17' rebuild_1899/out/keymap_1912_*.json 2>/dev/null | head; python3 -
<<'PY' import glob,json for f in sorted(glob.glob('rebuild_1899/out/...
```

CLAUDE 31 Aug 2026, 23:33 UTC

Confirmed — and it's a collision, not a typo. Two keymap files both claim sheet 17, and my loader is last-write-wins.

3 TOOL CALLS

```
Bash python3 - <<'PY' import glob,json,collections seen=collections.defaultdict(list) for f in
sorted(glob.glob('rebuild_1899/out/keymap_1912_*.json')): for e in ...
Bash python3 - <<'PY' import cv2,sys sys.path.insert(0,'tools') from recipelib import Recipe
r=Recipe(1912) img=cv2.imread(r.fetch(r.sheet_file('32'))),cv2.IMREAD_C...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/scratchpad/u32_top.jpg
```

CLAUDE 31 Aug 2026, 23:34 UTC

Sheet 32's top margin prints "25" and "12TH ST." — matching the NW entry (streets 12–15), so the NE entry is the spurious one. Both collisions silently dropped a sheet from my selection.

3 TOOL CALLS

```
Bash python3 - <<'PY' p='tools/paircrops.py' s=open(p).read() s=s.replace(''def keymap(year):
km = {} for f in glob.glob(os.path.join(REPO, "rebuild_1899", "out"...
Bash python3 tools/crossrow.py --year 1912 2>&l | head -8; echo ...; git add -A tools
outputs/1912/recipe/controls state && git commit -q -m "Fix two silent keyma...
Bash python3 tools/crossrow.py --year 1912 2>/dev/null | sed -n '4,20p'; echo "=== components
==="; python3 - <<'PY' import json,os,sys,collections sys.path.inser...
```

CLAUDE 31 Aug 2026, 23:35 UTC

All six remaining joins are already in flight in wave 2 — no new dispatch needed.

The count has moved decisively: **94 avenue(x) controls, and the 26 independent x-components have collapsed to 4**. 81 of the 93 sheets are now a single rigid body in x. What's left is three small islands — {75, 76, 83, 84, 91, 92, 93}, {95, 96, 97}, and {90, 98} — and the pairs that would join each of them (67|75, 85|93, 87|95, 88|96, 82|90, 90|99) are all in the five batches still running.

The keymap fix contributed directly here: correcting the sheet 17 and 32 collisions took the usable key maps from 91 sheets to 92 and the candidate pairs from 98 to 65 well-founded ones, because the loser entries that had been winning carried no avenues at all.

Also recorded from batch 6: 24|30 carries a genuine ~1.3% drafted-scale disagreement between the plates — Broadway gives a sheet-to-sheet shift of +17.6 px where Ave K gives +4.0. The observer reported Ave K and wrote the discrepancy and a ± 7 px residual into the file rather than reconciling it away.

STOP HOOK 31 Aug 2026, 23:35 UTC

Stop hook feedback:

[~/claude/stop-hook-git-check.sh]: There are untracked files in the repository. Please commit and push these changes to the remote branch.

2 TOOL CALLS

Bash git status --porcelain

Bash python3 - <<'PY' import json for n in ("12_14","57_63","67_75","88_96"): d=json.load(open(f'outputs/1912/recipe/controls/pair_{n}_x.json')) print(n, d.get('s...

CLAUDE 31 Aug 2026, 23:35 UTC

57|63 is resolved — the very pair that exposed the shear. Ave M½ reads 2199.8 on sheet 57 and 2183.3 on 63: a 16.5 px difference, about 5.7 ft. The current placement has them 1,643 ft apart.

1 TOOL CALL

Bash git add -A outputs/1912/recipe/controls state && git commit -q -m "57|63 resolved: the pair that exposed the shear is 5.7 ft apart, not 1,643 Ave M 1/2 reads...

CLAUDE 31 Aug 2026, 23:35 UTC

The trap warning I added to the wave-2 brief earned its place: on 67|75, sheet 67's Ave C (1180.5) and sheet 75's Ave D (1129.0) sit only ~51 native px apart, and on 88|96 the gap is ~21 px. Native-x matching would have produced a full-block shear on both. Let me see where the solve stands with 94 controls.

2 TOOL CALLS

Bash python3 tools/netsolve.py --year 1912 2>&1 | tail -25

Bash python3 - <<'PY' import json,os,sys,collections sys.path.insert(0,'tools') from recipelib import Recipe from netsolve import load_controls r=Recipe(1912) gj=j...

CLAUDE 31 Aug 2026, 23:35 UTC

The network has essentially closed. **26 x-components down to 2**: 91 of 93 sheets are one rigid body, with only {90, 98} still floating — and the pairs that would join it (82|90, 90|99) are in the two batches still running.

A dry solve with the 94 avenue controls in place:

196 usable controls (98 avenue, 98 street)
residuals (ft): median 0.7, 90th 5.9, max 33.6
81 free sheets constrained, 0 unconstrained
moves (ft): median 271, max 2964

Two things worth reading carefully there. **Zero unconstrained sheets** — previously the ring had sheets the solve could only leave where they sat. And the **moves are large** (median 271 ft, up to 2,964 ft for the 95/96/97 group) because that is the shear being taken out; the residuals staying at median 0.7 ft while sheets move thousands of feet is the sign the controls agree with each other rather than fighting.

The eight worst residuals are all *street* (y) controls at 15th and 18th St, 28–34 ft — pre-existing constraints now being pulled slightly by the stiffer network. Max is unchanged from the previous solve at 33.6 ft, so nothing regressed. I'll hold off applying until the last two batches land and {90, 98} is tied in.

BACKGROUND TASK 31 Aug 2026, 23:37 UTC

```
a6352f81cd7e33622
toolu_01QJeBe5wP5nBmpDYVMNJTZC
/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-
a56890968b0c/tasks/a6352f81cd7e33622.output
completed
```

Agent "Cross-row wave2 b2" finished

A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once. Sanity check passed: all four files parse, carry exactly the schema keys from each `task.json`, coordinates at 1 decimal, `axis / observer` correct, and only my four `pair_*_x.json` files were written (temp scripts removed; the modified `transforms_controls.json` / `events.jsonl` are other agents', untouched by me). I used `read-only git status` for that check only — no commits, no `netsolve/render`, nothing under `inputs/` touched.

Method: I identified each corridor from printed evidence only — the "AVENUE <letter>" lettered in the roadway plus the hundred-block address runs on its two faces — then measured the roadway centreline (midpoint of the facing block lines) against the native pixel grid the green ruler labels, in seven y bands per sheet, fitted for scan tilt and evaluated at mid-sheet. Every pair was cross-checked on a second lettered avenue.

Three of the four pairs are genuinely offset by whole blocks in native x, and in all three the near-equal-native-x trap would have given the wrong answer:

- 61/68: sheet 61's Ave G is within 16 px of sheet 68's Ave F — matching on x would pair Winnie with Church (one block).
- 87/95: sheet 87's Ave O ½ is within 10 px of sheet 95's Ave O (one block).
- 85/94: sheet 85's Ave M is within 26 px of sheet 94's Ave N — two blocks, the same failure flagged for 86/94.

56/62 turned out genuinely near-co-registered (–22 px), but that was the output of the name/address identification, not an input.

Corroborating avenue pairs agreed to 0.3, 1.5, 1.8 and 4.2 px respectively, against a one-block scale of ~1000 px. Alleys (~59 px, dashed, water-pipe labelled, no hundred-block change) were never confused with avenues (~211 px = 70 ft). The red marker landed mid-block on every crop, as warned, and none of the offered candidates was a lettered avenue.

result:

```
56_62_x, Ave K, 2198.5, 2176.6, ACCEPTED | 87_95_x, Ave O 1/2, 1149.8, 2140.2, ACCEPTED |
61_68_x, Ave G (Winnie), 1148.2, 2165.7, ACCEPTED | 85_94_x, Ave M, 2196.9, 174.3, ACCEPTED
170692611054152
```

1 TOOL CALL

```
Bash git add -A outputs/1912/recipe/controls state && git commit -q -m "Cross-row wave 2 batch
2: four more, three of them one-block traps 61|68, 87|95 and 85|94 ...
```

CLAUDE 31 Aug 2026, 23:37 UTC

Four more in, and three were one-block traps the near-equal-native-x warning caught: sheet 61's Ave G sits within 16 px of sheet 68's Ave F, 87's Ave O½ within 10 px of 95's Ave O, and 85's Ave M within 26 px of 94's Ave N — two blocks off, the same failure mode already flagged on 86|94. Matching by native proximity would have taken every one of them.

56|62 *did* come out near-co-registered at −22 px, but the observer was careful to note that came out as the **output** of the name-and-address identification rather than as an input to it. That's the distinction that keeps the method non-circular.

Three agents left.

STOP HOOK 31 Aug 2026, 23:37 UTC

Stop hook feedback:

[~/claude/stop-hook-git-check.sh]: There are uncommitted changes in the repository. Please commit and push these changes to the remote branch.

1 TOOL CALL

Bash git status --porcelain

CLAUDE 31 Aug 2026, 23:38 UTC

That's the dry-run solve output — regenerable, but it does record the network's state at 94 controls, so I'll commit it rather than discard it.

1 TOOL CALL

Bash git add outputs/1912/recipe/transforms_controls.json && git commit -q -m "Dry-run solve at 94 avenue controls (not applied) transforms_controls.json is netso..."

CLAUDE 31 Aug 2026, 23:38 UTC

Committed and pushed at 311597e. To be clear about what that file is: it's netsolve's *proposal*, not the live placement — transforms_city.json is untouched, so the published mosaic still has the shear. I'll regenerate and apply it once the last three batches land and the {90, 98} island is tied in.

BACKGROUND TASK 31 Aug 2026, 23:41 UTC

ad235e3be1991da69
toolu_011WzrZoqaXFTQRxngqPA7r2
/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/tasks/ad235e3be1991da69.output
completed

Agent "Cross-row wave2 b1" finished

A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once. Sanity check done: all four files re-parse, match their task.json return schemas exactly (no missing/extra keys, correct pair / axis, coordinates at 1 dp), and every read was reproduced on 2–3 separate block rows per sheet. Every reported corridor measures 207–214 native px between block

faces — the printed 70 ft — so none is a ~20 ft alley. Nothing under `inputs/` was touched, and I ran no `netsolve` or `render`.

All four resolved on printed evidence (avenue name lettered in the roadway on both sheets + matching hundred-block address break + the avenue's own frontage numbers continuing between sheets + outlot column numbers):

- **46_52_x** — Ave O — `a_native` 2175.3, `b_native` 2188.5 — ACCEPTED
- **75_83_x** — Ave E (Post Office) — `a_native` 2140.3, `b_native` 1167.7 — ACCEPTED
- **26_47_x** — Ave P — `a_native` 1168.0, `b_native` 1184.2 — ACCEPTED
- **82_90_x** — Ave S 1/2 — `a_native` 2185.3, `b_native` 173.8 — ACCEPTED

Three things worth flagging to the batch:

1. **The near-equal-native-x trap fired on two of my four pairs.** On 75/83 the plates are framed one avenue-block apart (75 covers C-F, 83 covers D-G), so sheet 75's Ave E (2140.3) sits 33 px from sheet 83's Ave F (2173.4) — an x-proximity match would have been one block off. On 82/90 the plates are framed **two** blocks apart; sheet 82's Ave S½ (2185.3) sits ~13 px from sheet 90's unopened platted Ave T½ (~2175), and sheet 82's Ave S (1178.0) sits ~9 px from sheet 90's Ave T (1169.3). True offset there is about -2011 px, not ~0.
2. **Sheet 26 carries a detached inset** in its upper-right Gulf corner (outlot quarters S.W. 71 / S.E. 70, 16TH-18TH ST) that is lettered "AVENUE O 1/2" at native $x \approx 2118$, beyond a heavy divider line at 2010. That lettered avenue is *not* on sheet 26's own lattice and lands within ~80 px of sheet 47's Ave P½ — a name-plus-nearest-x match would tie two different avenues, neither in the main frame. Other observers working sheets with seawall/beach corners should watch for this.
3. **The proposed candidates were unusable on all four pairs** — every one landed in a block interior, on a rail pair, or on an interior lot line, never on a 212 px roadway. All reported values are ruler-read avenue centres from the block faces.

The key-map rectangles turned out to be a cheap independent check on framing shift (46/52 both x 3040-3410 → no shift; 75 x 1550-1925 vs 83 x 1675-2045 → +125 = exactly one avenue column), and I cited it in each file.

Files written (only my own four):

```
- /home/user/claude-code/outputs/1912/recipe/controls/pair_46_52_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_75_83_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_26_47_x.json
- /home/user/claude-code/outputs/1912/recipe/controls/pair_82_90_x.json
```

result: All 4 cross-row avenue controls resolved ACCEPTED on printed evidence — 46/52 Ave O (2175.3/2188.5), 75/83 Ave E (2140.3/1167.7), 26/47 Ave P (1168.0/1184.2), 82/90 Ave S½ (2185.3/173.8); the near-equal-native-x trap would have mis-tied 75/83 by one block and 82/90 by two.

185511851288614

CLAUDE 31 Aug 2026, 23:41 UTC

82|90 landed — that ties the last island in. Let me commit and confirm the network is closed.

1 TOOL CALL

```
Bash git add -A outputs/1912/recipe/controls state && git commit -q -m "Wave 2 batch 1: the last island tied in, plus a detached-inset trap 82|90 on Ave S 1/2 joi..."
```

CLAUDE 31 Aug 2026, 23:41 UTC

The x network is closed. 102 avenue controls, one single component — every one of the 93 sheets is now rigidly tied in x to the frozen downtown core. That was 26 disconnected components when I started this.

Batch 1 also turned up a new failure mode worth guarding against: sheet 26 carries a **detached inset** in its upper-right Gulf corner, beyond a heavy divider at native 2010, lettered "AVENUE O ½" at $x \approx 2118$. That lettered avenue isn't on sheet 26's own lattice, and it lands within ~80 px of sheet 47's Ave P½ — so a name-plus-nearest-x match would tie two different avenues, *neither* of which is in the main frame. Any plate with a seawall or beach corner can do this.

The same observer found a cheap independent check I hadn't thought of: the key maps' own rectangles measure the framing shift directly. Sheets 46 and 52 both sit at x 3040–3410, so no shift; 75 at 1550–1925 against 83 at 1675–2045 is +125, exactly one avenue column. Let me build that into a validator.

1 TOOL CALL

```
Bash cat > tools/ctlcheck.py <<'PY' #!/usr/bin/env python3 """Check accepted controls against evidence they were not fitted to. python3 tools/ctlcheck.py --year 1...
```

CLAUDE 31 Aug 2026, 23:42 UTC

My framing check is measuring the right quantity with a wrong scale — a key-map rectangle spans several avenue columns, not one. Let me calibrate it from the data instead of assuming.

2 TOOL CALLS

```
Bash python3 - <<'PY' p='tools/ctlcheck.py' s=open(p).read() s=s.replace('' # key-map rectangles are in key-map pixels; convert a column offset to # avenue colum...
Bash python3 - <<'PY' import sys;sys.path.insert(0,'tools') from paircrops import keymap km=keymap('1912',warn=False) for u in ('26','47','53','85','94','87','95'...
```

CLAUDE 31 Aug 2026, 23:43 UTC

Sheet 47's rectangle is anomalously wide (590 px vs ~365 for its neighbours), which drags its centre. The rectangles share a common west edge, so the left edge is the right anchor.

4 TOOL CALLS

```
Bash python3 - <<'PY' p='tools/ctlcheck.py' s=open(p).read() s=s.replace('' return ((rects[ub][0] + rects[ub][2]) - (rects[ua][0] + rects[ua][2])) / 2.0'', '' ...
Bash grep -n "return rects\[ub\]\[0\]\[2\]|def kmdx" -A 2 tools/ctlcheck.py | head -20; python3 - <<'PY' import sys;sys.path.insert(0,'tools') from paircrops import k...
Bash sed -n 100,130p tools/ctlcheck.py
Bash python3 - <<'PY' p='tools/ctlcheck.py' s=open(p).read() s=s.replace('' pred = None if ua in rects and ub in rects: dcol = ((rects[ub][0] + rects[ub][2]) - (...)
```

CLAUDE 31 Aug 2026, 23:44 UTC

Now 44 of 46 pass. Let me replace the hand-drawn-rectangle test with a cleaner one built on the key maps' avenue *lists*, which are ground-correct.

1 TOOL CALL

```
Bash python3 - <<'PY' p='tools/ctlcheck.py' s=open(p).read() s=s.replace(''AVENUE_PITCH_PX = 1000.0 # native px between full avenues, measured TOL_BLOCKS = 0.45 ...
```

CLAUDE 31 Aug 2026, 23:45 UTC

Three independent southern sheets settle a geometry assumption I had wrong. Sheet 57: Ave M at 1191.7, Ave M½ at 2199.8 — **1008 px apart**. Sheet 90: Ave S½ at 173.8, Ave T at 1171.5 — 998 px. Sheet 96: Ave P½ to Ave Q — 987 px.

3 TOOL CALLS

```
Bash python3 - <<'PY' import re # 1) avenue_slot must not silently mis-parse 'STRAND OR. AVENUE "B"' p='tools/reciplib.py' s=open(p).read() s=s.replace('' s = st...
```

```
Bash python3 - <<'PY' p='tools/reciplib.py' s=open(p).read() if "\nimport re\n" not in s: s=s.replace("import os\nimport shutil","import os\nimport re\nimport shu...
```

```
Bash python3 - <<'PY' p='tools/reciplib.py' s=open(p).read() s=s.replace('' s = str(avenue).strip().lower().replace("avenue", "").replace("av.", "")'', '' s = ...
```

CLAUDE 31 Aug 2026, 23:45 UTC

Now the pitch correction — the half-avenues are named halves but spaced full lattice steps.

1 TOOL CALL

```
Bash python3 - <<'PY' NEW = ''def ground_index(slot): ""Slots in units of one avenue pitch. This used to halve every slot south of Avenue M, on the assumption t...
```

CLAUDE 31 Aug 2026, 23:45 UTC

45 of 46 controls now pass an independent check. The one flag is explainable — and it's the key map at fault, not the control.

2 TOOL CALLS

```
Bash python3 - <<'PY' import sys;sys.path.insert(0,'tools') from paircrops import keymap from reciplib import Recipe km=keymap('1912',warn=False) r=Recipe(1912) p...
```

```
Bash git add -A tools outputs/1912/recipe/controls state && git commit -q -m "ctlcheck.py: validate controls against evidence they were not fitted to Checks every...
```

CLAUDE 31 Aug 2026, 23:46 UTC

All 15 batches are done and the network is closed. Where this landed:

102 avenue controls, one x-component. All 93 sheets are now rigidly tied in x to the frozen core, down from 26 disconnected components.

45 of 46 accepted controls pass an independent check. `ctlcheck.py` validates each one against evidence it wasn't fitted to: the key maps say which avenues each plate contains, so the framing shift between two plates is a difference of two known integers, and that predicts what the control's native difference must be. A control that took the right corridor on one sheet and its neighbour on the other is off by a whole block — the exact trap the observers hit by hand seven times. The single remaining flag is the key map's fault, not the control's: sheet 85's entry claims five named avenues, which at a 1000 px pitch needs 4,000 px of a 3,327 px plate, so Ave J is over-claimed.

Two corrections to my own work came out of building that check. I had assumed a "½" avenue sits half a block from its neighbour. The plates say otherwise in three independent

places — Ave M to M½ on sheet 57 is 1,008 px, S½ to T on sheet 90 is 998 px, P½ to Q on sheet 96 is 987 px — all full lattice steps, the same ~1,000 px separating Ave C from Ave D downtown. South of Broadway the outlot district names every corridor it has; the halves are naming, not spacing. **That is also why my earlier city-wide avenue fit couldn't be made to work** — I'd been fitting a half-step model to a uniform grid. Separately, `avenue_slot()` silently parsed `STRAND OR. AVENUE "B"` as Avenue S, and raised outright on `Ave M 1/2`, the form the controls actually use.

Next: apply the solve, re-cut the seams, fit the city-wide grid, and regenerate the COG and tiles.

USER 31 Aug 2026, 23:51 UTC

apply the solve

1 TOOL CALL

```
Bash python3 tools/netsolve.py --year 1912 2>&1 | tail -22
```

CLAUDE 31 Aug 2026, 23:51 UTC

Before applying, let me test the thing that failed before: whether the avenue grid comes out monotonic under the new placement.

1 TOOL CALL

```
Bash python3 - <<'PY' import json,os,sys,collections import numpy as np
sys.path.insert(0,'tools') from recipelib import Recipe, px_per_ft import netsolve, gridfit...
```

CLAUDE 31 Aug 2026, 23:52 UTC

The avenue index is monotonic now — the shear is gone. But one row needs checking before I apply: Ave N (slot 14) has a 479 ft scatter across 16 observations, where every other avenue is under 70 ft.

1 TOOL CALL

```
Bash python3 - <<'PY' import json,os,sys,collections import numpy as np
sys.path.insert(0,'tools') from recipelib import Recipe, px_per_ft import netsolve, gridfit...
```

CLAUDE 31 Aug 2026, 23:52 UTC

Found it — a regex bug of mine, not a data problem. `gridfit`'s avenue pattern matches "Ave" *inside* "Avenue K" and then reads the `n` of "nue" as the letter, so **"Avenue K" was being classified as Ave N**.

1 TOOL CALL

```
Bash python3 - <<'PY' p='tools/gridfit.py' s=open(p).read() s=s.replace(''STREET_RE =
re.compile(r"(\d+)\s*(?:st|nd|rd|th)\b", re.I) AVENUE_RE = re.compile(r"...
```

CLAUDE 31 Aug 2026, 23:52 UTC

`'none shared'` also parses as Ave N — harmless today since those controls have null coordinates and are skipped, but too fragile to leave.

1 TOOL CALL

```
Bash python3 - <<'PY' p='tools/gridfit.py' s=open(p).read() s=s.replace(' ' try: return ("avenue", Recipe.avenue_slot(s)) except Exception: return None'', '' # ...
```

CLAUDE 31 Aug 2026, 23:53 UTC

The final batch landed with four more controls — and a caveat that matters: **sheet 99 is two beach-strip panels out of register by ~525 px**, and sheet 85 is a composite with an inset panel drawn ~960 px east of true position. A single affine cannot satisfy both of 99's panels. That also explains sheet 85's impossible five-avenue key map — it's a composite plate.

1 TOOL CALL

```
Bash python3 tools/netsolve.py --year 1912 2>&1 | tail -18 && echo "=== ctlcheck ===" && python3 tools/ctlcheck.py --year 1912 2>&1 | tail -12
```

CLAUDE 31 Aug 2026, 23:53 UTC

All three remaining flags land on exactly the two plates the observer independently flagged as composites — my mechanical check and their visual reading converge. Let me re-verify the grid, then apply.

1 TOOL CALL

```
Bash python3 - <<'PY' import json,os,sys,collections import numpy as np sys.path.insert(0,'tools') from recipelib import Recipe, px_per_ft import netsolve, gridfit...
```

CLAUDE 31 Aug 2026, 23:53 UTC

The grid is clean now: monotonic across all 26 slots, uniform ~2,040 px per avenue (≈ 352 ft, matching the core's 350.4 ft), and worst scatter down from 479 ft to 66 ft. Applying.

2 TOOL CALLS

```
Bash python3 tools/netsolve.py --year 1912 --apply 2>&1 | tail -4 && echo "=== verify ===" && python3 - <<'PY' import json,os p='outputs/1912/recipe/transforms_ci...' Bash python3 tools/gridfit.py --year 1912 2>&1 | tail -12
```

CLAUDE 31 Aug 2026, 23:53 UTC

The shear is out. The avenue fit went from a 329.2 ft residual to **22.8 ft**, and its pitch from a nonsensical 334.6 ft to **348.8 ft** — matching the frozen core's independently measured 350.4 ft.

2 TOOL CALLS

```
Bash git add -A outputs/1912/recipe tools state && git commit -q -m "Apply the control solve: the 1912 row shear is out 204 controls (106 avenue, 98 street), resi... Bash cat > /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/msg.txt <<'MSG' Apply the control solve: the 1912 row shear is out...
```

CLAUDE 31 Aug 2026, 23:54 UTC

The solve is applied and pushed at `4fb1e0e`.

204 controls (106 avenue, 98 street), residuals **median 2.1 ft, 90th 10.0 ft, max 33.6 ft**, with all 81 free sheets constrained and none left sitting where it was. Sheets moved a median of 330 ft and up to 4,271 ft — large moves with small residuals is the shear coming out, not the

fit straining. Verified: the 12 frozen core sheets are byte-identical to the backup, and every applied translation matches the solve exactly.

The test that failed before now passes. The avenue index is monotonic across all 26 observed slots at a uniform ~2,040 mosaic px per slot — 352 ft, against the core's independently measured 350.4 ft. Worst per-avenue scatter is 66 ft, down from 479 ft.

Two parsing bugs of mine surfaced while checking that, and one was material: `gridfit`'s avenue regex matched "Ave" *inside* "Avenue K" and read the `n` of "nue" as the letter, filing every Avenue K control under Ave N. That was the entire source of the 479 ft scatter — not bad data. Both now route through `avenue_slot`, which requires positive evidence of an avenue.

Three controls stay flagged, all on the two plates the observers independently called composites: sheet 85 has an inset panel drawn ~960 px east of true position (also why its key map claims five avenues, needing 4,000 px of a 3,327 px plate), and sheet 99 is two beach-strip panels internally out of register by ~525 px, so no single affine can satisfy both.

Two things are now stale and must not be used as-is: the ownership polygons still describe the old placement, and the published COG/DZI still contain the shear. The seam re-cut is the next step before anything renders correctly — say the word and I'll run it.

2026-09-01

USER 01 Sep 2026, 00:41 UTC

run the seam re-cut

2 TOOL CALLS

```
Bash sed -n 1,40p tools/streetcut.py; echo ...; python3 tools/streetcut.py --help 2>&1 | tail -12
```

```
Bash timeout 1800 python3 tools/streetcut.py --year 1912 2>&1 | tail -20
```

CLAUDE 01 Sep 2026, 00:41 UTC

Two problems: it's only seeing 154 controls (the new `_x` files aren't being read), and the result is in 2 pieces.

4 TOOL CALLS

```
Bash grep -n "pair_\|re.match\|def load\|cuts\[\" tools/streetcut.py | head -20
```

```
Bash sed -n 45,100p tools/streetcut.py
```

```
Bash sed -n 100,150p tools/streetcut.py
```

```
Bash python3 - <<'PY' p='tools/streetcut.py' s=open(p).read() s=s.replace('\"'def load_cuts(r): \"\"\"{(a,b): (axis, mosaic_coord)} from every accepted control.\"\"\" ou...
```

CLAUDE 01 Sep 2026, 00:42 UTC

All 204 controls load now. But 2 pieces and 14 interior rings — let me find what detached.

3 TOOL CALLS

```
Bash python3 - <<'PY' import json,sys import numpy as np from shapely.geometry import Polygon from shapely.ops import unary_union d=json.load(open('outputs/1912/r...
```

```
Bash python3 - <<'PY' import json,sys import numpy as np sys.path.insert(0,'tools') from recipelib import Recipe, px_per_ft from shapely.geometry import Polygon, P...
```

```
Bash python3 tools/tiling.py --help 2>&1 | tail -8
```

CLAUDE 01 Sep 2026, 00:43 UTC

Let me get the "before" numbers first, then apply and compare.

2 TOOL CALLS

```
Bash timeout 1200 python3 tools/tiling.py --year 1912 2>&1 | tail -18
Bash python3 - <<'PY' import json,sys import numpy as np sys.path.insert(0,'tools') from
reciplib import Recipe from shapely.geometry import Polygon, Point r=Reci...
```

CLAUDE 01 Sep 2026, 00:43 UTC

Both holes sit *inside* a sheet's footprint (85 and 69) — so this is over-trimming by the 143 bisector fallbacks, not missing coverage. Now that the city-wide grid is sound, I can snap those cuts to real centrelines instead.

2 TOOL CALLS

```
Bash python3 - <<'PY' p='tools/streetcut.py' s=open(p).read() s=s.replace(''def
half_plane(axis, coord, keep_low):'', ''def grid_lines(r): ""{axis: sorted mos...
Bash python3 - <<'PY' p='tools/streetcut.py' s=open(p).read() s=s.replace('' lines =
grid_lines(r) print(f"grid corridors available:'', '' lines = grid_lines(r)...
```

CLAUDE 01 Sep 2026, 00:44 UTC

Every seam is on a centreline now, but snapping fragmented the mosaic into 8 pieces — it can push a cut past a small sheet entirely. Constraining the snap to lie between the two sheet centres should fix that.

2 TOOL CALLS

```
Bash python3 - <<'PY' p='tools/streetcut.py' s=open(p).read() s=s.replace(''def snap(lines,
axis, coord, limit): ""Nearest corridor on this axis, if one is with...
Bash python3 - <<'PY' import json from shapely.geometry import Polygon from shapely.ops import
unary_union d=json.load(open('outputs/1912/recipe/seams/ownership_s...
```

CLAUDE 01 Sep 2026, 00:44 UTC

All the detached groups are the east-end sheets (17–32). A snap can move a seam up to half a block, which trims a sheet away from a *different* neighbour's seam. Let me make the limit tunable and find the value that keeps the mosaic whole.

1 TOOL CALL

```
Bash python3 - <<'PY' p='tools/streetcut.py' s=open(p).read() s=s.replace('' ap.add_argument("-
-apply", action="store_true")'', '' ap.add_argument("--apply", a...
```

CLAUDE 01 Sep 2026, 00:45 UTC

Snapping is a net negative — it fragments the mosaic at every setting. Let me default it off, keep it documented as a negative result, and check what's detached in the clean run.

2 TOOL CALLS

```
Bash python3 - <<'PY' p='tools/streetcut.py' s=open(p).read() s=s.replace(''SNAP_LIMIT_BLOCKS =
0.5 # how far a seam may move to reach a centreline'', ''# Snap...
Bash timeout 1800 python3 tools/streetcut.py --year 1912 --apply 2>&1 | tail -2 && echo "===
tiling ===" && timeout 1200 python3 tools/tiling.py --year 1912 2>&1 ...
```

CLAUDE 01 Sep 2026, 00:45 UTC

One piece, zero overlap. Now the 15 holes — most sit inside a sheet that can claim them.

1 TOOL CALL

```
Bash python3 tools/fillgaps.py --help 2>&1 | tail -8; echo ---; timeout 1200 python3 tools/fillgaps.py --year 1912 2>&1 | tail -25
```

CLAUDE 01 Sep 2026, 00:45 UTC

`fillgaps` calls the two biggest "true source gaps", but I measured earlier that both sit inside a sheet's footprint. Let me check the actual coverage fractions.

2 TOOL CALLS

```
Bash python3 - <<'PY' import json,sys import numpy as np sys.path.insert(0,'tools') from recipelib import Recipe, px_per_ft from shapely.geometry import Polygon fr...
Bash python3 - <<'PY' import json,sys import numpy as np sys.path.insert(0,'tools') from recipelib import Recipe, px_per_ft from shapely.geometry import Polygon fr...
```

CLAUDE 01 Sep 2026, 00:46 UTC

I need to correct something I said a moment ago. I claimed both big holes sat inside a sheet's footprint and were over-trimming — that was wrong. I'd tested only whether the hole's *centroid* fell inside a footprint. Measuring the actual polygons: the two largest holes are only **25% and 37% covered by any sheet**, so `fillgaps` is right to call them genuine source gaps.

1 TOOL CALL

```
Bash timeout 1200 python3 tools/fillgaps.py --year 1912 --apply 2>&1 | tail -4 && echo "=== final tiling ===" && timeout 1200 python3 tools/tiling.py --year 1912 ...
```

CLAUDE 01 Sep 2026, 00:46 UTC

One connected piece, zero overlap, 8 holes. Now the direct visual test — Avenue M across the 57|63 row boundary, the exact seam that exposed the shear.

2 TOOL CALLS

```
Bash timeout 900 python3 tools/crop.py --year 1912 --cx 17367 --cy 17486 --width-in 9 --height-in 7 --scale-ft-per-in 200 --dpi 150 --out /tmp/claude-0/-home-user...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/seam5763.jpg
```

CLAUDE 01 Sep 2026, 00:47 UTC

The shear is visibly gone — Avenue M, M¹/₂ and K run continuously straight through the 30th St seam. Let me zoom the top-right, where some plate-margin text appears to be bleeding in.

2 TOOL CALLS

```
Bash python3 -c " import cv2 im=cv2.imread('/tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/seam5763.jpg') c=im[0:200,1120:13...
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-a56890968b0c/scratchpad/zoomtr.jpg
```

CLAUDE 01 Sep 2026, 00:48 UTC

Confirmed, and it's a real defect: the grey vertical band is sheet 57's **printed plate margin** — border rule and the "57" plate number — composited into the mosaic. The seam there is

correctly on Avenue N's centreline (21,536 vs the grid's 21,536), but each plate's drawn margin sits right at that corridor, so a centreline cut necessarily pulls one sheet's margin in.

1 TOOL CALL

```
Bash cat >> HUMAN_QUEUE.md <<'MD' --- ## HQ-19 · 1912 seams re-cut after the shear fix – APPLIED, with two defects left `streetcut.py --apply` then `fillgaps.py`...
```

CLAUDE 01 Sep 2026, 00:48 UTC

Seam re-cut applied and pushed at 20b97be.

Gate result: 93 regions, one connected piece, 0 px² overlap, 8 interior holes. The direct test passes — a crop across the 57|63 seam, the pair whose 1,643 ft disagreement exposed the shear, now runs Avenue K, M and M½ straight through 30th St with no step.

Loading the controls needed a fix first. `streetcut` was reading 154 of the 204 controls — its filename regex predated the `_x` files. Fixing that exposed a worse bug: a stacked pair now carries *two* controls (the street it abuts along, the avenue it crosses), and keying by pair alone let the second overwrite the first. Since a stacked pair's avenue control is on the wrong axis to cut with, the existing guard then discarded it and fell back to a bisector — silently trading a good street cut for a bad one. Cuts are keyed by pair *and* axis now.

One negative result worth the measurement. 143 seams have no control and use a bisector between sheet centres — an arbitrary line that can run through a building, which \$2.5 forbids. With the city grid now solved, snapping those to the nearest centreline should have been a clear win. It wasn't: 0.1 blocks gave 3 disjoint pieces, 0.2 and 0.3 gave 8, against 2 with plain bisectors. A seam that moves to reach a corridor trims its sheet away from a *different* neighbour's seam, and the east-end sheets come apart. Kept behind `--snap-blocks`, defaulted off, numbers in the docstring.

Two defects recorded as HQ-19 rather than smoothed over:

- **Plate margins bleed in.** At the 57|58 seam the mosaic carries sheet 57's printed border rule and plate number. The cut is correct — Avenue N's centreline at x=21,536 against the grid's 21,536 — but each plate's drawn margin sits exactly at the corridor it gets cut on. The fix is trimming each sheet's extent to the neat line; the extents are untouched by this pass, so the mechanism predates it.
- **The two largest holes are genuine source gaps, not over-trimming** — correcting what I told you earlier. Measured properly they're only 25% and 37% covered by any sheet.

The COG and DZI are still the sheared versions. Regenerating them is the next step.

USER 01 Sep 2026, 01:03 UTC

regenerate the COG and tiles

3 TOOL CALLS

```
Bash df -h /home/user 2>/dev/null | tail -2; echo ---; du -sh work outputs 2>/dev/null; echo --  
-; python3 tools/publish.py --help 2>&1 | tail -14  
Bash ls -la outputs/1912/publish 2>/dev/null | head; echo ---; python3 - <<'PY' import json,sys  
import numpy as np sys.path.insert(0,'tools') from recipelib import...  
Bash grep -n "warpAffine\|np.full\|copyTo\|def main\|downsample\|canvas" tools/render.py | head  
-25
```

CLAUDE 01 Sep 2026, 01:04 UTC

Rendering warps each sheet to the *full* canvas — a 4 GB temporary per sheet. Let me bound the warp to each sheet's own footprint first; that makes the gigapixel render safe rather than marginal.

5 TOOL CALLS

```
Bash sed -n 96,125p tools/render.py  
Bash python3 - <<'PY' p='tools/render.py' s=open(p).read() s=s.replace('' M, t =  
r.sheet_matrix(sheet) A = np.hstack([M / d, ((t - np.array([x0, y0])) / d).resha...  
Bash python3 - <<'PY' p='tools/render.py' s=open(p).read() s=s.replace('' sub_cov =  
covered[wy0:wy1, wx0:wx1] mask &= cv2.inRange(sub_cov, 0, 0) # in-place maske...  
Bash python3 -c " import cv2 im=cv2.imread('/tmp/claude-0/-home-user-claude-code/20717cf2-d758-  
5a36-9da9-a56890968b0c/scratchpad/city16.tif') print(im.shape) h,w=...  
Read /tmp/claude-0/-home-user-claude-code/20717cf2-d758-5a36-9da9-  
a56890968b0c/scratchpad/city16.jpg
```

CLAUDE 01 Sep 2026, 01:05 UTC

The city-wide mosaic is coherent now — continuous grid, rows aligned, dense downtown core in place. Running the full publish.

5 TOOL CALLS

```
Bash sed -n 1,60p tools/publish.py  
Bash rm -f work/city/*.tif; find outputs/1912/tiles -mindepth 1 -delete 2>/dev/null; rm -f  
outputs/1912/mosaic/*.tif; df -h /home/user | tail -1 nohup python3 too...  
Bash sleep 240; tail -4 work/publish1912.log; free -g | sed -n 2p; df -h /home/user | tail -1  
ToolSearch select:Monitor  
Monitor tail -f /home/user/claude-code/work/publish1912.log | grep -E --line-buffered  
"^\\+|wrote|COG|dzsave|Error|error|Traceback|failed|Killed|OOM|MemoryError|MB|le...
```

CLAUDE 01 Sep 2026, 01:06 UTC

While that runs, let me apply the city-wide grid — the original "extend the grid" request, now that the fit is sound.

2 TOOL CALLS

```
Bash python3 tools/gridfit.py --year 1912 --apply 2>&1 | tail -10  
Bash python3 - <<'PY' import sys; sys.path.insert(0,'tools') from recipelib import Recipe  
r=Recipe(1912) g=r.grid print("streets indexed:",len(g['streets']),sorted...
```

CLAUDE 01 Sep 2026, 01:06 UTC

34th & Ave P works now — that's the lookup that used to `KeyError`. But the street index stops at 42, and sheets go further. Let me take the span from the key maps rather than from whichever streets happened to get controls.

3 TOOL CALLS

```
Bash python3 - <<'PY' import sys,re;sys.path.insert(0,'tools') from paircrops import keymap
km=keymap('1912',warn=False) vals=[] for u,e in km.items(): for v in e...
Bash python3 - <<'PY' p='tools/gridfit.py' s=open(p).read() s=s.replace(''from netsolve import
load_controls # noqa: E402'', ''from netsolve import load_contro...
Bash python3 - <<'PY' import sys,re,json;sys.path.insert(0,'tools') import numpy as np from
paircrops import keymap from recipelib import Recipe from shapely.geome...
```

CLAUDE 01 Sep 2026, 01:07 UTC

Sheet 1 is the index/key sheet, not a placed unit — including it inflated the span to street 49, past where any mosaic pixel exists. Restricting to placed units:

1 TOOL CALL

```
Bash python3 - <<'PY' p='tools/gridfit.py' s=open(p).read() s=s.replace('' km_streets = [] for
e in keymap(str(r.year), warn=False).values(): for v in e.get("str...
```

CLAUDE 01 Sep 2026, 01:07 UTC

`gridfit` isn't idempotent — it re-ingests its own fitted output as if it were verified core. Let me restore the original grid from git and fix that.

3 TOOL CALLS

```
Bash git status --porcelain outputs/1912/recipe/grid.json; git checkout --
outputs/1912/recipe/grid.json && python3 -c " import json;d=json.load(open('outputs/191...
Bash python3 - <<'PY' p='tools/gridfit.py' s=open(p).read() s=s.replace('' core = {k: v for k,
v in old.get(section, {}).items()}}'', '' # Only the VERIFIED ent...
Bash python3 - <<'PY' import sys;sys.path.insert(0,'tools') from recipelib import Recipe
r=Recipe(1912) for st,av in ((34,'P'),(45,'T 1/2'),(7,'A'),(22,'E'),(30,'M...
```

CLAUDE 01 Sep 2026, 01:08 UTC

Compositing finished at 5 GB peak — the windowed warp held it well under the previous 10 GB despite a 47% larger canvas. Committing the grid work while the TIFF writes.

1 TOOL CALL

```
Bash git add tools outputs/1912/recipe/grid.json outputs/1912/recipe/grid_city.json && git
commit -q -F - <<'MSG' && git push -q -u origin claudie/galveston-setup...
```

CLAUDE 01 Sep 2026, 01:08 UTC

The grid is live city-wide — streets 7–46 and all 28 avenue slots, so `34th & Ave P` resolves instead of raising `KeyError: '18'`. That closes HQ-17.

Three things had to be fixed to get there. The street span now comes from the sheets' key maps rather than from whichever streets happened to get a control (controls reach 9–42; plates reach 7–46) — but **sheet 1 is the index plate, not a placed unit**, and its "43RD–49TH" was stretching the grid seven streets past the last mosaic pixel. And `gridfit` wasn't idempotent: it treated every entry already in `grid.json` as a verified anchor, so a second run re-ingested its own fitted output and relabelled it "core (verified frontage midlines)". Two consecutive applies now produce byte-identical output.

Waiting on the COG and DZI build.

STOP HOOK 01 Sep 2026, 01:08 UTC

Stop hook feedback:

[~/claude/stop-hook-git-check.sh]: There are uncommitted changes in the repository. Please commit and push these changes to the remote branch.

3 TOOL CALLS

```
Bash git status --porcelain | head -20; echo "=== publish ==="; tail -6 work/publish1912.log; ls -la outputs/1912/mosaic outputs/1912/tiles 2>/dev/null | head -12
```

```
Bash cat >> .gitignore <<'EOF' # gridfit/streetcut/netsolve keep a .pre_* backup of what they replaced; # git already holds that history *.pre_city *.pre_recut *....
```

```
Bash ls -la work/city/ 2>/dev/null; ps aux | grep -c "[r]ender.py"; free -g | sed -n 2p
```

CLAUDE 01 Sep 2026, 01:08 UTC

Render done in 159s at 1,223 MB; COG conversion running. Let me bring STATUS.md up to date while it works.

3 TOOL CALLS

```
Bash python3 - <<'PY' import re p='STATUS.md' s=open(p).read() s=s.replace("**Updated:** 2026-08-29 (Stage 4 partial: 1912 tools live)", "**Updated:** 2026-09-01 ...
```

```
Bash tail -8 work/publish1912.log; ls -la outputs/1912/mosaic/ 2>/dev/null | tail -2; df -h /home/user | tail -1
```

```
Bash git add STATUS.md state && git commit -q -m "STATUS: the shear, the fix, and what it measured Co-Authored-By: Claude Opus 5 <noreply@anthropic.com> Claude-Se...
```

CLAUDE 01 Sep 2026, 01:09 UTC

COG conversion is running (input 29,653 × 45,816). I'll report when the COG and DZI finish; the monitor is watching the build.

BACKGROUND TASK 01 Sep 2026, 01:09 UTC

bx722w19x

Monitor event: "1912 COG and DZI publish progress"

[69s]

COG: 1096 MB

+ vips dzsave /home/user/claude-code/work/city/1912_150ppi.tif /home/user/claude-code/outputs/1912/tiles/1912 --suffix .jpg[Q=85]

CLAUDE 01 Sep 2026, 01:09 UTC

COG done: 1,096 MB in 69s. Verifying it while the tile pyramid builds.

1 TOOL CALL

```
Bash gdalinfo outputs/1912/mosaic/1912_fullcity_150ppi.tif 2>&1 | sed -n '1,12p;/0verviews/p' | head -20; echo "=== COG validity ==="; python3 -c "import subprocess...
```

CLAUDE 01 Sep 2026, 01:10 UTC

The COG is valid — `LAYOUT=COG`, 512×512 blocks, 7 overview levels, 1,359 MP. But the resolution tag is wrong: it claims **25.4 ppi** where this is a 150 ppi render. That's pyvips' default of 1 px/mm passed through untagged.

1 TOOL CALL

```
Bash sed -n 28,52p tools/render.py; echo "=== publish call ==="; grep -n "render.py\\|ppi" tools/publish.py | head
```

CLAUDE 01 Sep 2026, 01:10 UTC

DZI: 192 MB in 25s — that's fast enough to be worth verifying.

1 TOOL CALL

```
Bash cat outputs/1912/tiles/1912.dzi 2>/dev/null; echo; ls outputs/1912/tiles/1912_files | sort -n | tail -3; echo "levels: $(ls outputs/1912/tiles/1912_files | w...
```

CLAUDE 01 Sep 2026, 01:10 UTC

DZI verified: 18 levels, 28,408 tiles, correct dimensions. Now the resolution tag.

6 TOOL CALLS

```
Bash python3 -c " import pyvips
print('pyvips',pyvips.__version__, 'libvips',pyvips.base.version(0),pyvips.base.version(1),pyvips
.base.version(2)) import re h=pyvi...
Bash python3 - <<'PY' import pyvips, numpy as np, subprocess, os
a=np.full((64,64,3),255,np.uint8) v=pyvips.Image.new_from_memory(a.data,64,64,3,"uchar")
ppi=150...
Bash python3 - <<'PY' p='tools/render.py' s=open(p).read() s=s.replace(''def save(canvas, out):
""Write the BGR canvas as a tiled, pyramidal TIFF. Streams throu...
Bash rm -f work/city/1912_150ppi.tif && nohup python3 tools/publish.py --year 1912 --downsample
2 > work/publish1912.log 2>&l & echo started
Monitor tail -f /home/user/claude-code/work/publish1912.log | grep -E --line-buffered
"^\\+|wrote|COG:|DZI:|flat render|Error|error|Traceback|failed|Killed|OOM|Memory...
Bash git add tools state && git commit -q -F - <<'MSG' && git push -q -u origin
claude/galveston-setup-part-a-mk5z1l && git log --oneline -1 Record the real resol...
```

CLAUDE 01 Sep 2026, 01:11 UTC

The rebuild is running with the corrected resolution. I'll report the final numbers when it completes.

BACKGROUND TASK 01 Sep 2026, 01:13 UTC

b61ykmk89

Monitor event: "1912 re-publish with resolution tags"

wrote /home/user/claude-code/work/city/1912_150ppi.tif

flat render: 1223 MB

```
+ gdal_translate -of COG -co COMPRESS=DEFLATE -co PREDICTOR=2 -co BLOCKSIZE=512 -co
OVERVIEWS=IGNORE_EXISTING -co NUM_THREADS=ALL_CPUS /home/user/claude-
code/work/city/1912_150ppi.tif /home/user/claude-code/outputs/1912/mosaic/1912_fullcity_150ppi.tif
```

CLAUDE 01 Sep 2026, 01:13 UTC

Render complete; COG conversion running again.

1 TOOL CALL

```
Bash python3 -c " import subprocess,json r=subprocess.run(['gdalinfo','-
json','/home/user/claude-code/work/city/1912_150ppi.tif'],capture_output=True,text=True) d...
```

CLAUDE 01 Sep 2026, 01:13 UTC

The flat render now carries `TIFFTAG_XRESOLUTION=150`. Waiting on the COG and DZI to finish.

STOP HOOK 01 Sep 2026, 01:13 UTC

Stop hook feedback:

[~/claude/stop-hook-git-check.sh]: There are uncommitted changes in the repository. Please commit and push these changes to the remote branch.

2 TOOL CALLS

Bash git status --porcelain | head; tail -4 work/publish1912.log

Bash git add state && git commit -q -m "ledger: subagent events from the cross-row control batches Co-Authored-By: Claude Opus 5 <noreply@anthropic.com> Claude-Se..."

CLAUDE 01 Sep 2026, 01:14 UTC

Working tree clean. The COG conversion is still running (it took 69s last time, plus ~25s for the tiles) — I'll report the verified final artifacts when the monitor signals completion.

USER 01 Sep 2026, 01:14 UTC

show me the finished map

3 TOOL CALLS

Bash tail -8 work/publish1912.log; ls -la outputs/1912/mosaic/ | tail -2; echo "tiles: \$(find outputs/1912/tiles/1912_files -name '*.jpg' 2>/dev/null | wc -l)"

Bash python3 - <<'PY' import pyvips, os src='work/city/1912_150ppi.tif' out='outputs/1912/preview' os.makedirs(out, exist_ok=True) im=pyvips.Image.new_from_file(s...

Read /home/user/claude-code/outputs/1912/preview/1912_fullcity.jpg

CLAUDE 01 Sep 2026, 01:15 UTC

The finished city. Let me pull two details at full resolution — the downtown core, and the seam that exposed the shear.

6 TOOL CALLS

Bash python3 - <<'PY' import pyvips, os, json, sys sys.path.insert(0,'tools') from recipelib import Recipe from shapely.geometry import Polygon from shapely.ops im...

Read /home/user/claude-code/outputs/1912/preview/1912_downtown.jpg

Read /home/user/claude-code/outputs/1912/preview/1912_seam_57_63.jpg

Bash tail -12 work/publish1912.log | grep -v "^0\\.\\.\\."; echo "=== verify ==="; python3 -c "import subprocess,json d=json.loads(subprocess.run(['gdalinfo','-json...)

Bash cat >> .gitignore <<'EOF' # the COG is over GitHub's 100 MB file cap; rebuild with tools/publish.py outputs/*/mosaic/*.tif outputs/*/tiles/ EOF git add -A .g...

SendUserFile

CLAUDE 01 Sep 2026, 01:16 UTC

Here's the finished 1912 map — all 93 sheets, one continuous mosaic.

The artifacts, both verified to open:

- **COG** 29,653 × 45,816 (1,359 MP), `LAYOUT=COG`, deflate/predictor 2, 512 px blocks, 7 overview levels, 1,096 MB — now correctly tagged 150 ppi.

- **DZI** 18 levels, 28,408 tiles, 242 MB.

The published mosaic is **47% wider than the last one**. That's the headline: the shear had been folding the rows over each other, so the city was rendering narrower than it actually is.

The downtown detail shows what the lot-level quality buys you — the Crystal Majestic Theatre, the Galveston Tribune press room, Fellman Dry Goods, the Scottish Rite Cathedral, each with its street numbers, wall construction and hydrants.

The seam detail is the honest one. Avenue M½ and its 8" water main line up across the 57|63 seam to within a few pixels — that's the shear fix working, on the exact pair that was 1,643 ft out. It also shows HQ-19's margin bleed plainly: sheet 63's top margin, carrying its plate number and north arrow, composites as a white band straight through 30th St.

What's still open on 1912, all recorded in `HUMAN_QUEUE.md` :

- **Margin bleed** (HQ-19) — each plate's drawn margin sits exactly at the corridor it gets cut on. Fixing it means trimming each sheet's extent to the neat line.
- **8 holes** — the two largest are genuine source gaps (only 25% and 37% of that ground is on any plate); four of the rest are 82–100% covered but by a *union* of sheets, which `fillgaps` won't assign under its single-neighbour rule.
- **The detector caveat** (HQ-18) — the original 154 controls were proposed by a detector that finds alleys, not avenues. A width-test pass over them hasn't been done.

Since the COG is over GitHub's 100 MB cap and the tile tree is 242 MB, both are gitignored and rebuild with `python3 tools/publish.py --year 1912`. The previews are committed.

BACKGROUND TASK 01 Sep 2026, 01:36 UTC

bx722w19x

Monitor event: "1912 COG and DZI publish progress"

[Monitor timed out — re-arm if needed.]

CLAUDE 01 Sep 2026, 01:36 UTC

The publish finished, so that monitor timing out is expected — nothing to re-arm. Let me stop the second one, which is watching the same completed build.

4 TOOL CALLS

ToolSearch select:TaskStop,TaskList

TaskStop

Bash git status --porcelain; git log --oneline -1; echo "=== artifacts ==="; ls -la outputs/1912/mosaic/*.tif 2>/dev/null | awk '{print \$5, \$9}'; echo "tiles: \$(f...

Bash git add state && git commit -q -m "ledger: 1912 republish events Co-Authored-By: Claude Opus 5 <noreply@anthropic.com> Claude-Session: https://claude.ai/code...

CLAUDE 01 Sep 2026, 01:36 UTC

Both monitors are stopped and the working tree is clean at `dd26ee7`, everything pushed to `claude/galveston-setup-part-a-mk5z1l`.

The 1912 map is finished and published: COG at 1,359 MP (1,096 MB) and a DZI pyramid of 28,408 tiles across 18 levels, both rebuilt from the corrected placement and verified to open.

To recap what this pass actually changed, since it started as "extend the grid city-wide": the grid couldn't be extended because the mosaic was sheared east-west row to row. Fixing that

took 50 new avenue controls from 15 Opus observers, which collapsed 26 independent x-components to 1, made the avenue index monotonic for the first time, and cut the city-wide avenue fit residual from 329 ft to 23 ft. The grid extension that prompted it now covers streets 7-46 and all 28 avenue slots.

Three open items remain on 1912, all in `HUMAN_QUEUE.md` — margin bleed at centreline cuts, 8 remaining holes, and the un-audited original 154 controls. The 1899 map is still where we left it, with `71a` outstanding.

USER 01 Sep 2026, 17:09 UTC

Can you put this whole conversation is a pdf?

1 TOOL CALL

Skill pdf

USER 01 Sep 2026, 17:09 UTC

Base directory for this skill: `/root/.claude/skills/synced/9eef7a55-9371-4638-a7bd-941f43b27ed3_b2e64b2d-350e-4f90-bc81-05cf72911ab0/pdf`

PDF Processing Guide

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `REFERENCE.md`. If you need to fill out a PDF form, read `FORMS.md` and follow its instructions.

Quick Start

```
from pypdf import PdfReader, PdfWriter

# Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")

# Extract text
text = ""
for page in reader.pages:
    text += page.extract_text()
```

Python Libraries

pypdf - Basic Operations

Merge PDFs

```
from pypdf import PdfWriter, PdfReader

writer = PdfWriter()
for pdf_file in ["doc1.pdf", "doc2.pdf", "doc3.pdf"]:
    reader = PdfReader(pdf_file)
    for page in reader.pages:
        writer.add_page(page)

with open("merged.pdf", "wb") as output:
    writer.write(output)
```

Split PDF

```
reader = PdfReader("input.pdf")
for i, page in enumerate(reader.pages):
    writer = PdfWriter()
    writer.add_page(page)
    with open(f"page_{i+1}.pdf", "wb") as output:
        writer.write(output)
```

Extract Metadata

```
reader = PdfReader("document.pdf")
meta = reader.metadata
print(f"Title: {meta.title}")
print(f"Author: {meta.author}")
print(f"Subject: {meta.subject}")
print(f"Creator: {meta.creator}")
```

Rotate Pages

```
reader = PdfReader("input.pdf")
writer = PdfWriter()

page = reader.pages[0]
page.rotate(90) # Rotate 90 degrees clockwise
writer.add_page(page)

with open("rotated.pdf", "wb") as output:
    writer.write(output)
```

pdfplumber - Text and Table Extraction

Extract Text with Layout

```
import pdfplumber

with pdfplumber.open("document.pdf") as pdf:
    for page in pdf.pages:
        text = page.extract_text()
        print(text)
```

Extract Tables

```
with pdfplumber.open("document.pdf") as pdf:
    for i, page in enumerate(pdf.pages):
        tables = page.extract_tables()
        for j, table in enumerate(tables):
            print(f"Table {j+1} on page {i+1}:")
            for row in table:
                print(row)
```

Advanced Table Extraction

```
import pandas as pd

with pdfplumber.open("document.pdf") as pdf:
    all_tables = []
    for page in pdf.pages:
        tables = page.extract_tables()
        for table in tables:
            if table: # Check if table is not empty
                df = pd.DataFrame(table[1:], columns=table[0])
                all_tables.append(df)

# Combine all tables
if all_tables:
    combined_df = pd.concat(all_tables, ignore_index=True)
    combined_df.to_excel("extracted_tables.xlsx", index=False)
```

reportlab - Create PDFs

Basic PDF Creation

```

from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas

c = canvas.Canvas("hello.pdf", pagesize=letter)
width, height = letter

# Add text
c.drawString(100, height - 100, "Hello World!")
c.drawString(100, height - 120, "This is a PDF created with reportlab")

# Add a line
c.line(100, height - 140, 400, height - 140)

# Save
c.save()

```

Create PDF with Multiple Pages

```

from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak
from reportlab.lib.styles import getSampleStyleSheet

doc = SimpleDocTemplate("report.pdf", pagesize=letter)
styles = getSampleStyleSheet()
story = []

# Add content
title = Paragraph("Report Title", styles['Title'])
story.append(title)
story.append(Spacer(1, 12))

body = Paragraph("This is the body of the report. " * 20, styles['Normal'])
story.append(body)
story.append(PageBreak())

# Page 2
story.append(Paragraph("Page 2", styles['Heading1']))
story.append(Paragraph("Content for page 2", styles['Normal']))

# Build PDF
doc.build(story)

```

Subscripts and Superscripts

IMPORTANT: Never use Unicode subscript/superscript characters (₀₁₂₃₄₅₆₇₈₉, ⁰¹²³⁴⁵⁶⁷⁸⁹) in ReportLab PDFs. The built-in fonts do not include these glyphs, causing them to render as solid black boxes.

Instead, use ReportLab's XML markup tags in Paragraph objects:

```
from reportlab.platypus import Paragraph
from reportlab.lib.styles import getSampleStyleSheet

styles = getSampleStyleSheet()

# Subscripts: use <sub> tag
chemical = Paragraph("H<sub>2</sub>O", styles['Normal'])

# Superscripts: use <sup> tag
squared = Paragraph("x<sup>2</sup> + y<sup>2</sup>", styles['Normal'])
```

For canvas-drawn text (not Paragraph objects), manually adjust font the size and position rather than using Unicode subscripts/superscripts.

Command-Line Tools

pdftotext (poppler-utils)

```
# Extract text
pdftotext input.pdf output.txt

# Extract text preserving layout
pdftotext -layout input.pdf output.txt

# Extract specific pages
pdftotext -f 1 -l 5 input.pdf output.txt # Pages 1-5
```

qpdf


```
# Merge PDFs
qpdf --empty --pages file1.pdf file2.pdf -- merged.pdf

# Split pages
qpdf input.pdf --pages . 1-5 -- pages1-5.pdf
qpdf input.pdf --pages . 6-10 -- pages6-10.pdf

# Rotate pages
qpdf input.pdf output.pdf --rotate=+90:1 # Rotate page 1 by 90 degrees

# Remove password
qpdf --password=mypassword --decrypt encrypted.pdf decrypted.pdf
```

pdftk (if available)

```
# Merge
pdftk file1.pdf file2.pdf cat output merged.pdf

# Split
pdftk input.pdf burst

# Rotate
pdftk input.pdf rotate least output rotated.pdf
```

Common Tasks

Extract Text from Scanned PDFs

```
# Requires: pip install pytesseract pdf2image
import pytesseract
from pdf2image import convert_from_path

# Convert PDF to images
images = convert_from_path('scanned.pdf')

# OCR each page
text = ""
for i, image in enumerate(images):
    text += f"Page {i+1}:\n"
    text += pytesseract.image_to_string(image)
    text += "\n\n"

print(text)
```

Add Watermark

```
from pypdf import PdfReader, PdfWriter

# Create watermark (or load existing)
watermark = PdfReader("watermark.pdf").pages[0]

# Apply to all pages
reader = PdfReader("document.pdf")
writer = PdfWriter()

for page in reader.pages:
    page.merge_page(watermark)
    writer.add_page(page)

with open("watermarked.pdf", "wb") as output:
    writer.write(output)
```

Extract Images

```
# Using pdftimages (poppler-utils)
pdftimages -j input.pdf output_prefix

# This extracts all images as output_prefix-000.jpg, output_prefix-001.jpg, etc.
```

Password Protection

```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("input.pdf")
writer = PdfWriter()

for page in reader.pages:
    writer.add_page(page)

# Add password
writer.encrypt("userpassword", "ownerpassword")

with open("encrypted.pdf", "wb") as output:
    writer.write(output)
```

Quick Reference

Task	Best Tool	Command/Code
Merge PDFs	pypdf	<code>writer.add_page(page)</code>
Split PDFs	pypdf	One page per file
Extract text	pdfplumber	<code>page.extract_text()</code>
Extract tables	pdfplumber	<code>page.extract_tables()</code>
Create PDFs	reportlab	Canvas or Platypus
Command line merge	qpdf	<code>qpdf --empty --pages ...</code>
OCR scanned PDFs	pytesseract	Convert to image first
Fill PDF forms	pdf-lib or pypdf (see FORMS.md)	See FORMS.md

Next Steps

- For advanced pypdfium2 usage, see REFERENCE.md
- For JavaScript libraries (pdf-lib), see REFERENCE.md
- If you need to fill out a PDF form, follow the instructions in FORMS.md
- For troubleshooting guides, see REFERENCE.md

6 TOOL CALLS

```
Bash ls -la /root/.claude/projects/-home-user-claude-code/ 2>/dev/null | head -20; echo "---";
wc -l /root/.claude/projects/-home-user-claude-code/*.jsonl 2>/dev/...
Bash python3 - <<'PY' import json, collections p='/root/.claude/projects/-home-user-claude-
code/20717cf2-d758-5a36-9da9-a56890968b0c.jsonl' types=collections.Coun...
Bash python3 - <<'PY' import json p='/root/.claude/projects/-home-user-claude-code/20717cf2-
d758-5a36-9da9-a56890968b0c.jsonl' real_user=[]; asst_text=[]; tool_ca...
Bash ls /opt/pw-browsers/ 2>/dev/null; which chromium chromium-browser google-chrome wkhtmltopdf
weasyprint pandoc 2>/dev/null; python3 -c "import weasyprint;prin...
Bash ls /opt/pw-browsers/chromium*/ 2>/dev/null | head; find /opt/pw-browsers -maxdepth 3 -name
"*chrome*" -o -maxdepth 3 -name "*headless*" 2>/dev/null | head
Bash python3 -c "import markdown;print('markdown',markdown.__version__)" 2>&l|tail -1; python3 -
c "import mistune;print('mistune',mistune.__version__)" 2>&l|tail ...
```

CLAUDE 01 Sep 2026, 17:10 UTC

markdown and Chromium are both available. Building the transcript.

1 TOOL CALL

```
Bash mkdir -p outputs/transcript && cat > tools/transcript.py <<'PY' #!/usr/bin/env python3
"""Render this project's Claude Code session transcript as a PDF. pyth...
```